DeepSeek-V4:

DeepSeek-V4:

Towards Highly Efficient Million-Token Context Intelligence

面向高效百万标记上下文智能

DeepSeek-AI

DeepSeek-AI

research@deepseek.com

research@deepseek.com

Abstract

摘要

We present a preview version of DeepSeek-V4 series, including two strong Mixture-of-Experts (MoE) language models — DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated) — both supporting a context length of one million tokens. DeepSeek-V4 series incorporate several key upgrades in architecture and optimization: (1) a hybrid attention architecture that combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) to improve long-context efficiency; (2) Manifold-Constrained Hyper-Connections (mHC) that enhance conventional residual connections; (3) and the Muon optimizer for faster convergence and greater training stability. We pre-train both models on more than 32T diverse and high-quality tokens, followed by a comprehensive post-training pipeline that unlocks and further enhances their capabilities. DeepSeek-V4-ProMax, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models, outperforming its predecessors in core tasks. Meanwhile, DeepSeek-V4 series are highly efficient in long-context scenarios. In the one-million-token context setting, DeepSeek-V4-Pro requires only 27% of single-token inference FLOPs and 10% of KV cache compared with DeepSeek-V3.2. This enables us to routinely support one-million-token contexts, thereby making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at https://huggingface.co/collections/deepseek-ai/deepseek-v4.

我们推出了 DeepSeek-V4 系列的预览版本，包括两个强大的专家混合（Mixture-of-Experts，MoE）语言模型——拥有 1.6 万亿参数（490 亿激活参数）的 DeepSeek-V4-Pro 和拥有 2840 亿参数（130 亿激活参数）的 DeepSeek-V4-Flash，两者均支持最多一百万 token 的上下文长度。DeepSeek-V4 系列在架构和优化方面进行了多项关键升级：（1）一种混合注意力架构，结合了压缩稀疏注意力（CSA）和高度压缩注意力（HCA），以提高长上下文效率；（2）流形约束超连接（mHC），增强了传统的残差连接；（3）以及用于更快收敛和更高训练稳定性的 Muon 优化器。我们使用超过 32 万亿个多样且高质量的 token 对这两个模型进行了预训练，并通过全面的后训练流程进一步释放和增强其能力。DeepSeek-V4-ProMax 是 DeepSeek-V4-Pro 的最大推理努力模式，重新定义了开放模型的最先进水平，在核心任务中优于其前身。同时，DeepSeek-V4 系列在长上下文场景中表现出高度的效率。在一百万 token 上下文设置下，DeepSeek-V4-Pro 仅需与 DeepSeek-V3.2 相比，单 token 推理 FLOPs 为 27%，KV 缓存为 10%。这使我们能够常规支持一百万 token 的上下文，从而使得长时间跨度任务和进一步的测试时间扩展更加可行。模型检查点可在 https://huggingface.co/collections/deepseek-ai/deepseek-v4 获取。

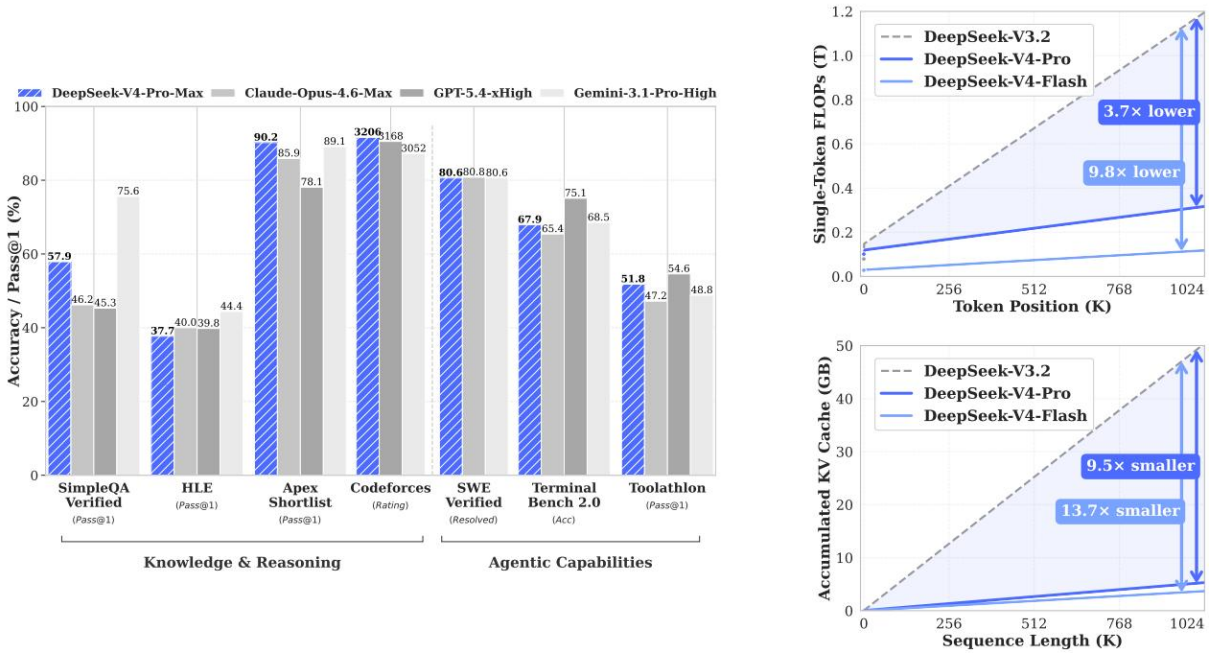

Figure 1 | Left: benchmark performance of DeepSeek-V4-Pro-Max and its counterparts. Right: inference FLOPs and KV cache size of DeepSeek-V4 series and DeepSeek-V3.2.

图 1 | 左：DeepSeek-V4-Pro-Max 及其同类模型的基准性能。右：DeepSeek-V4 系列和 DeepSeek-V3.2 的推理 FLOPs 和 KV 缓存大小。

Contents

目录

1. Introduction

1. 引言

The emergence of reasoning models (DeepSeek-AI, 2025; OpenAI, 2024c) has established a new paradigm of test-time scaling, driving substantial performance gains for Large Language Models (LLMs). However, this scaling paradigm is fundamentally constrained by the quadratic computational complexity of the vanilla attention mechanism (Vaswani et al., 2017), which creates a prohibitive bottleneck for ultra-long contexts and reasoning processes. Concurrently, the emergence of long-horizon scenarios and tasks — from complex agentic workflows to massive cross-document analysis — has also made efficient support for ultra-long contexts critical for future progress. While recent open-source efforts (Bai et al., 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025) have advanced general capabilities, this core architectural inefficiency in handling ultra-long sequences remains a key impediment, limiting further gains from test-time scaling and hindering further exploration into long-horizon scenarios and tasks.

推理模型的出现（DeepSeek-AI，2025；OpenAI，2024c）建立了一种新的测试时扩展范式，推动了大型语言模型（LLMs）性能的显著提升。然而，这种扩展范式受到 vanilla 注意力机制（Vaswani 等，2017）二次计算复杂度的根本性限制，这为超长上下文和推理过程创建了一个具有禁止性的瓶颈。同时，长视野场景和任务——从复杂的代理工作流程到大规模跨文档分析——的出现也使得对超长上下文的高效支持对未来进展至关重要。尽管最近的开源努力（Bai 等，2025a；DeepSeek-AI，2024；MiniMax，2025；Qwen，2025）提升了通用能力，但处理超长序列的核心架构低效性仍然是一个关键障碍，限制了测试时扩展带来的进一步提升，并阻碍了对长视野场景和任务的进一步探索。

In order to break the efficiency barrier in ultra-long contexts, we develop the DeepSeek-V4 series, including the preview versions of DeepSeek-V4-Pro with 1.6T parameters (49B activated) and

DeepSeek-V4-Flash with 284B parameters (13B activated). Through architectural innovations, DeepSeek-V4 series achieve a dramatic leap in computational efficiency for processing ultra-long sequences. This breakthrough enables efficient support for a context length of one million tokens, ushering in a new era of million-length contexts for next-generation LLMs. We believe our capability to efficiently handle ultra-long sequences unlocks the next frontier of test-time scaling, paves the way for deeper research into long-horizon tasks, and establishes a necessary foundation for exploring future paradigms like online learning.

为了突破超长上下文中的效率瓶颈，我们开发了 DeepSeek-V4 系列，包括预览版本的 DeepSeek-V4-Pro（1.6 万亿参数，4900 亿激活参数）和 DeepSeek-V4-Flash（2840 亿参数，130 亿激活参数）。通过架构创新，DeepSeek-V4 系列在处理超长序列时实现了计算效率的显著提升。这一突破使得支持一百万 token 的上下文长度成为可能，为下一代大语言模型开启了百万长度上下文的新纪元。我们认为，我们高效处理超长序列的能力解锁了测试时扩展的新前沿，为深入研究长期任务奠定了基础，并为未来范式如在线学习的探索提供了必要的基础。

Compared with the DeepSeek-V3 architecture (DeepSeek-AI, 2024), DeepSeek-V4 series retain the DeepSeekMoE framework (Dai et al., 2024) and Multi-Token Prediction (MTP) strategy, while introducing several key innovations in architecture and optimization. To enhance long-context efficiency, we design a hybrid attention mechanism combining Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA). CSA compresses the KV caches along the sequence dimension and then performs DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025), whereas HCA applies more aggressive compression to the KV caches but keeps dense attention. To strengthen modeling capability, we incorporate Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) that upgrade conventional residual connections. Additionally, we introduce the Muon (Jordan et al., 2024; Liu et al., 2025) optimizer to the training of DeepSeek-V4 series, leading to faster convergence and improved training stability.

与 DeepSeek-V3 架构（DeepSeek-AI，2024）相比，DeepSeek-V4 系列保留了 DeepSeekMoE 框架（Dai 等，2024）和 Multi-Token Prediction（MTP）策略，同时在架构和优化方面引入了多项关键创新。为了提升长上下文效率，我们设计了一种混合注意力机制，结合了压缩稀疏注意力（CSA）和高度压缩注意力（HCA）。CSA 沿着序列维度压缩 KV 缓存，然后执行 DeepSeek 稀疏注意力（DSA）（DeepSeek-AI，2025），而 HCA 对 KV 缓存进行更为激进的压缩，但保持密集注意力。为了增强建模能力，我们引入了流形约束超连接（mHC）（Xie 等，2026），升级了传统的残差连接。此外，我们在 DeepSeek-V4 系列的训练中引入了 Muon 优化器（Jordan 等，2024；Liu 等，2025），从而实现了更快的收敛速度和更稳定的训练效果。

To enable efficient training and inference for DeepSeek-V4 series as well as productive development, we introduce several infrastructure optimizations. First, we design and implement a single fused kernel for MoE modules that fully overlaps computation, communication, and memory access. Second, we employ TileLang (Wang et al., 2026), a Domain-Specific Language (DSL) to balance development productivity and runtime efficiency. Third, we provide efficient batch-invariant and deterministic kernel libraries to ensure bitwise reproducibility across training and inference. Fourth, for the training framework, we extend the autograd framework with tensor-level checkpointing for fine-grained recomputation control; and we enhance training efficiency with a hybrid ZeRO strategy for the Muon optimizer, cost-effective mHC implementations via recomputation and fused kernels, and two-stage contextual parallelism to manage compressed attention. Fifth, for the inference framework, we design a heterogeneous KV cache structure with on-disk storage strategies to enable efficient shared-prefix reuse. In addition, during the post-training stage, we incorporate FP4 quantization-aware training for MoE expert weights and the indexer QK path to reduce memory and computation.

为了实现 DeepSeek-V4 系列模型的高效训练和推理，以及促进生产力开发，我们引入了多项基础设施优化。首先，我们设计并实现了一个融合的单个内核，用于 MoE 模块，该内核完全重叠计算、通信和内存访问。其次，我们采用 TileLang（Wang 等，2026），一种领域特定语言（DSL），以平衡开发生产力和运行时效率。第三，我们提供了高效的批处理不变和确定性内核库，以确保训练和推理之间的位级可重复性。第四，针对训练框架，我们扩展了自动微分框架，引入张量级检查点以实现细粒度的重新计算控制；并通过混合 ZeRO 策略提升 Muon 优化器的训练效率，利用重新计算和融合内核实现成本效益高的 mHC 实现，并通过两阶段上下文并行性管理压缩注意力。第五，针对推理框架，我们设计了一种异构的 KV 缓存

结构，并采用磁盘存储策略以实现高效的共享前缀重用。此外，在微调阶段，我们引入了 FP4 量化感知训练，用于 MoE 专家权重和索引器 QK 路径，以减少内存和计算量。

By employing hybrid CSA and HCA, along with precision optimizations on computation and storage, DeepSeek-V4 series achieve significantly lower inference FLOPs and a substantially reduced KV cache size compared with DeepSeek-V3.2, especially in long-context settings. The right part of Figure 1 demonstrates the estimated single-token inference FLOPs and accumulated KV cache size of DeepSeek-V3.2 and DeepSeek-V4 series. In the scenario of 1M-token context, even DeepSeek-V4-Pro, which has a larger number of activated parameters, attains only 27% of the single-token FLOPs (measured in equivalent FP8 FLOPs) and 10% of the KV cache size relative to DeepSeek-V3.2. Furthermore, DeepSeek-V4-Flash, with its smaller number of activated parameters, pushes efficiency even further: in the 1M-token context setting, it achieves only 10% of the single-token FLOPs and 7% of the KV cache size compared with DeepSeek-V3.2. Additionally, for DeepSeek-V4 series, the routed expert parameters utilize FP4 precision. While the peak FLOPs for FP4 × FP8 operations are currently the same as FP8 × FP8 on existing hardware, they can theoretically be implemented to be 1/3 more efficient on future hardware, which will further enhance the efficiency of DeepSeek-V4 series.

通过结合混合 CSA 和 HCA，以及在计算和存储方面的精确优化，DeepSeek-V4 系列在与 DeepSeek-V3.2 相比时，在长上下文设置中实现了显著降低的推理 FLOPs 和大幅减少的 KV 缓存大小。图 1 的右侧展示了 DeepSeek-V3.2 和 DeepSeek-V4 系列的单标记推理 FLOPs 和累计 KV 缓存大小的估计值。在 1M 标记上下文场景中，即使具有更多激活参数的 DeepSeek-V4-Pro，其单标记 FLOPs（以等效 FP8 FLOPs 计算）也仅为 DeepSeek-V3.2 的 27%，其 KV 缓存大小仅为 DeepSeek-V3.2 的 10%。此外，DeepSeek-V4-Flash 由于激活参数数量更少，进一步提升了效率：在 1M 标记上下文设置中，其单标记 FLOPs 仅为 DeepSeek-V3.2 的 10%，其 KV 缓存大小仅为 DeepSeek-V3.2 的 7%。另外，对于 DeepSeek-V4 系列，路由专家参数使用 FP4 精度。虽然目前 FP4 × FP8 操作的峰值 FLOPs 与现有硬件上的 FP8 × FP8 相同，但理论上可以在未来硬件上实现更高的效率，提高 1/3，这将进一步提升 DeepSeek-V4 系列的效率。

During pre-training, we train DeepSeek-V4-Flash on 32T tokens and DeepSeek-V4-Pro on 33T tokens, respectively. After pre-training, these two models can natively and efficiently support 1M-length contexts. In our internal evaluations, DeepSeek-V4-Flash-Base already surpasses DeepSeek-V3.2-Base across a majority of benchmarks with its more parameter-efficient design. DeepSeek-V4-Pro-Base further extends this advantage to set a new performance standard among DeepSeek foundation models, achieving comprehensive superiority across reasoning, coding, long-context, and world knowledge tasks.

在预训练阶段，我们分别使用 32T 个 token 对 DeepSeek-V4-Flash 进行训练，使用 33T 个 token 对 DeepSeek-V4-Pro 进行训练。预训练完成后，这两个模型能够原生且高效地支持 1M 长度的上下文。在我们的内部评估中，DeepSeek-V4-Flash-Base 由于其更参数高效的架构设计，在大多数基准测试中已经超越了 DeepSeek-V3.2-Base。DeepSeek-V4-Pro-Base 进一步扩展了这一优势，在 DeepSeek 基础模型中树立了新的性能标准，在推理、编程、长上下文和世界知识任务中实现了全面的优越性。

The post-training pipeline of DeepSeek-V4 series features a two-stage paradigm: the independent cultivation of domain-specific experts, followed by unified model consolidation via on-policy distillation (Gu et al., 2024; Lu and Lab, 2025). Initially, for each target domain — such as mathematics, coding, agent, and instruction following — a separate expert model is trained independently. The base model first undergoes Supervised Fine-Tuning (SFT) on high-quality, domain-specific data to establish foundational capabilities. Subsequently, Reinforcement Learning (RL) is applied using Group Relative Policy Optimization (GRPO) (DeepSeek-AI, 2025), which further optimizes the model for domain-aligned behaviors guided by reward models tailored to specific success criteria. This phase yields a diverse set of specialized experts, each excelling in its respective field. Finally, to integrate these distinct proficiencies, a single unified model is trained through on-policy distillation, wherein the unified model acts as the student learning to optimize the reverse KL loss with teacher models.

DeepSeek-V4 系列的微调后处理流程采用了一个两阶段范式：首先独立培养领域专家，然后通过基于策略的蒸馏方法统一模型（Gu 等，2024；Lu 和 Lab，2025）。最初，对于每个目标领域——例如数学、编程、代理和遵循指令——都会独立训练一个专门的专家模型。基础模型首先在高质量的领域特定数据上进行监

督微调（SFT），以建立基础能力。随后，使用群体相对策略优化（GRPO）（DeepSeek-AI，2025）进行强化学习，该方法进一步优化模型，使其行为符合特定成功标准的奖励模型引导的领域对齐行为。这一阶段产生了一组多样化的专业专家，每个专家在其各自领域表现卓越。最后，为了整合这些不同的专业技能，通过基于策略的蒸馏方法训练一个统一的模型，其中统一模型作为学生，利用教师模型优化反向 KL 损失。

Summary of Core Evaluation Results

核心评估结果摘要

- Knowledge: In assessments of broad world knowledge, DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, significantly outperforms leading open-source models on the SimpleQA (OpenAI, 2024d) and Chinese-SimpleQA (He et al., 2024) benchmarks. Regarding educational knowledge — evaluated via MMLU-Pro (Wang et al., 2024b), HLE (Phan et al., 2025), and GPQA (Rein et al., 2023) — DeepSeek-V4-Pro-Max shows a marginal lead over its open-source counterparts. DeepSeek-V4-Pro-Max has significantly closed the gap with the leading proprietary model, Gemini-3.1-Pro, despite still trailing it in these knowledge-based evaluations.

- 知识：在广泛世界知识评估中，DeepSeek-V4-Pro-Max（DeepSeek-V4-Pro 的最大推理努力模式）在 SimpleQA（OpenAI, 2024d）和 Chinese-SimpleQA（He 等人，2024）基准测试中显著优于领先的开源模型。关于教育知识——通过 MMLU-Pro（Wang 等人，2024b）、HLE（Phan 等人，2025）和 GPQA（Rein 等人，2023）进行评估——DeepSeek-V4-Pro-Max 在这些基于知识的评估中仅略优于其开源竞争对手。尽管在这些知识评估中仍落后于领先的专有模型 Gemini-3.1-Pro，但 DeepSeek-V4-Pro-Max 已显著缩小了与 Gemini-3.1-Pro 的差距。

- Reasoning: Through the expansion of reasoning tokens, DeepSeek-V4-Pro-Max demonstrates superior performance relative to GPT-5.2 and Gemini-3.0-Pro on standard reasoning benchmarks. Nevertheless, its performance falls marginally short of GPT-5.4 and Gemini-3.1-Pro, suggesting a developmental trajectory that trails state-of-the-art frontier models by approximately 3 to 6 months. Furthermore, DeepSeek-V4-Flash-Max achieves comparable

    - 推理：通过扩展推理标记，DeepSeek-V4-Pro-Max 在标准推理基准测试中表现出优于 GPT-5.2 和 Gemini-3.0-Pro 的性能。然而，其性能略微落后于 GPT-5.4 和 Gemini-3.1-Pro，表明其发展轨迹比最先进的前沿模型落后约 3 到 6 个月。此外，DeepSeek-V4-Flash-Max 也达到了可比的

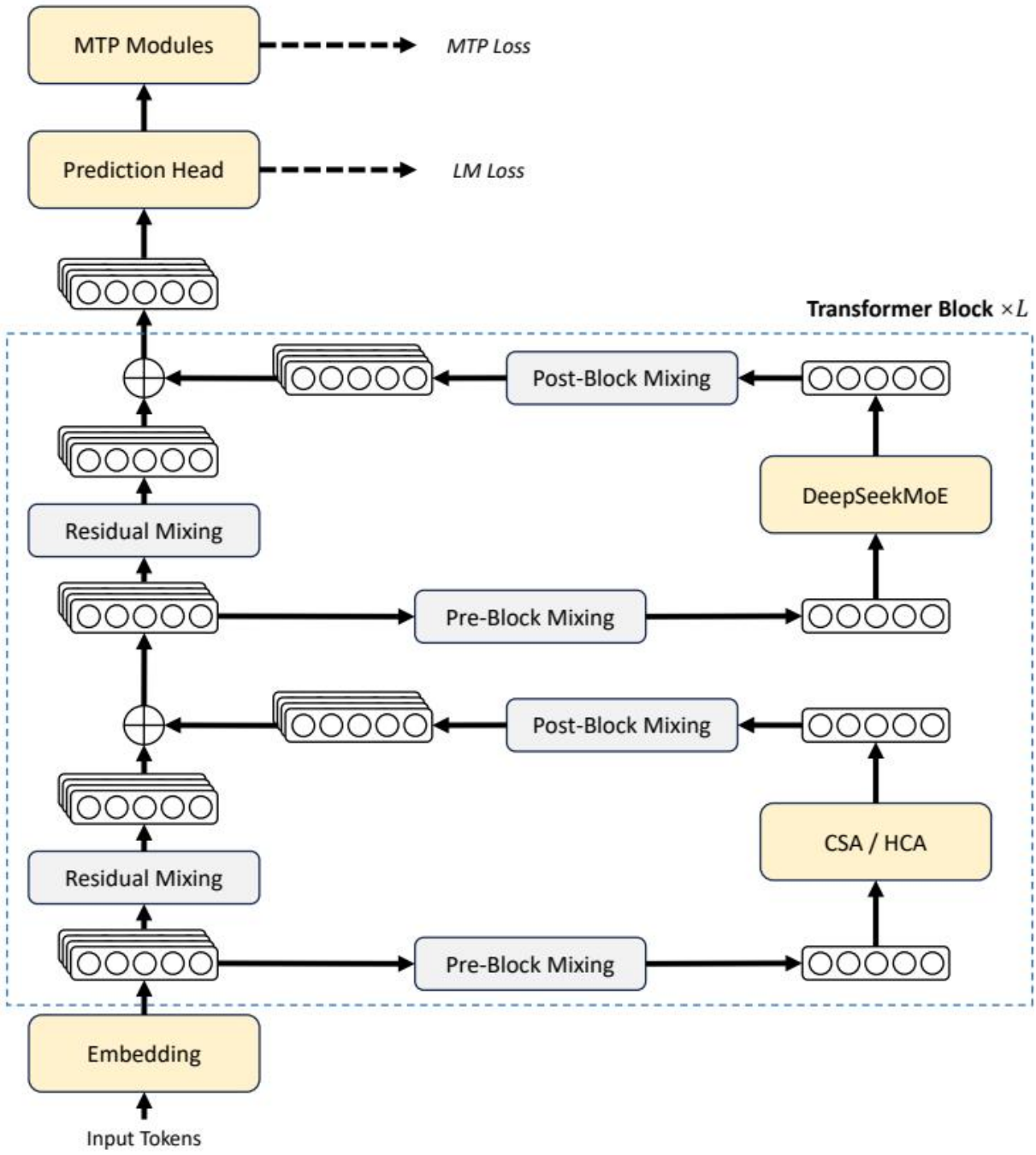

Figure 1: images/2c5ffc992a56d87715c81c76ccc8e01e6186c02cf3a03d4a75ca416f22237b79.jpg

Figure 2 | Overall architecture of DeepSeek-V4 series. We use hybrid CSA (Compressed Sparse Attention) and HCA (Heavily Compressed Attention) for attention layers, DeepSeekMoE for feed-forward layers, and strengthen conventional residual connections with mHC.

图 2 | DeepSeek-V4 系列的整体架构。我们在注意力层中使用混合 CSA（压缩稀疏注意力）和 HCA（高度压缩注意力），在前馈层中使用 DeepSeekMoE，并通过 mHC 加强传统的残差连接。

performance to GPT-5.2 and Gemini-3.0-Pro, establishing itself as a highly cost-effective architecture for complex reasoning tasks.

性能超越 GPT-5.2 和 Gemini-3.0-Pro，确立了其作为复杂推理任务的高性价比架构。

- Agent: On public benchmarks, DeepSeek-V4-Pro-Max is on par with leading open-source models, such as Kimi-K2.6 and GLM-5.1, but slightly worse than frontier closed models. In our internal evaluation, DeepSeek-V4-Pro-Max outperforms Claude Sonnet 4.5 and approaches the level of Opus 4.5.

- Long-Context: DeepSeek-V4-Pro-Max delivers strong results on synthetic and real use cases with a 1-million-token context window, surpassing even Gemini-3.1-Pro on academic benchmarks.

- DeepSeek-V4-Pro v.s. DeepSeek-V4-Flash: DeepSeek-V4-Flash-Max exhibits lower performance in knowledge evaluations due to its smaller parameter scale. However, it achieves comparable results on reasoning tasks when allocated a larger thinking budget. In agent evaluations, while DeepSeek-V4-Flash-Max matches the performance of DeepSeek-V4-Pro-Max on several benchmarks, it still trails its larger counterpart on more complex, high-difficulty tasks.

- Agent: 在公开基准测试中，DeepSeek-V4-Pro-Max 与领先的开源模型（如 Kimi-K2.6 和 GLM-5.1）表现相当，但略逊于前沿的闭源模型。在我们内部评估中，DeepSeek-V4-Pro-Max 表现优于 Claude Sonnet 4.5，并接近 Opus 4.5 的水平。

- 长上下文：DeepSeek-V4-Pro-Max 在拥有 100 万 token 上下文窗口的合成和真实使用案例中表现出色，在学术基准测试中甚至超过了 Gemini-3.1-Pro。

- DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 的对比：由于参数规模较小，DeepSeek-V4-Flash-Max 在知识评估中表现较低。然而，当分配更大的思考预算时，它在推理任务中能够取得可比的结果。在代理评估中，尽管 DeepSeek-V4-Flash-Max 在多个基准测试中与 DeepSeek-V4-Pro-Max 表现相当，但在更复杂、难度更高的任务上仍落后于其更大的对应版本。

2. Architecture

2. 架构

Overall, DeepSeek-V4 series retain the Transformer (Vaswani et al., 2017) architecture and Multi-Token Prediction (MTP) modules (DeepSeek-AI, 2024; Gloeckle et al., 2024), while introducing several key upgrades over DeepSeek-V3: (1) firstly, we introduce the Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) to strengthen conventional residual connections;

总体而言，DeepSeek-V4 系列保留了 Transformer（Vaswani 等，2017）架构和多标记预测（MTP）模块（DeepSeek-AI，2024；Gloeckle 等，2024），并在 DeepSeek-V3 的基础上引入了若干关键升级：（1）首先，我们引入了流形约束超连接（mHC）（Xie 等，2026），以增强传统的残差连接；

(2) secondly, we design a hybrid attention architecture, which greatly improves long-context efficiency through Compressed Sparse Attention and Heavily Compressed Attention. (3) thirdly, we employ Muon (Jordan et al., 2024; Liu et al., 2025) as the optimizer. For the Mixture-of-Experts (MoE) components, we still adopt the DeepSeekMoE (Dai et al., 2024) architecture, with only minor adjustments from DeepSeek-V3. The Multi-Token Prediction (MTP) (DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024; Qi et al., 2020) configuration remains identical to that of DeepSeek-V3. All other unspecified details follow the settings established in DeepSeek-V3 (DeepSeek-AI, 2024). Figure 2 illustrates the overall architecture of DeepSeek-V4, and the details are described below.

(2) 其次，我们设计了一种混合注意力架构，通过压缩稀疏注意力和高度压缩注意力大大提高了长上下文的效率。(3) 第三，我们采用 Muon（Jordan 等，2024；Liu 等，2025）作为优化器。对于专家混合

(Mixture-of-Experts，MoE)组件，我们仍然采用 DeepSeekMoE(Dai 等，2024)架构，仅对 DeepSeek-V3 进行了少量调整。多标记预测（MTP）(DeepSeek-AI，2024；Gloeckle 等，2024；Li 等，2024；Qi 等，2020）配置与 DeepSeek-V3 相同。所有未指定的其他细节均遵循 DeepSeek-V3（DeepSeek-AI，2024）中设定的配置。图 2 展示了 DeepSeek-V4 的整体架构，下面将详细描述其细节。

2.1. Designs Inherited from DeepSeek-V3

2.1. 从 DeepSeek-V3 继承的设计

Mixture-of-Experts. As previous DeepSeek-series models (DeepSeek-AI, 2024; DeepSeek-AI, 2024), DeepSeek-V4 series also adopt the DeepSeekMoE paradigm (Dai et al., 2024) for Feed-Forward Networks (FFNs), which sets fine-grained routed experts and shared experts. Different from DeepSeek-V3, we change the activation function that computes the affinity scores from Sigmoid($\cdot$) into Sqrt(Softplus($\cdot$)) . For load balancing, we also employ the auxiliary-loss-free strategy (DeepSeek-AI, 2024; Wang et al., 2024a), augmented by a slight sequence-wise balance loss that prevents extreme imbalance within individual sequences. For DeepSeek-V4, we remove the constraint on the number of routing target nodes, and carefully redesign the parallelism strategy to maintain training efficiency. Furthermore, compared with DeepSeek-V3, we replace the dense FFN layers in the initial several Transformer blocks with MoE layers that employ Hash routing (Roller et al., 2021). The Hash routing strategy determines the target experts of each token according to a predefined hash function with regard to the input token ID.

专家混合（Mixture-of-Experts）。与之前的 DeepSeek 系列模型（DeepSeek-AI, 2024; DeepSeek-AI, 2024）一样，DeepSeek-V4 系列也采用了 DeepSeekMoE 框架（Dai 等，2024）用于前馈网络（FFNs），该框架设置了细粒度路由专家和共享专家。与 DeepSeek-V3 不同，我们将用于计算亲和度分数的激活函数从 Sigmoid($\cdot$) 更换为 Sqrt(Softplus($\cdot$)) 。为了实现负载均衡，我们还采用了无辅助损失策略(DeepSeek-AI, 2024; Wang 等，2024a)，并辅以轻微的序列级平衡损失，以防止单个序列内部出现极端不平衡。对于 DeepSeek-V4，我们去除了对路由目标节点数量的限制，并仔细重新设计了并行策略以保持训练效率。此外，与 DeepSeek-V3 相比，我们用采用哈希路由（Roller 等，2021）的 MoE 层替换了初始几个 Transformer 块中的密集型 FFN 层。哈希路由策略根据预定义的哈希函数，依据输入 token ID 确定每个 token 的目标专家。

Multi-Token Prediction. As DeepSeek-V3, DeepSeek-V4 series also set MTP modules and objectives. Given that the MTP strategy has been validated in DeepSeek-V3, we adopt the same strategy for DeepSeek-V4 series without modification.

多标记预测。与 DeepSeek-V3、DeepSeek-V4 系列一样，也设置了 MTP 模块和目标。鉴于 MTP 策略在 DeepSeek-V3 中已经得到验证，我们对 DeepSeek-V4 系列采用相同的策略，无需修改。

2.2. Manifold-Constrained Hyper-Connections

2.2. 拓扑约束的超连接

As shown in Figure 2, DeepSeek-V4 series incorporate Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) to strengthen the conventional residual connections between adjacent Transformer blocks. Compared with naive Hyper-Connections (HC) (Zhu et al., 2025), the core idea of mHC is to constrain the residual mapping onto a specific manifold, and thus enhance the stability of signal propagation across layers while preserving model expressivity. This subsection briefly introduces the standard HC and describes how we design mHC for stable training.

如图 2 所示，DeepSeek-V4 系列引入了流形约束超连接（Manifold-Constrained Hyper-Connections，mHC）(Xie et al., 2026)，以增强相邻 Transformer 块之间的传统残差连接。与简单的超连接（Hyper-Connections，HC）(Zhu et al., 2025）相比，mHC 的核心思想是将残差映射约束在特定的流形上，从而在保持模型表达能力的同时，增强跨层信号传播的稳定性。本小节简要介绍标准的 HC，并描述我们如何设计 mHC 以实现稳定的训练。

Standard Hyper-Connections. The standard HC expands the width of the residual stream by a factor of n_{hc} . Specifically, the shape of the residual stream is expanded from R^d to $R^{n_{hc}\times d}$, where d is the hidden size of the actual layer input. Let $X_l = [\mathbf{x}_{l,1}; \ldots ; \mathbf{x}_{l,n_{hc}}]^T \in \mathbb{R}^{n_{hc}\times d}$ be the residual state before the l-th layer. HC introduces three linear mappings: an input mapping $A_l \in R^{1\times n_{hc}}$, a residual transformation $B_l \in R^{n_{hc}\times n_{hc}}$, and an output mapping $C_l \in R^{n_{hc}\times 1}$. The update of the residual state is then formulated as:

标准超连接。标准 HC 通过因子 n_{hc} 扩展残差流的宽度。具体来说，残差流的形状从 R^d 扩展到 $R^{n_{hc}\times d}$ ，其中 d 是实际层输入的隐藏大小。设 $X_l = [\mathbf{x}_{l,1}; \ldots ; \mathbf{x}_{l,n_{hc}}]^T \in \mathbb{R}^{n_{hc}\times d}$ 为第 l 层之前的残差状态。HC 引入了三个线性映射：输入映射 $A_l \in R^{1\times n_{hc}}$ 、残差变换 $B_l \in R^{n_{hc}\times n_{hc}}$ 和输出映射 $C_l \in R^{n_{hc}\times 1}$ 。残差状态的更新则可以表示为：

$$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l), \tag{1}$$

where F_l denotes the l-th layer (e.g., an MoE layer), whose input and output shapes are both R^d . Note that the actual layer input $A_l X_l \in R^d$ is also d-dimensional, so the expanded residual

其中 F_l 表示第 l 层（例如，一个 MoE 层），其输入和输出形状均为 R^d 。请注意，实际的层输入 $A_l X_l \in R^d$ 也是 d 维的，因此扩展后的残差

width does not influence the design of the inner layers. HC decouples the residual width from the actual hidden size, offering a complementary scaling axis with minimal computational overhead, as n_{hc} is typically much smaller than the hidden size d. However, even though HC has demonstrated potential in improving model performance, we find that the training will frequently exhibit numerical instability when stacking multiple layers, which hinders the scaling of HC.

宽度不会影响内部层的设计。HC 将残差宽度与实际隐藏层大小解耦，提供了一个计算开销最小的互补扩展轴，因为 n_{hc} 通常远小于隐藏层大小 d。然而，尽管 HC 在提升模型性能方面展现出潜力，我们发现当堆叠多层时，训练过程经常会表现出数值不稳定，这阻碍了 HC 的扩展。

Manifold-Constrained Residual Mapping. The core innovation of mHC is to constrain the residual mapping matrix B_l to the manifold of doubly stochastic matrices (the Birkhoff polytope) M, and thus enhance the stability of signal propagation across layers:

流形约束的残差映射。mHC 的核心创新在于将残差映射矩阵 B_l 约束在双随机矩阵（Birkhoff 多面体）M 的流形上，从而增强信号在各层之间的传播稳定性：

$$B_l \in \mathcal{M} := \{M \in \mathbb{R}^{n\times n} \mid M\mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geqslant 0\}. \tag{2}$$

This constraint ensures that the spectral norm of the mapping matrix $\|B_l\|_2$ is bounded by 1, so the residual transformation is non-expansive, which increases the numerical stability during both the forward pass and backpropagation. Besides, the set M is closed under multiplication, which guarantees stability in the scenarios of deep stacks of mHC. In addition, the input transformation A_l and output transformation C_l are also constrained to be non-negative and bounded via a Sigmoid function to avoid the risk of signal cancellation.

这个约束确保了映射矩阵 $\|B_l\|_2$ 的谱范数被限制在 1 以内，因此残差变换是非扩展的，这在前向传播和反向传播过程中提高了数值稳定性。此外，集合 M 在乘法下是封闭的，这保证了在深度堆叠的 mHC 场景中的稳定性。另外，输入变换 A_l 和输出变换 C_l 也被限制为通过 Sigmoid 函数进行非负和有界，以避免信号抵消的风险。

Dynamic Parameterization. The parameters of three linear mappings are dynamically generated, which are decomposed into a dynamic (input-dependent) component and a static (input-independent) component. Given the input $X_l \in R^{n_{hc}\times d}$, it is first flattened and normalized:

$\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1\times n_{hc}d}$. Then, we follow the conventional HC to generate the unconstrained raw parameters $\tilde{A}_l \in R^{1\times n_{hc}}$, $\tilde{B}_l \in R^{n_{hc}\times n_{hc}}$, and $\tilde{C}_l \in R^{n_{hc}\times 1}$:

动态参数化。三个线性映射的参数是动态生成的，它们被分解为动态 (输入依赖) 部分和静态 (输入独立) 部分。给定输入 $X_l \in R^{n_{hc}\times d}$，首先将其展平并归一化：$\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1\times n_{hc}d}$。然后，我们按照传统的 HC 方法生成无约束的原始参数 $\tilde{A}_l \in R^{1\times n_{hc}}$、$\tilde{B}_l \in R^{n_{hc}\times n_{hc}}$ 和 $\tilde{C}_l \in R^{n_{hc}\times 1}$：

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \tag{3}$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \tag{4}$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \tag{5}$$

where $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}d\times n_{\text{hc}}}$ and $W_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}d\times n_{\text{hc}}^2}$ are learnable parameters for generating the dynamic components; Mat($\cdot$) reshapes a vector of size $1 \times n_{\text{hc}}^2$ into a matrix of size $n_{\text{hc}} \times n_{\text{hc}}$; $S_l^{\text{pre}} \in \mathbb{R}^{1\times n_{\text{hc}}}, S_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}\times 1}$, and $S_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}\times n_{\text{hc}}}$ are learnable static biases; and $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}} \in \mathbb{R}$ are learnable gating factors initialized to small values.

其中 $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}d\times n_{\text{hc}}}$ 和 $W_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}d\times n_{\text{hc}}^2}$ 是用于生成动态组件的可学习参数；Mat($\cdot$) 将大小为 $1 \times n_{\text{hc}}^2$ 的向量重塑为大小为 $n_{\text{hc}} \times n_{\text{hc}}$ 的矩阵；$S_l^{\text{pre}} \in \mathbb{R}^{1\times n_{\text{hc}}}, S_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}\times 1}$ 和 $S_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}\times n_{\text{hc}}}$ 是可学习的静态偏置；而 $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}} \in \mathbb{R}$ 是初始化为小值的可学习门控因子。

Applying Parameter Constraints. After obtaining the unconstrained raw parameters $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$, we then apply constraints described earlier to them to enhance the numerical stability. To be specific, for the input and output mappings, we employ a Sigmoid function $\sigma(\cdot)$ to ensure their non-negativity and boundedness:

应用参数约束。在获得无约束的原始参数 $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$ 后，我们随后应用之前描述的约束条件，以提高数值稳定性。具体来说，对于输入和输出映射，我们采用 Sigmoid 函数 $\sigma(\cdot)$ 来确保其非负性和有界性：

$$A_l = \sigma(\tilde{A}_l), \tag{6}$$

$$C_l = 2\sigma(\tilde{C}_l). \tag{7}$$

As for the residual mapping $\tilde{B}_l$, we project it onto the manifold of doubly stochastic matrices M. This is achieved by the Sinkhorn-Knopp algorithm, which first applies an exponential function to $\tilde{B}_l$ to ensure positivity, getting $M^{(0)} = \exp(\tilde{B}_l)$, and then iteratively performs column and row normalization:

关于残差映射 $\tilde{B}_l$ ，我们将其投影到双重随机矩阵流形 M 上。这是通过 Sinkhorn-Knopp 算法实现的，该算法首先对 $\tilde{B}_l$ 应用指数函数以确保其为正值，得到 $M^{(0)} = \exp(\tilde{B}_l)$ ，然后迭代地进行列和行归一化：

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})), \tag{8}$$

where T_r and T_c denote row and column normalization, respectively. This iteration converges to a constrained doubly stochastic matrix $B_l = M^{(t_{\max})}$. We choose $t_{max} = 20$ as a practical value.

其中 T_r 和 T_c 分别表示行归一化和列归一化。这种迭代会收敛到一个受约束的双随机矩阵 $B_l = M^{(t_{\max})}$。我们选择 $t_{max} = 20$ 作为实际值。

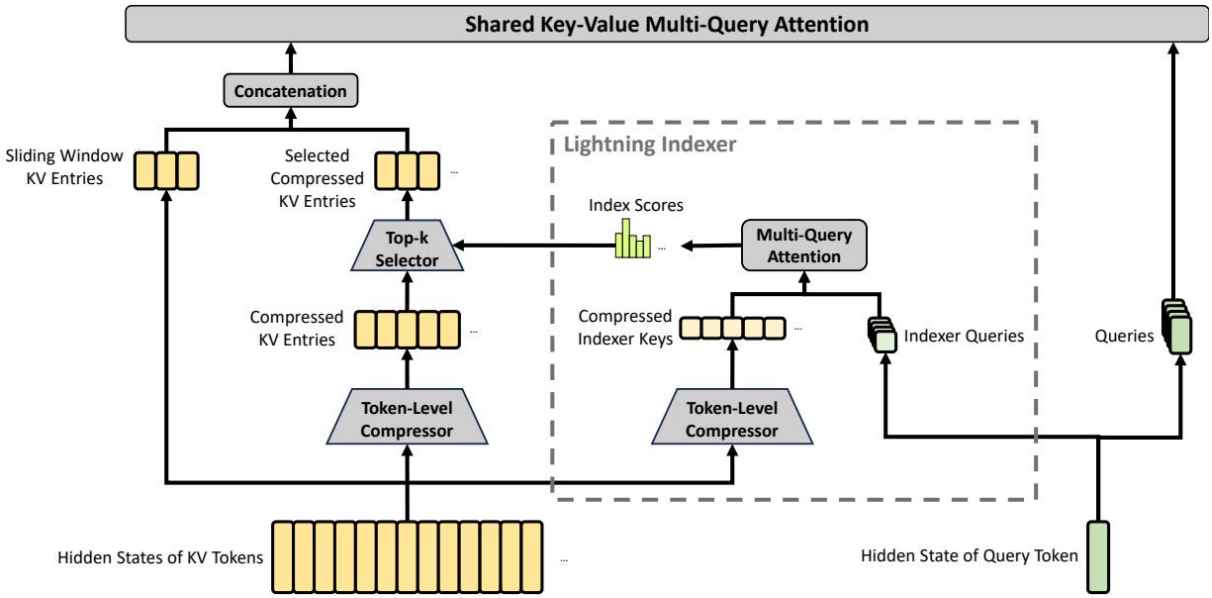

Figure 2: images/cff2b362b699adbc334a2f504449cbfbb7010bb6f736884475dc536b11b1d80b.jpg

Figure 3 | Core architectures of CSA. It compresses the number of KV entries to $\frac{1}{m}$ times, and then applies DeepSeek Sparse Attention for further acceleration. Additionally, a small set of sliding window KV entries is combined with the selected compressed KV entries to enhance local fine-grained dependencies.

图 3 | CSA 的核心架构。它将 KV 条目数量压缩到 $\frac{1}{m}$ 倍，然后应用 DeepSeek 稀疏注意力以进一步加速。此外，一个小型的滑动窗口 KV 条目集与所选的压缩 KV 条目相结合，以增强局部细粒度依赖关系。

2.3. Hybrid Attention with CSA and HCA

2.3. 基于 CSA 和 HCA 的混合注意力机制

As the context length reaches extreme scales, the attention mechanism emerges as the dominant computational bottleneck in a model. For DeepSeek-V4, we design two efficient attention architectures — Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) — and employ their interleaved hybrid configuration, which substantially reduces the computational cost of attention in long-text scenarios. CSA integrates both compression and sparse attention strategies: it first compresses the Key-Value (KV) cache of every m tokens into one entry, and then applies DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025) where each query token attends to only k compressed KV entries. HCA aims for extreme compression by consolidating the KV cache of every $m'(\gg m)$ tokens into a single entry. The hybrid architecture of CSA and HCA remarkably improves the long-context efficiency of DeepSeek-V4 series, making one-million-token context feasible in practice. This subsection describes the core techniques of our hybrid attention architecture, and we also provide an open-source implementation [1] to specify more details unambiguously.

当上下文长度达到极端规模时，注意力机制成为模型中的主要计算瓶颈。对于 DeepSeek-V4，我们设计了两种高效的注意力架构 —— 压缩稀疏注意力（Compressed Sparse Attention，CSA）和高度压缩注意力（Heavily Compressed Attention，HCA），并采用它们交错的混合配置，显著降低了长文本场景下注意力计算的成本。CSA 集成了压缩和稀疏注意力策略：它首先将每 m 个 token 的 Key-Value（KV）缓存压缩为一个条目，然后应用 DeepSeek 稀疏注意力（DSA）（DeepSeek-AI，2025），其中每个查询 token 仅关注 k 个压缩后的 KV 条目。HCA 通过将每 $m'(\gg m)$ 个 token 的 KV 缓存合并为一个条目，实现极端压缩。CSA 和 HCA 的混合架构显著提升了 DeepSeek-V4 系列在长上下文场景下的效率，使得实际应用中实现一百万 token 的上下文成为可能。本小节描述了我们混合注意力架构的核心技术，我们还提供了开源实现 [1] 以更明确地说明更多细节。

2.3.1. Compressed Sparse Attention

2.3.1. 压缩稀疏注意力

The core architecture of CSA is illustrated in Figure 3, which first compresses the KV cache of each m tokens into one entry, and then applies DeepSeek Sparse Attention for further acceleration.

CSA 的核心架构如图 3 所示，其首先将每个 m tokens 的 KV 缓存压缩为一个条目，然后应用 DeepSeek 稀疏注意力以进一步加速。

Compressed Key-Value Entries. Let $H \in R^{n\times d}$ be a sequence of input hidden states, where n is the sequence length and d is the hidden size. CSA first computes two series of KV entries $C^a, C^b \in R^{n\times c}$ and their corresponding compression weights $Z^a, Z^b \in R^{n\times c}$, where c is the head

压缩的键值条目。设 $H \in R^{n\times d}$ 为输入隐藏状态的序列，其中 n 是序列长度，d 是隐藏状态的维度。CSA 首先计算两个系列的 KV 条目 $C^a, C^b \in R^{n\times c}$ 及其对应的压缩权重 $Z^a, Z^b \in R^{n\times c}$，其中 c 是头的数目

dimension:

维度：

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \tag{9}$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \tag{10}$$

where $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in R^{d\times c}$ are trainable parameters. Next, each m KV entries in C^a and C^b will be compressed into one entry according to their compression weights and learnable positional biases $B^a, B^b \in R^{m\times c}$, producing $C^{Comp} \in R^{\frac{n}{m}\times c}$. Each compressed entry $C^{Comp}_i \in R^c$ is computed by

其中 $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in R^{d\times c}$ 是可训练参数。接下来，每个 m KV 条目在 C^a 和 C^b 中将根据其压缩权重和可学习的位置偏差 $B^a, B^b \in R^{m\times c}$ 压缩成一个条目，生成 $C^{Comp} \in R^{\frac{n}{m}\times c}$。每个压缩条目 $C^{Comp}_i \in R^c$ 是通过

$$\left[S^a_{mi:m(i+1)-1}; S^b_{m(i-1):mi-1}\right] = \text{Softmax}_{\text{row}}\left(\left[Z^a_{mi:m(i+1)-1} + B^a; Z^b_{m(i-1):mi-1} + B^b\right]\right), \tag{11}$$

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b, \tag{12}$$

where $\odot$ denotes the Hadamard product; Softmax $_{\text{row}}(\cdot)$ denotes the softmax operation along the row dimension, which performs normalization across the total of 2m elements from both Z^a and Z^b . When i = 0, $Z^b_{m(i-1):mi-1}$ is padded with negative infinity and $C^b_{m(i-1):mi-1}$ is padded with zeros. Note that each C_i^{Comp} is derived from 2m KV entries, but the indexes of C^b used for C_i^{Comp} and the indexes of C^a used for C_{i-1}^{Comp} are overlapped. Therefore, CSA in fact compresses the sequence length to $\frac{1}{m}$ times.

其中 $\odot$ 表示 Hadamard 乘积；Softmax $_{\text{row}}(\cdot)$ 表示沿行维度进行的 softmax 操作，它对来自 Z^a 和 Z^b 的总共 2m 个元素进行归一化。当 i = 0 时，$Z^b_{m(i-1):mi-1}$ 用负无穷进行填充，$C^b_{m(i-1):mi-1}$ 用零进行填充。请注意，每个 C_i^{Comp} 都来源于 2m 个 KV 元素，但用于 C_i^{Comp} 的 C^b 的索引和用于 C_{i-1}^{Comp} 的 C^a 的索引是重叠的。因此，CSA 实际上将序列长度压缩到 $\frac{1}{m}$ 倍。

Lightning Indexer for Sparse Selection. After obtaining the compressed KV entries C^{Comp} , CSA applies the DSA strategy to select top-k compressed KV entries for core attention. First, CSA performs the same compression operation used for C^{Comp} to get compressed indexer keys $K^{IComp} \in R_m^{\frac{n}{m} \times c^I}$, where c^I is the indexer head dimension. Then, for a query token t, we produce the indexer queries $\{q^I_{t,1}; q^I_{t,2}; ...; q^I_{t,n^I_h}\}$ in a low-rank manner:

闪电索引器用于稀疏选择。在获取压缩后的 KV 条目 C^{Comp} 后，CSA 应用 DSA 策略来选择用于核心注意力的前 k 个压缩 KV 条目。首先，CSA 使用与 C^{Comp} 相同的压缩操作来获取压缩的索引器键 $K^{IComp} \in R_m^{\frac{n}{m} \times c^I}$，其中 c^I 是索引器头的维度。然后，对于一个查询 token t，我们以低秩方式生成索引器查询 $\{q^I_{t,1}; q^I_{t,2}; ...; q^I_{t,n^I_h}\}$：

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \tag{13}$$

$$\left[\mathbf{q}^I_{t,1}; \mathbf{q}^I_{t,2}; \dots; \mathbf{q}^I_{t,n^I_h}\right] = \mathbf{q}^I_t = \mathbf{c}_t^Q \cdot W^{IUQ}, \tag{14}$$

where $\mathbf{h}_t \in \mathbb{R}^d$ is the input hidden state of the query token t ; $\mathbf{c}_t^Q \in \mathbb{R}^{d_c}$ is the compressed latent vector for queries; d_c denotes the query compression dimension; n^I_h denotes the number of indexer query heads; $W^{DQ} \in \mathbb{R}^{d \times d_c}$ and $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n^I_h}$ are the down-projection and up-projection matrices for indexer queries, respectively. Next, the index score $I_{t,s} \in \mathbb{R}$ between the query token t and a preceding compressed block s ($s <$ Floor$(\frac{t}{m})$) is computed by

其中 $\mathbf{h}_t \in \mathbb{R}^d$ 是查询标记 t 的输入隐藏状态；$\mathbf{c}_t^Q \in \mathbb{R}^{d_c}$ 是查询的压缩潜在向量；d_c 表示查询压缩维度；n^I_h 表示索引器查询头的数量；$W^{DQ} \in \mathbb{R}^{d \times d_c}$ 和 $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n^I_h}$ 分别是索引器查询的下投影和上投影矩阵。接下来，查询标记 t 与前面的压缩块 s（$s <$ Floor$(\frac{t}{m})$）之间的索引得分 $I_{t,s} \in \mathbb{R}$ 是通过以下方式计算的：

$$\left[w^I_{t,1}; w^I_{t,2}; \dots; w^I_{t,n^I_h}\right] = \mathbf{w}^I_t = \mathbf{h}_t \cdot W^w, \tag{15}$$

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}\left(\mathbf{q}_{t,h}^I \cdot K_s^{\text{IComp}}\right), \tag{16}$$

where $W^w \in R^{d \times n_h^I}$ is a learnable matrix; $w_{t,h}^I \in R$ is the weight of the h-th indexer head. For a query token t, given its index scores $I_{t,:}$, we employ a top-k selector to selectively retain a subset of compressed KV entries $C_t^{SprsComp}$ for subsequent core attention:

其中 $W^w \in R^{d \times n_h^I}$ 是一个可学习的矩阵；$w_{t,h}^I \in R$ 是第 h 个索引器头的权重。对于一个查询标记 t，给定其索引得分 $I_{t,:}$ ，我们采用一个 top-k 选择器，有选择地保留一组压缩的 KV 条目 $C_t^{SprsComp}$ 以供后续的核心注意力使用：

$$\mathcal{C}_t^{\text{SprsComp}} = \left\{ C_s^{\text{Comp}} \mid I_{t,s} \in \text{Top-k}\left(I_{t,:}\right) \right\}. \tag{17}$$

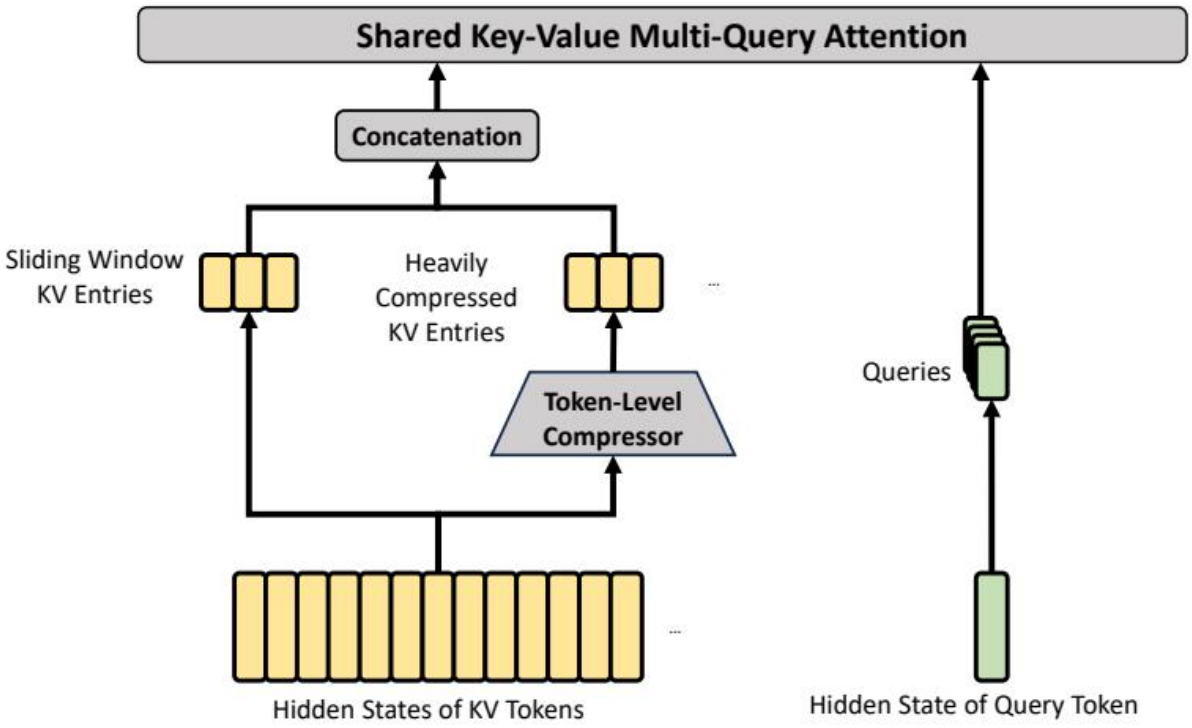

Figure 3: images/8497144997a0443bd87b2320f5d869a60f826fcc6bc4c143b4c52eaa9f6069d7.jpg

Figure 4 | Core architectures of HCA. It performs heavier compression, where the KV entries of $m'(\gg m)$ tokens will be consolidated into one. Also, we additionally introduce a small set of sliding window KV entries to enhance local fine-grained dependencies.

图 4 | HCA 的核心架构。它执行更重的压缩，其中 $m'(\gg m)$ 的键值对将被合并为一个。此外，我们还引入了一小部分滑动窗口键值对，以增强局部细粒度依赖关系。

Shared Key-Value MQA. After selecting the sparse KV entries, CSA then performs core attention in a Multi-Query Attention (MQA) (Shazeer, 2019) manner, where each compressed KV entry in C_t^{SprsComp} serves as both attention key and value. To be specific, for a query token t , we first produce attention queries $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \ldots; \mathbf{q}_{t,n_h}\}$ from the compressed latent vector $\mathbf{c}_t^Q$:

共享密钥-值 MQA。在选择稀疏的 KV 条目后，CSA 以多查询注意力（MQA）（Shazeer, 2019）的方式执行核心注意力，其中 C_t^{SprsComp} 中的每个压缩 KV 条目同时作为注意力键和值。具体来说，对于查询标记 t，我们首先从压缩的潜在向量 $\mathbf{c}_t^Q$ 生成注意力查询 $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; ...; \mathbf{q}_{t,n_h}\}$：

$$\left[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\right] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \tag{18}$$

where n_h denotes the number of query heads; $W^{UQ} \in R^{d_c \times cn_h}$ is the up-projection matrices for queries. Note that the latent query vector c_t^Q is shared with that used for the indexer queries. Next, we perform MQA on $\{q_{t,i}\}$ and $C_t^{SprsComp}$:

其中 n_h 表示查询头的数量；$W^{UQ} \in R^{d_c \times cn_h}$ 是查询的上投影矩阵。请注意，潜在的查询向量 c_t^Q 与用于索引器查询的向量共享。接下来，我们在 $\{q_{t,i}\}$ 和 $C_t^{SprsComp}$ 上执行 MQA：

$$\mathbf{o}_{t,i} = \text{CoreAttn}\left(\text{query} = \mathbf{q}_{t,i}, \text{key} = C_t^{\text{SprsComp}}, \text{value} = C_t^{\text{SprsComp}}\right), \tag{19}$$

where $o_{t,i} \in R^c$ is the core attention output of the i-th head at the t-th token; CoreAttn(·) denotes the core attention operation.

其中 $o_{t,i} \in R^c$ 是第 t 个 token 的第 i 个头的核心注意力输出；CoreAttn(·) 表示核心注意力操作。

Grouped Output Projection. In the configuration of DeepSeek-V4, cn_h is quite large. Therefore, directly projecting the outputs of the core attention operation $[o_{t,1}; o_{t,2}; ...; o_{t,n_h}] = o_t \in R^{cn_h}$ to a d-dimensional hidden state will impose a substantial computational burden. To mitigate this cost, we design a grouped output projection strategy. To be specific, we first split n_h outputs into g groups, and then for each group of output $o_{t,i}^G \in R^{c\frac{n_h}{g}}$, we project it to a d_g -dimensional intermediate output $o_{t,i}^{G'} \in R^{d_g}$, where $d_g < c\frac{n_h}{g}$. Finally, we project the intermediate output $[o_{t,1}^{G'}; o_{t,2}^{G'}; ...; o_{t,g}^{G'}] \in R^{d_g g}$ to the final attention output $\hat{o}_t \in R^d$.

分组输出投影。在 DeepSeek-V4 的配置中，cn_h 非常大。因此，直接将核心注意力操作 $[o_{t,1}; o_{t,2}; ...; o_{t,n_h}] = o_t \in R^{cn_h}$ 的输出投影到 d 维的隐藏状态将带来显著的计算负担。为了缓解这一成本，我们设计了一种分组输出投影策略。具体来说，我们首先将 n_h 的输出分成 g 个组，然后对于每个组的输出 $o_{t,i}^G \in R^{c\frac{n_h}{g}}$，将其投影到一个 d_g 维的中间输出 $o_{t,i}^{G'} \in R^{d_g}$，其中 $d_g < c\frac{n_h}{g}$。最后，我们将中间输出 $[o_{t,1}^{G'}; o_{t,2}^{G'}; ...; o_{t,g}^{G'}] \in R^{d_g g}$ 投影到最终的注意力输出 $\hat{o}_t \in R^d$。

2.3.2. Heavily Compressed Attention

2.3.2. 高压缩注意力

The core architecture of HCA is illustrated in Figure 4, which compresses the KV cache in a heavier manner, but does not employ sparse attention.

HCA 的核心架构如图 4 所示，其以更重的方式压缩 KV 缓存，但并未采用稀疏注意力。

Compressed Key-Value Entries. By and large, the compression strategy of HCA is similar to that of CSA, but employs a larger compression rate $m'(\gg m)$ and does not perform overlapped

压缩的键值条目。总体而言，HCA 的压缩策略与 CSA 类似，但采用了更高的压缩率 $m'(\gg m)$，并且不进行重叠

compression. Let $H \in \mathbb{R}^{n \times d}$ be a sequence of input hidden states, HCA first computes the original KV entries $C \in \mathbb{R}^{n \times c}$ and their corresponding compression weights $Z \in \mathbb{R}^{n \times c}$:

压缩。设 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态的序列，HCA 首先计算原始的 KV 条目 $C \in \mathbb{R}^{n \times c}$ 及其对应的压缩权重 $Z \in \mathbb{R}^{n \times c}$ ：

$$C = H \cdot W^{KV}, \tag{20}$$

$$Z = H \cdot W^{Z}, \tag{21}$$

where $W^{KV}, W^{Z} \in \mathbb{R}^{d \times c}$ are trainable parameters. Next, each m' KV entries in C will be compressed into one according to the compression weights and learnable positional biases $B \in \mathbb{R}^{m' \times c}$, producing $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$. Each compressed entry $C_i^{\text{Comp}} \in \mathbb{R}^{c}$ is computed by

其中 $W^{KV}, W^{Z} \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来，每个 m' 的 KV 条目在 C 中将根据压缩权重和可学习的位置偏差 $B \in \mathbb{R}^{m' \times c}$ 进行压缩，生成 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$ 。每个压缩后的条目 $C_i^{\text{Comp}} \in \mathbb{R}^{c}$ 是通过以下方式计算的：

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B), \tag{22}$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j. \tag{23}$$

Through this compression operation, HCA compresses the sequence length to $\frac{1}{m'}$ times.

通过这种压缩操作，HCA 将序列长度压缩到 $\frac{1}{m'}$ 倍。

Shared Key-Value MQA and Grouped Output Projection. HCA also employs the shared KV MQA and grouped output projection strategies as CSA does. After the KV compression, for a query token t, HCA first produces attention queries $\{q_{t,1}; q_{t,2}; ...; q_{t,n_h}\}$ in a low-rank manner:

共享的键值 MQA 和分组输出投影。HCA 也采用了与 CSA 相同的共享 KV MQA 和分组输出投影策略。在 KV 压缩之后，对于一个查询标记 t，HCA 首先以低秩的方式生成注意力查询 $\{q_{t,1}; q_{t,2}; ...; q_{t,n_h}\}$：

$$\mathbf{c}_t^{Q} = \mathbf{h}_t \cdot W^{DQ}, \tag{24}$$

$$\left[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; ...; \mathbf{q}_{t,n_h}\right] = \mathbf{q}_t = \mathbf{c}_t^{Q} \cdot W^{UQ}, \tag{25}$$

where $h_t \in R^d$ is the input hidden state of the query token t; n_h denotes the number of query heads; $W^{DQ} \in R^{d \times d_c}$ and $W^{UQ} \in R^{d_c \times cn_h}$ are the down-projection and up-projection matrices for queries, respectively. Next, we perform MQA on $\{q_{t,i}\}$ and C^{Comp} :

其中 $h_t \in R^d$ 是查询标记 t 的输入隐藏状态；n_h 表示查询头的数量；$W^{DQ} \in R^{d \times d_c}$ 和 $W^{UQ} \in R^{d_c \times cn_h}$ 分别是查询的下投影和上投影矩阵。接下来，我们在 $\{q_{t,i}\}$ 和 C^{Comp} 上执行 MQA：

$$\mathbf{o}_{t,i} = \text{CoreAttn}\left(\text{query} = \mathbf{q}_{t,i}, \text{key} = C^{\text{Comp}}, \text{value} = C^{\text{Comp}}\right), \tag{26}$$

where $\mathbf{o}_{t,i} \in \mathbb{R}^c$ is the core attention output of the i -th head at the t -th token. Next, as CSA does, HCA splits n_h outputs into g groups, and for each group of output $\mathbf{o}^G_{t,i} \in \mathbb{R}^{c\frac{n_h}{g}}$, HCA projects it to a d_g -dimensional intermediate output $\mathbf{o}^{G'}_{t,i} \in \mathbb{R}^{d_g}$, where $d_g < c\frac{n_h}{g}$. Finally, HCA projects the intermediate output $[\mathbf{o}^{G'}_{t,1}; \mathbf{o}^{G'}_{t,2}; ...; \mathbf{o}^{G'}_{t,g}] \in \mathbb{R}^{d_g g}$ to the final attention output $\hat{\mathbf{o}}_t \in \mathbb{R}^d$.

其中 $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 是第 t 个 token 的第 i 个头的核心注意力输出。接下来，如同 CSA（可能指某种机制或模型）一样，HCA 将 n_h 的输出划分为 g 个组，对于每个组的输出 $\mathbf{o}^G_{t,i} \in \mathbb{R}^{c\frac{n_h}{g}}$ ，HCA 将其投影到一个 d_g 维的中间输出 $\mathbf{o}^{G'}_{t,i} \in \mathbb{R}^{d_g}$ ，其中 $d_g < c\frac{n_h}{g}$ 。最后，HCA 将中间输出 $[\mathbf{o}^{G'}_{t,1}; \mathbf{o}^{G'}_{t,2}; ...; \mathbf{o}^{G'}_{t,g}] \in \mathbb{R}^{d_g g}$ 投影到最终的注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.3. Other Details

2.3.3. 其他细节

In addition to the core architectures of CSA and HCA described above, our hybrid attention incorporates several other techniques. For writing clarity, we omit these additional techniques from the above introduction and will briefly describe them in this subsection. Also, this subsection focuses only on the core ideas of them and may omit some tiny details for simplicity. We encourage the readers to refer to our open-source implementation for unambiguous details.

除了上述描述的 CSA 和 HCA 的核心架构外，我们的混合注意力机制还结合了其他几种技术。为了写作清晰，我们在上述介绍中省略了这些额外的技术，将在本小节中简要描述它们。此外，本小节仅关注这些技术的核心思想，为了简化可能省略一些细小的细节。我们鼓励读者参考我们的开源实现以获得更明确的细节。

Query and Key-Value Entry Normalization. For both CSA and HCA, we perform an additional RMSNorm operation on each head of the queries and the only head of the compressed KV entries, just before the core attention operation. This normalization avoids exploding attention logits and may improve training stability.

查询和键值条目归一化。对于 CSA 和 HCA，我们在核心注意力操作之前，对查询的每个头以及压缩 KV 条目唯一的头执行额外的 RMSNorm 操作。这种归一化可以避免注意力对数的爆炸，并可能提高训练的稳定性。

Partial Rotary Positional Embedding. For both CSA and HCA, we partially employ the Rotary Positional Embedding (RoPE) (Su et al., 2024) to the attention queries, KV entries, and the core attention outputs. To be specific, for each query vector and KV entry vector used in CSA and HCA, we apply RoPE to its last 64 dimensions. Since the KV entries serve as both attention keys and values, the naive core attention outputs $\{\mathbf{o}_{t,i}\}$ will carry absolute position embeddings, derived from the weighted sum of KV entries. As a countermeasure, we also apply RoPE with position $-i$ on the last 64 dimensions of each $\mathbf{o}_{t,i}$. In this way, the output of the core attention will also carry relative position embeddings — the contribution of each KV entry to the core attention outputs will also be related to the distance between the query and the KV entry.

部分旋转位置嵌入。对于 CSA 和 HCA，我们部分地将旋转位置嵌入（RoPE）（Su 等，2024）应用于注意力查询、KV 条目和核心注意力输出。具体而言，对于 CSA 和 HCA 中使用的每个查询向量和 KV 条目向量，我们对其最后的 64 个维度应用 RoPE。由于 KV 条目同时作为注意力键和值，因此原始的核心注意力输出 $\{\mathbf{o}_{t,i}\}$ 将携带绝对位置嵌入，这些嵌入来源于 KV 条目的加权和。作为一种对策，我们也在每个 $\mathbf{o}_{t,i}$ 的最后 64 个维度上应用带有位置 $-i$ 的 RoPE。这样，核心注意力的输出也将携带相对位置嵌入——每个 KV 条目对核心注意力输出的贡献也将与查询和 KV 条目之间的距离相关。

Additional Branch of Sliding Window Attention. In order to strictly preserve causality in CSA and HCA, each query attends to only preceding compressed KV blocks. Consequently, a query cannot access information from other tokens within its own compressed block. Meanwhile, recent tokens usually possess greater relevance to the query token in language modeling. For these reasons, we introduce a supplementary attention branch to both CSA and HCA in a sliding window manner, for better modeling of local dependencies. To be specific, for each query token, we additionally produce n_{win} uncompressed KV entries corresponding to the recent n_{win} tokens. In the core attention of CSA and HCA, these KV entries in the sliding window will be used along with the compressed KV entries.

滑动窗口注意力的附加分支。为了严格保持 CSA 和 HCA 中的因果性，每个查询只能关注前面的压缩 KV 块。因此，查询无法访问其自身压缩块内其他标记的信息。同时，在语言建模中，最近的标记通常与查询标记的相关性更高。基于这些原因，我们以滑动窗口的方式为 CSA 和 HCA 引入一个附加的注意力分支，以更好地建模局部依赖关系。具体而言，对于每个查询标记，我们额外生成对应于最近 n_{win} 个标记的 n_{win} 个未压缩的 KV 条目。在 CSA 和 HCA 的核心注意力中，这些滑动窗口中的 KV 条目将与压缩的 KV 条目一起使用。

Attention Sink. In the core attention of CSA and HCA, we employ the trick of attention sink (OpenAI, 2025; Xiao et al., 2024). To be specific, we set a series of learnable sink logits $\{z'_1, z'_2, ..., z'_{n_h}\}$. For the h -th attention head, $\text{Exp}(z'_h)$ will be added to the denominator of the attention score:

注意力汇聚。在 CSA 和 HCA 的核心注意力中，我们采用了注意力汇聚（attention sink）的技巧（OpenAI，2025；Xiao 等，2024）。具体而言，我们设置了一系列可学习的汇聚 logits $\{z'_1, z'_2, ..., z'_{n_h}\}$。对于第 h 个注意力头，$\text{Exp}(z'_h)$ 将被添加到注意力分数的分母中：

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \tag{27}$$

where $s_{h,i,j}, z_{h,i,j} \in R$ denote the attention score and attention logit of the h-th attention head between the i-th query token and the j-th preceding token or compressed block. This technique allows each query head to adjust its total attention scores to be not equal to 1, and even to be near 0.

其中 $s_{h,i,j}, z_{h,i,j} \in R$ 表示第 i 个查询词与第 j 个先前词或压缩块之间第 h 个注意力头的注意力得分和注意力对数。这种技术使得每个查询头可以调整其总注意力得分，使其不等于 1，甚至接近 0。

2.3.4. Efficiency Discussion

2.3.4. 效率讨论

Due to the employment of hybrid CSA and HCA, together with low-precision computation and storage, the attention module of DeepSeek-V4 series achieves remarkable efficiency in both attention FLOPs and KV cache size, especially in long-context scenarios. First, we adopt a mixed storage format for KV entries: BF16 precision is used for the rotary positional embedding (RoPE) dimensions, while FP8 precision is applied to the remaining dimensions. This hybrid representation reduces the KV cache size by nearly half compared with pure BF16 storage. Second, attention computation within the lightning indexer is performed in FP4 precision, which accelerates the attention operation under extremely long contexts. Third, relative to DeepSeek-V3.2, a smaller attention top-k is chosen in DeepSeek-V4 series, thereby improving model efficiency on short- and medium-length texts. Finally, and most importantly, compressed attention and hybrid attention techniques substantially reduce both the KV cache size and the computational FLOPs.

由于采用了混合的 CSA 和 HCA，结合低精度计算和存储，DeepSeek-V4 系列的注意力模块在注意力 FLOPs 和 KV 缓存大小方面实现了显著的效率提升，尤其是在长上下文场景中。首先，我们采用了一种混

合的 KV 条目存储格式：旋转位置嵌入（RoPE）维度使用 BF16 精度，其余维度则使用 FP8 精度。这种混合表示方式相比纯 BF16 存储，将 KV 缓存大小减少了将近一半。其次，在闪电索引器中进行的注意力计算采用 FP4 精度，这在极长上下文条件下加速了注意力操作。第三，与 DeepSeek-V3.2 相比，DeepSeek-V4 系列选择了较小的注意力 top-k，从而提高了在短文本和中等长度文本上的模型效率。最后，而且最为重要的是，压缩注意力和混合注意力技术显著减少了 KV 缓存大小和计算 FLOPs。

Taking BF16 GQA8 (Ainslie et al., 2023) with a head dimension of 128 as the baseline — one of the common configurations of LLM attention — the KV cache size of DeepSeek-V4 series can be dramatically reduced to approximately 2% times of that baseline in the 1M-context setting.

以头维度为 128 的 BF16 GQA8 (Ainslie 等，2023) 作为基准——这是 LLM 注意力的一种常见配置——在 1M 上下文设置下，DeepSeek-V4 系列的 KV 缓存大小可以大幅减少到基准值的约 2%。

Algorithm 1 Muon Optimizer for DeepSeek-V4

算法 1 DeepSeek-V4 的 Muon 优化器

Require: Learning rate η, momentum μ, weight decay λ, update rescaling factor γ for each training step t do for each logically independent weight $W \in \mathbb{R}^{n\times m}$ do $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$ □ Compute gradients $M_t = \mu M_{t-1} + G_t$ □ Accumulate momentum buffer $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$ □ Nesterov trick and hybrid Newton-Schulz $O_t = O'_t \cdot \sqrt{\max(n,m)} \cdot \gamma$ □ Rescale the update RMS $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$ □ Perform weight decay and update end for end for

要求：学习率 η, 动量 μ, 权重衰减 λ, 更新重标因子 γ 对于每个训练步骤 t 做对于每个逻辑上独立的权重 $W \in \mathbb{R}^{n\times m}$ 做 $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$ □ 计算梯度 $M_t = \mu M_{t-1} + G_t$ □ 积累动量缓冲区 $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$ □ Nesterov 技巧和混合 Newton-Schulz $O_t = O'_t \cdot \sqrt{\max(n,m)} \cdot \gamma$ □ 重标更新 RMS $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$ □ 执行权重衰减和更新结束循环结束循环

Moreover, even when compared with DeepSeek-V3.2 (DeepSeek-AI, 2025) — already an efficient baseline — DeepSeek-V4 series still exhibits substantial advantages in efficiency. A comparison of their inference FLOPs and KV cache size is provided in the right part of Figure 1.

此外，即使与已经是一个高效基线的 DeepSeek-V3.2（DeepSeek-AI，2025）相比，DeepSeek-V4 系列在效率方面仍然表现出显著的优势。图 1 的右侧部分提供了它们的推理 FLOPs 和 KV 缓存大小的对比。

2.4. Muon Optimizer

2.4. Muon 优化器

We employ the Muon (Jordan et al., 2024; Liu et al., 2025) optimizer for the majority of modules in DeepSeek-V4 series due to its faster convergence and improved training stability. The full algorithm of our Muon optimization is summarized in Algorithm 1.

我们采用 Muon（Jordan 等，2024；Liu 等，2025）优化器用于 DeepSeek-V4 系列中的大多数模块，因为它具有更快的收敛速度和改进的训练稳定性。我们的 Muon 优化算法的完整算法总结在算法 1 中。

Basic Configurations. We maintain the AdamW (Loshchilov and Hutter, 2017) optimizer for the embedding module, the prediction head module, the static biases and gating factors of mHC modules, and the weights of all RMSNorm modules. All other modules are updated with Muon. Following Liu et al. (2025), we also apply weight decay to Muon parameters, use the Nesterov (Jordan et al., 2024; Nesterov, 1983) trick, and rescale the Root Mean Square (RMS) of the update matrix for reutilization of our AdamW hyper-parameters. Different from them, we use hybrid Newton-Schulz iterations for orthogonalization.

基础配置。我们为嵌入模块、预测头模块、mHC 模块的静态偏置和门控因子以及所有 RMSNorm 模块的权重维护 AdamW（Loshchilov 和 Hutter, 2017）优化器。所有其他模块则使用 Muon 进行更新。根据 Liu 等人（2025）的方法，我们还对 Muon 参数应用权重衰减，使用 Nesterov（Jordan 等人，2024；Nesterov, 1983）技巧，并重新调整更新矩阵的均方根（RMS）以重用我们的 AdamW 超参数。与他们不同的是，我们使用混合 Newton-Schulz 迭代来进行正交化。

Hybrid Newton-Schulz Iterations. For a given matrix M, let its Singular Value Decomposition (SVD) be $M = U\Sigma V^T$. The Newton-Schulz iterations aim to approximately orthogonalize M to be UV^T . Usually, M will be first normalized as $M_0 = M/||M||_F$ to ensure its maximum singular value does not exceed 1. Then, each Newton-Schulz iteration performs the following operation:

混合牛顿-舒尔茨迭代。对于给定的矩阵 M，其奇异值分解（SVD）为 $M = U\Sigma V^T$ 。牛顿-舒尔茨迭代旨在近似正交化 M，使其成为 UV^T 。通常，M 会首先被归一化为 $M_0 = M/||M||_F$ 以确保其最大奇异值不超过 1。然后，每次牛顿-舒尔茨迭代执行以下操作：

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1}. \tag{28}$$

Our hybrid Newton-Schulz performs 10 iterations over two distinct stages. During the first 8 steps, we use coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ to drive rapid convergence, bringing the singular values close to 1. In the final 2 steps, we switch to coefficients $(a, b, c) = (2, -1.5, 0.5)$, which stabilize the singular values precisely at 1.

我们的混合 Newton-Schulz 方法在两个不同的阶段进行 10 次迭代。在前 8 步中，我们使用系数 $(a, b, c) = (3.4445, -4.7750, 2.0315)$ 来实现快速收敛，使奇异值接近 1。在最后 2 步中，我们切换到系数 $(a, b, c) = (2, -1.5, 0.5)$，使奇异值精确地稳定在 1。

Avoiding Exploding Attention Logits. The attention architecture of DeepSeek-V4 series allows us to directly apply RMSNorm on the attention queries and KV entries, which effectively prevents attention logits from exploding. Consequently, we do not employ the QK-Clip technique (Liu et al., 2025) in our Muon optimizer.

避免注意力对数输出爆炸。DeepSeek-V4 系列的注意力架构使我们能够直接对注意力查询和 KV 条目应用 RMSNorm，这有效防止了注意力对数输出的爆炸。因此，在我们的 Muon 优化器中，我们没有采用 QK-Clip 技术（Liu 等，2025）。

3. General Infrastructures

3. 通用基础设施

3.1. Fine-Grained Communication-Computation Overlap in Expert Parallelism

3.1. 在专家并行中的细粒度通信-计算重叠

Mixture-of-Experts (MoE) can be accelerated via Expert Parallelism (EP). However, EP requires complex inter-node communication and imposes substantial demands on interconnect bandwidth and latency. To alleviate the communication bottleneck in EP and achieve higher end-to-end performance under lower interconnection bandwidth requirements, we propose a fine-grained EP scheme that fuses communication and computation into a single pipelined kernel for communication-computation overlapping.

专家混合（Mixture-of-Experts, MoE）可以通过专家并行（Expert Parallelism, EP）进行加速。然而，

EP 需要复杂的节点间通信，并对互连带宽和延迟提出了较高的要求。为了解决 EP 中的通信瓶颈，并在较低的互连带宽要求下实现更高的端到端性能，我们提出了一种细粒度的 EP 方案，将通信与计算融合到一个单一的流水线内核中，实现通信与计算的重叠。

Communication Latency Can Be Hidden. The key insight of our EP scheme is that the communication latency can be effectively hidden beneath computation in MoE layers. As shown in Figure 5, in DeepSeek-V4 series, each MoE layer can be decomposed mainly into four stages: two communication-bound stages, Dispatch and Combine, and two computation-bound stages, Linear-1 and Linear-2. Our profiling reveals that within a single MoE layer, the total time of communication is less than that of the computation. Therefore, after fusing communication and computation into a unified pipeline, computation remains the dominant bottleneck, implying that the system can tolerate lower interconnect bandwidth without degrading end-to-end performance.

通信延迟可以被隐藏。我们 EP 方案的关键洞察是：通信延迟可以有效地在 MoE 层的计算中被隐藏。如图 5 所示，在 DeepSeek-V4 系列中，每个 MoE 层主要可以分解为四个阶段：两个通信受限的阶段，即调度（Dispatch）和组合（Combine），以及两个计算受限的阶段，即线性 1（Linear-1）和线性 2（Linear-2）。我们的分析结果显示，在单个 MoE 层中，通信的总时间少于计算时间。因此，将通信和计算融合到统一的流水线后，计算仍然是主要的瓶颈，这意味着系统可以在不降低端到端性能的情况下容忍较低的互联带宽。

(a) Naive Solution

(a) 暴力解法

Figure 4: images/8b23e5557505eb2dc117954766f0060293fee2c710a1d990bd4148b2c9cc2f57.jpg

(b) Comet

(b) 彗星

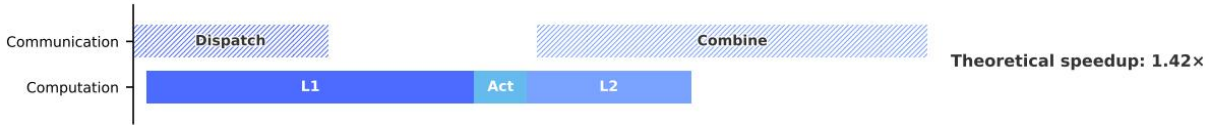

Figure 5: images/9ac831b408a4a8226e285255fc054425e989253c02f269d33e2233ac15d8a013.jpg

(c) Ours

(c) 我们的

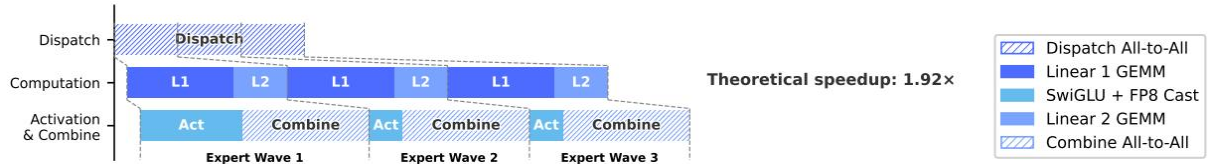

Figure 6: images/c94953ca952c11c3d35634d70a0d152a6fc3974858891a3ff6d61e32916f9218.jpg

Figure 5 | Illustration of our EP scheme with related works. Comet (Zhang et al., 2025b) overlaps

Dispatch with Linear-1, and Linear-2 with Combine, separately. Our EP scheme achieves a finer-grained overlapping by splitting and scheduling experts into waves. The theoretical speedup is evaluated in the configuration of the DeepSeek-V4-Flash architecture.

图 5 | 我们的 EP 方案与相关工作的对比示意图。Comet（Zhang 等人，2025b）将 Dispatch 与 Linear-1 重叠，将 Linear-2 与 Combine 重叠。我们的 EP 方案通过将专家拆分为波浪形式并进行调度，实现了更细粒度的重叠。理论上的加速效果在 DeepSeek-V4-Flash 架构配置中进行了评估。

Fine-Grained EP Scheme. To further lower the interconnect bandwidth requirement and amplify the benefits of overlapping, we introduce a finer-grained expert partitioning scheme. Inspired by many related works (Aimuyo et al., 2025; Zhang et al., 2025b), we split and schedule the experts into waves. Each wave consists of a small portion of experts. As soon as all experts within the wave have completed their communication, computation can commence immediately without waiting for other experts. In steady state, computation of current wave, token transfer for the next wave, and result sending of completed experts all proceed concurrently, as demonstrated in Figure 5. This forms a fine-grained pipeline among experts, keeping both computation and communication continuous throughout the wave. The wave-based scheduling speeds up the

细粒度的专家方案。为了进一步降低互连带宽需求并放大重叠带来的好处，我们引入了一种更细粒度的专家划分方案。受许多相关工作的启发（Aimuyo 等，2025；Zhang 等，2025b），我们将专家划分并安排成波浪形式。每个波浪包含一小部分专家。一旦波浪中的所有专家完成通信，计算可以立即开始，而无需等待其他专家。在稳定状态下，当前波浪的计算、下一波浪的令牌传输以及已完成专家的结果发送同时进行，如图 5 所示。这在专家之间形成了细粒度的流水线，使得在整个波浪过程中计算和通信都能持续进行。基于波浪的调度加快了

performance on extreme cases such as Reinforcement Learning (RL) rollout, which usually encounters long-tail small batches.

在极端情况下的表现，例如强化学习（RL）rollout，通常会遇到长尾小批量的问题。

Performance and Open-Sourced Mega-Kernel. We validated the fine-grained EP scheme on both NVIDIA GPUs and HUAWEI Ascend NPUs platforms. Compared against strong non-fused baselines, it achieves $1.50 \sim 1.73\times$ speedup for general inference workloads, and up to $1.96\times$ for latency-sensitive scenarios such as RL rollouts and high-speed agent serving. We have open-sourced the CUDA-based mega-kernel implementation named MegaMoE [2] as a component of DeepGEMM.

性能与开源的巨核。我们在 NVIDIA GPU 和华为昇腾 NPUs 平台上验证了细粒度的 EP 方案。与强非融合基线相比，它在通用推理工作负载中实现了 $1.50 \sim 1.73\times$ 的加速效果，在对延迟敏感的场景（如强化学习 rollouts 和高速代理服务）中，加速效果高达 $1.96\times$。我们已将基于 CUDA 的巨核实现，名为 MegaMoE [2]，作为 DeepGEMM 的一部分开源。

Observations and Proposals. We share observations and lessons from kernel development and offer some proposals to hardware vendors, in the hope of aiding efficient hardware design and achieving better software-hardware co-design:

观察与建议。我们分享来自内核开发的观察和经验，并向硬件厂商提出一些建议，希望能帮助实现高效硬件设计，并取得更好的软硬件协同设计效果：

- Computation-Communication Ratio. Full communication-computation overlap hinges on the computation-communication ratio, rather than the bandwidth solely. Denoting peak compute throughput as C and interconnect bandwidth as B , communication can be fully hidden when $C/B \leqslant V_{\text{comp}}/V_{\text{comm}}$, where V_{comp} denotes the computation volume and V_{comm} refers to the communication volume. For DeepSeek-V4-Pro, where each token-expert pair requires 6hd FLOPs (SwiGLU gate, up, and down projections) but only 3h bytes of communication (FP8 Dispatch + BF16 Combine), this simplifies to:

  - 计算-通信比率。完全的通信-计算重叠取决于计算-通信比率，而不是仅依赖带宽。设峰值计算吞吐

量为 C，互连带宽为 B，当 $C/B \leqslant V_{\text{comp}}/V_{\text{comm}}$ 时，通信可以完全隐藏，其中 V_{comp} 表示计算量，V_{comm} 表示通信量。对于 DeepSeek-V4-Pro，每个 token-expert 对需要 6hd FLOPs（SwiGLU gate、up 和 down 投影），但仅需要 3h 字节的通信（FP8 分发 + BF16 合并），这简化为：

$$\frac{C}{B} \leqslant 2d = 6144 \text{FLOPs / Byte}.$$

That is, each GBps of interconnect bandwidth suffices to hide the communication for 6.1 TFLOP/s of compute. Once bandwidth meets this threshold, it ceases to be the bottleneck, and devoting additional silicon area to further bandwidth brings diminishing returns. We encourage future hardware designs to target such balance points rather than scale bandwidth unconditionally.

也就是说，每 GBps 的互连带宽足以隐藏 6.1 TFLOP/s 的计算通信。一旦带宽达到这一阈值，它就不再成为瓶颈，而将更多的硅面积用于进一步增加带宽所带来的收益将逐渐减少。我们鼓励未来的硬件设计以达到这种平衡点为目标，而不是无条件地扩展带宽。

- Power Budget. Extreme kernel fusion drives compute, memory, and network to high load simultaneously, making power throttling a key performance limiter. We suggest that future hardware designs provide sufficient power headroom for such fully concurrent workloads.

- Communication Primitives. In the dispatch stage, we adopt a pull-based approach where each GPU actively reads activations from remote GPUs, avoiding the high notification latency that fine-grained push entails. Future hardware with lower-latency cross-GPU signaling would make push viable and enable more natural communication patterns.

- Activation Function. We propose replacing SwiGLU with a low-cost element-wise activation that involves no exponential or division operations. This directly reduces the overhead of post-GEMM processing, preventing the GEMM pipeline from being stalled by activation function computation, thereby enhancing overall computational throughput and resource utilization.

- **功耗预算**。极端内核融合驱动计算、内存和网络同时处于高负载状态，使得功耗限制成为关键的性能瓶颈。我们建议未来的硬件设计为这种完全并发的工作负载提供足够的功耗余量。

- **通信原语**。在调度阶段，我们采用一种基于拉取（pull-based）的方法，每个 GPU 主动从远程 GPU 读取激活值（activations），避免了细粒度推送（push）所导致的高通知延迟。未来具备更低延迟的跨 GPU 通信信号的硬件将使推送变得可行，并能够实现更自然的通信模式。

- **激活函数**。我们建议用一种低成本的逐元素激活函数来替代 SwiGLU，这种激活函数不涉及指数运算或除法运算。这将直接降低后 GEMM 处理的开销，防止 GEMM 流水线因激活函数计算而停滞，从而提升整体计算吞吐量和资源利用率。

3.2. Flexible and Efficient Kernel Development with TileLang

3.2. 使用 TileLang 实现灵活高效的内核开发

In practice, our elaborate model architecture would have resulted in hundreds of fine-grained Torch ATen operators. We adopt TileLang (Wang et al., 2026) to develop a set of fused kernels

实际上，我们的精心设计的模型架构会导致数百个细粒度的 Torch ATen 操作符。我们采用 TileLang（Wang 等，2026）来开发一组融合内核

to replace the vast majority of them, delivering optimal performance with minimal effort. It also allows us to quickly prototype operators like attention variants during validation. These kernels play critical roles in model architecture development, large-scale training, and ultimately production deployment of inference services. As a Domain-Specific Language (DSL), TileLang balances development productivity with runtime efficiency, enabling rapid development while supporting deep, iterative optimizations within the same codebase. Additionally, we collaborate closely with the TileLang community to foster a more agile, efficient, and stable kernel development workflow.

为了替换其中的大多数内容，以最小的努力实现最佳性能。它还允许我们在验证过程中快速原型化注意力变体等操作符。这些内核在模型架构开发、大规模训练以及最终推理服务的生产部署中起着关键作用。作为领域特定语言（DSL），TileLang 在开发效率和运行时效率之间取得了平衡，使开发能够快速进行，同时在同一代码库中支持深入的、迭代性的优化。此外，我们与 TileLang 社区紧密合作，以促进更加敏捷、高效和稳定的内核开发流程。

Reducing Invocation Overhead with Host Codegen. As accelerators continue to grow in performance, CPU-side orchestration overhead becomes increasingly prominent. For small, highly optimized kernels, such fixed host overhead can easily cap utilization and throughput. A common source of this overhead is that host-side logic, such as runtime contract checks, is typically written in Python for flexibility and thus incurs a fixed per-invocation cost.

通过主机代码生成减少调用开销。随着加速器在性能上的持续提升，CPU 端的协调开销变得越来越显著。对于小型且高度优化的内核来说，这种固定的主机开销很容易限制利用率和吞吐量。这种开销的一个常见来源是，主机端的逻辑（如运行时契约检查）通常为了灵活性而用 Python 编写，因此每次调用都会产生固定的开销。

We mitigate this overhead with Host Codegen, which moves most host-side logic into generated host code. Specifically, we first co-generate the device kernel and a lightweight host launcher at the IR (Intermediate Representation) level, embedding the necessary metadata—such as data types, rank/shape constraints, and stride/layout assumptions—parsed from the language frontend. The launcher is then lowered to the host source code built on top of the TVM-FFI (Chen et al., 2018) framework, whose compact calling convention and zero-copy tensor interop together minimize host-side overhead. At runtime, this generated host code performs validation and argument marshaling, shifting all per-invocation checks out of the Python execution path. Our measurements show that CPU-side validation overhead drops from tens or hundreds of microseconds to less than one microsecond per invocation.

我们通过 Host Codegen 来减轻这种开销，它将大部分主机端逻辑移动到生成的主机代码中。具体来说，我们首先在 IR（中间表示）级别共同生成设备内核和一个轻量级的主机启动器，嵌入从语言前端解析出的必要元数据—例如数据类型、秩/形状约束以及步幅/布局假设。然后，启动器被降低到基于 TVM-FFI（Chen 等人，2018）框架的主机源代码，其紧凑的调用约定和零拷贝张量互操作性共同最小化了主机端开销。在运行时，生成的主机代码执行验证和参数打包，将所有每次调用的检查移出 Python 执行路径。我们的测量结果显示，CPU 端的验证开销从每次调用几十或几百微秒降至不到一微秒。

SMT-Solver-Assisted Formal Integer Analysis. TileLang kernels involve complex tensor index arithmetic that requires strong formal integer analysis. During compilation passes such as layout inference, memory hazard detection, and bound analysis, the compiler must verify whether integer expressions satisfy specific properties to enable the corresponding optimizations. Therefore, stronger formal analysis capabilities can unlock more advanced and complex optimization opportunities.

SMT-Solver 辅助的正式整数分析。TileLang 内核涉及复杂的张量索引算术，这需要强大的正式整数分析。在布局推断、内存危险检测和边界分析等编译阶段，编译器必须验证整数表达式是否满足特定属性，以启用相应的优化。因此，更强的正式分析能力可以解锁更多先进且复杂的优化机会。

To this end, we integrate the Z3 SMT solver (De Moura and Bjørner, 2008) into TileLang's algebraic system, providing formal analysis capability for most integer expressions in tensor programs. We strike a balance between computational overhead and formal expressiveness by translating TileLang's integer expressions into Z3's quantifier-free non-linear integer arithmetic (QF_NIA).

Based on Integer Linear Programming (ILP) solvers, QF_NIA seamlessly resolves standard linear integer expressions common in kernels. Furthermore, its inherent non-linear reasoning capacity effectively addresses advanced challenges like vectorization over variable tensor shapes. Under reasonable resource limits, Z3 elevates overall optimization performance while restricting compilation time overhead to just a few seconds. The impact is substantial across multiple passes, including vectorization, barrier insertion, and code simplification.

为此，我们将 Z3 SMT 求解器（De Moura 和 Bjørner，2008）集成到 TileLang 的代数系统中，为张量程序中的大多数整数表达式提供形式化分析能力。我们通过将 TileLang 的整数表达式转换为 Z3 的无量词非线性整数算术（QF_NIA），在计算开销和形式表达能力之间取得平衡。基于整数线性规划（ILP）求解器，QF_NIA 无缝地解决在内核中常见的标准线性整数表达式。此外，其内在的非线性推理能力有效应对了诸如在可变张量形状上进行向量化等高级挑战。在合理的资源限制下，Z3 提高了整体优化性能，同时将编译时间开销限制在几秒钟之内。其影响在多个处理阶段中都十分显著，包括向量化、屏障插入和代码简化等。

Numerical Precision and Bitwise Reproducibility. In production settings, numerical correctness and reproducibility are as critical as raw throughput. We therefore prioritize accuracy by default: fast-math optimizations are disabled at the compiler level, and precision-affecting approximations are provided only as explicit, opt-in frontend operators (e.g., T.__exp, T.__log, and T.__sin). Conversely, when strict IEEE-754 semantics are required, TileLang provides

数值精度和位运算可重复性。在生产环境中，数值的正确性和可重复性与原始吞吐量一样重要。因此，我们默认优先保证准确性：编译器级别禁用快速数学优化，精度影响的近似计算仅作为显式、可选的前端运算符提供（例如 T.__exp、T.__log 和 T.__sin）。相反，当需要严格的 IEEE-754 语义时，TileLang 提供

IEEE-compliant intrinsics with explicit rounding modes (e.g., T.ieee_fsqrt, T.ieee_fdiv, and T.ieee_add), enabling developers to precisely specify numerical behavior.

IEEE 合规的内建函数，带有明确的舍入模式（例如 T.ieee_fsqrt、T.ieee_fdiv 和 T.ieee_add），使开发人员能够精确指定数值行为。

We also target bitwise reproducibility for validating kernels against hand-written CUDA baselines. We align TileLang's algebraic simplification and lowering rules with mainstream CUDA toolchains (e.g., NVCC) to avoid transformations that introduce unintended bit-level differences. Layout annotations (e.g., T. annotate_layout) further allow users to pin down layout-dependent lowering decisions, keeping evaluation and accumulation order consistent with the reference CUDA implementation and thus enabling bit-identical outputs when desired.

我们还针对位级可重复性进行目标设定，以验证内核与手写 CUDA 基准之间的对比。我们将 TileLang 的代数简化和降低规则与主流 CUDA 工具链（例如 NVCC）对齐，以避免引入意外位级差异的转换。布局注释（例如 T. annotate_layout）进一步允许用户确定与布局相关的降低决策，保持评估和累积顺序与参考 CUDA 实现一致，从而在需要时实现完全相同的位输出。

Our evaluation shows that these accuracy- and reproducibility-oriented design choices do not sacrifice performance: under conservative defaults, TileLang kernels remain competitive, while exposing knobs to selectively relax numerical constraints for higher speed.

我们的评估表明，这些以准确性和可重复性为导向的设计选择并不会牺牲性能：在保守默认设置下，TileLang 内核仍然具有竞争力，同时通过暴露旋钮，可以选择性地放松数值约束以获得更高的速度。

3.3. High-Performance Batch-Invariant and Deterministic Kernel Libraries

3.3. 高性能批处理不变和确定性内核库

To enable efficient training and inference, we develop a comprehensive set of high-performance computational kernels. Beyond basic functionalities and maximizing hardware utilization, another pivotal design goal is to ensure training reproducibility and bitwise alignment among pre-

training, post-training, and inference pipelines. Therefore, we implement end-to-end, bitwise batch-invariant, and deterministic kernels with minimal performance overhead. These kernels are helpful for debugging, stability analysis, and consistent post-training behavior.

为了实现高效的训练和推理，我们开发了一套全面的高性能计算内核。除了基本功能和最大化硬件利用率之外，另一个关键的设计目标是确保预训练、后训练和推理流程之间的训练可重复性和位级对齐。因此，我们实现了端到端、位级批处理不变性和确定性的内核，且性能开销最小。这些内核有助于调试、稳定性分析以及后训练行为的一致性。

Batch Invariance. Batch invariance ensures that the output of any given token remains bitwise identical, regardless of its position within a batch. To implement batch invariance, the primary challenges are listed as follows:

批量不变性。批量不变性确保任何给定标记的输出在批量中的位置无关，位级完全相同。为了实现批量不变性，主要挑战如下：

- Attention. To achieve batch invariance, we cannot use the split-KV method (Dao et al., 2023), which distributes the attention computation for a single sequence across multiple Stream Multiprocessors (SMs) to balance the load of SMs. However, abandoning this technique will lead to severe wave-quantization problems [3], which can adversely affect GPU utilization. To address this, we develop a dual-kernel strategy for batch-invariant decoding. The first kernel computes the attention output for an entire sequence within a single SM, ensuring high throughput for fully occupied waves. The second kernel, to minimize the latency of the final partially-filled wave and thus alleviate wave-quantization, uses multiple SMs for a single sequence. For the bitwise identity of these two kernels, we carefully design the calculation path of the second kernel to ensure its accumulation order is the same as that of the first kernel. Additionally, the second kernel utilizes distributed shared memory [4] within thread-block clusters, enabling high-speed data exchange across SMs. This dual-kernel method effectively confines the overhead of batch-invariant decoding to be negligible.

  - 注意。为了实现批次不变性，我们不能使用 split-KV 方法（Dao 等，2023），该方法将单个序列的注意力计算分布到多个流多处理器（SMs）上，以平衡 SMs 的负载。然而，放弃这项技术会导致严重的波量化问题 [3]，这会不利地影响 GPU 的利用率。为了解决这个问题，我们开发了一种用于批次不变解码的双内核策略。第一个内核在一个 SM 内计算整个序列的注意力输出，确保完全占用的波具有高吞吐量。第二个内核为了最小化最终部分填充波的延迟并从而缓解波量化，使用多个 SM 处理单个序列。为了这两个内核的位级一致性，我们精心设计第二个内核的计算路径，确保其累加顺序与第一个内核的累加顺序相同。此外，第二个内核利用线程块簇内的分布式共享内存 [4]，实现了 SM 之间的高速数据交换。这种双内核方法有效地将批次不变解码的开销限制到可以忽略不计的程度。

- Matrix Multiplication. Traditional cuBLAS library (NVIDIA Corporation, 2024) cannot achieve batch invariance. Therefore, we replace it end-to-end with DeepGEMM (Zhao et al., 2025). Furthermore, for very small batch sizes, conventional implementation usually employs split-k (Osama et al., 2023) techniques to improve performance. Unfortunately, split-k techniques cannot guarantee batch invariance, a pivotal feature in DeepSeek-V4.

  - 矩阵乘法。传统的 cuBLAS 库（NVIDIA Corporation, 2024）无法实现批次不变性。因此，我们将其端到端地替换为 DeepGEMM（Zhao 等，2025）。此外，对于非常小的批次大小，常规实现通常采用 split-k（Osama 等，2023）技术来提升性能。不幸的是，split-k 技术无法保证批次不变性，而这是 DeepSeek-V4 的一个关键特性。

Therefore, we abandon split-k in most scenarios, which, however, may cause performance degradation. To address this, we introduce a set of optimizations that enable our implementation of matrix multiplication to match or even surpass the performance of standard split-k in most major scenarios.

因此，我们大多数情况下放弃 split-k，这可能会导致性能下降。为了解决这个问题，我们引入了一组优化措施，使我们的矩阵乘法实现能够达到甚至超越标准 split-k 在大多数主要场景中的性能。

Determinism. Deterministic training is highly beneficial for debugging hardware or software issues. Moreover, when training exhibits anomalies such as loss spikes, determinism enables researchers to more easily pinpoint numerical causes and further refine the model design. Non-determinism in training typically stems from non-deterministic accumulation order, often due to the use of atomic addition instructions. This issue primarily occurs during the backward pass, notably at the following parts:

确定性。确定性的训练对于调试硬件或软件问题非常有益。此外，当训练过程中出现异常情况（如损失值突然激增）时，确定性可以使研究人员更容易定位数值原因，并进一步优化模型设计。训练中的非确定性通常源于累加顺序的非确定性，通常由于使用了原子加法指令所致。这个问题主要出现在反向传播过程中，尤其是在以下部分：

- Attention Backward. In conventional implementations of backward propagation for sparse attention, we use atomicAdd to accumulate gradients for the KV tokens. This introduces non-determinism due to the non-associativity of floating-point addition. To address this problem, we allocate separate accumulation buffers for each SM, followed by a global deterministic summation across all buffers.

- MoE Backward. When multiple SMs from different ranks concurrently write data to the same buffer on a receiving rank, negotiating writing positions also introduces non-determinism. To resolve this, we design a token order pre-processing mechanism within each single rank, combined with buffer isolation across multiple ranks. This strategy ensures determinism of both the send results of expert parallelism and the accumulation order in the MoE backward pass.

- Matrix Multiplication in mHC. mHC involves a matrix multiplication with an output dimension of only 24. For very small batch sizes, we are compelled to use the split-k (Osama et al., 2023) algorithm, whose naive implementation will cause non-determinism. To overcome this, we output each split part separately and perform a deterministic reduction in a subsequent kernel, thereby preserving both performance and determinism.

- 注意后向传播。在稀疏注意力的常规实现中，我们使用 atomicAdd 来累积 KV token 的梯度。这会由于浮点加法的非结合性而引入非确定性。为了解决这个问题，我们为每个 SM 分配独立的累加缓冲区，然后在所有缓冲区上进行全局确定性求和。

- MoE 后向传播。当来自不同 rank 的多个 SM 并发地向接收方的同一缓冲区写入数据时，协商写入位置也会引入非确定性。为了解决这个问题，我们在每个单个 rank 内设计一种 token 顺序预处理机制，并结合多个 rank 之间的缓冲区隔离。这种策略确保了专家并行的发送结果以及 MoE 后向传播中的累加顺序的确定性。

- mHC 中的矩阵乘法。mHC 涉及一个输出维度仅为 24 的矩阵乘法。对于非常小的批次大小，我们被迫使用 split-k（Osama 等，2023）算法，其朴素实现会导致非确定性。为了解决这个问题，我们分别输出每个 split 部分，并在后续的内核中进行确定性归约，从而在保持性能的同时确保确定性。

3.4. Training Framework

3.4. 训练框架

Our training framework is built upon the scalable and efficient infrastructure developed for DeepSeek-V3 (DeepSeek-AI, 2024). In training DeepSeek-V4, we inherit this robust foundation while introducing several key innovations to accommodate its novel architectural components — specifically the Muon optimizer, mHC, and the hybrid attention mechanism — while maintaining high training efficiency and stability.

我们的训练框架是基于为 DeepSeek-V3 开发的可扩展且高效的基础设施（DeepSeek-AI, 2024）。在训练 DeepSeek-V4 时，我们继承了这一稳固的基础，同时引入了若干关键创新，以适应其新的架构组件——特别是 Muon 优化器、mHC 以及混合注意力机制——同时保持高训练效率和稳定性。

3.4.1. Efficient Implementation of Muon

3.4.1. 深入实施缪子

The Muon optimizer requires the full gradient matrix to compute parameter updates, which presents a challenge when combined with the Zero Redundancy Optimizer (ZeRO) (Rajbhandari et al., 2020). Traditional ZeRO is designed for element-wise optimizers like AdamW, where a single parameter matrix can be partitioned and updated across multiple ranks. To address this conflict, we design a hybrid strategy of ZeRO bucket assignment for Muon.

Muon 优化器需要完整的梯度矩阵来计算参数更新，这在与零冗余优化器（ZeRO）（Rajbhandari 等，2020）结合使用时带来了挑战。传统的 ZeRO 是为像 AdamW 这样的逐元素优化器设计的，其中单个参数矩阵可以被分割并在多个秩上进行更新。为了解决这一冲突，我们设计了一种针对 Muon 的混合 ZeRO 桶分配策略。

For dense parameters, we limit the maximum size of ZeRO parallelism and employ a knapsack algorithm to assign parameter matrices to these ranks, ensuring each rank manages a roughly balanced load. The bucket on each rank is padded to match the size of the largest bucket across ranks, facilitating efficient reduce-scatter operations. This padding typically incurs less than 10% memory overhead in our setup, where each rank manages no more than five parameter

对于密集参数，我们限制了 ZeRO 并行性的最大规模，并采用背包算法将参数矩阵分配给这些等级，确保每个等级管理的负载大致平衡。每个等级上的桶被填充以匹配跨等级的最大桶大小，从而实现高效的 reduce-scatter 操作。在我们的设置中，这种填充通常带来的内存开销小于 10%，其中每个等级管理的参数不超过五个

matrices. When the overall size of data parallelism exceeds the limit for ZeRO, we compute the Muon update redundantly across the extra data-parallel groups, trading computation for reduced total bucket memory.

矩阵。当数据并行的整体规模超过 ZeRO 的限制时，我们会在额外的数据并行组中冗余地计算 Muon 更新，以换取减少总桶内存的计算开销。

For MoE parameters, we optimize each expert independently. We first flatten all down projection matrices in SwiGLU (Shazeer, 2020) of all experts across all layers, followed by flattened up projection matrices and gate matrices. Then, we pad the flattened vector to ensure we can evenly distribute this vector across all ranks without splitting any logically independent matrix. Given the large number of experts, we do not impose a limit of ZeRO parallelism for MoE parameters, and the padding overhead is also negligible.

对于 MoE 参数，我们独立地优化每个专家。我们首先将所有层中所有专家的 SwiGLU（Shazeer, 2020）的下投影矩阵展平，接着展平上投影矩阵和门控矩阵。然后，我们将展平后的向量进行填充，以确保可以将该向量均匀地分布在所有秩上，而不会分割任何逻辑上独立的矩阵。鉴于专家数量很大，我们不对 MoE 参数施加 ZeRO 并行性的限制，填充的开销也是可以忽略不计的。

Additionally, on each rank, consecutive parameters of identical shape will be automatically merged, enabling batched execution of the Newton-Schulz iterations for better hardware utilization. Furthermore, we observe that the Newton-Schulz iterations in Muon remain stable when computed with BF16 matrix multiplications. Leveraging this, we further quantize, in a stochastic rounding manner, the MoE gradients to be synchronized across data-parallel ranks to the BF16 precision, halving the communication volume. To avoid accumulation errors introduced by low-precision adders, we replace conventional tree- or ring-based reduce-scatter collectives with a two-phase approach. First, an all-to-all operation exchanges local gradients across ranks,

and then each rank performs a local sum in FP32. This design maintains numerical robustness.

此外，在每一层中，形状相同的连续参数将被自动合并，从而实现 Newton-Schulz 迭代的批量执行，以更好地利用硬件资源。此外，我们观察到，当使用 BF16 矩阵乘法计算 Muon 中的 Newton-Schulz 迭代时，其保持稳定。利用这一点，我们进一步以随机舍入的方式对 MoE 梯度进行量化，使其在数据并行的各个 rank 之间以 BF16 精度同步，从而将通信量减半。为了避免低精度加法器引入的累积误差，我们将传统的树状或环形 reduce-scatter 收集操作替换为两阶段方法。首先，一个 all-to-all 操作在各个 rank 之间交换局部梯度，然后每个 rank 在 FP32 中执行局部求和。这种设计保持了数值的鲁棒性。

3.4.2. Cost-Effective and Memory-Efficient Implementation of mHC

3.4.2. mHC 的高效且内存节省的实现方式

The introduction of mHC increases both activation memory consumption and communication volume between pipeline stages, compared with conventional residual connections. To mitigate these costs, we implement several optimization strategies.

引入 mHC 相比于传统的残差连接，会增加流水线阶段之间的激活内存消耗和通信量。为了缓解这些开销，我们实施了多种优化策略。

Firstly, we carefully design and implement fused kernels of mHC for both training and inference. Secondly, we introduce a recomputation strategy that selectively checkpoints intermediate tensors. Specifically, we recompute most hidden states between layers and all normalized layer inputs, while avoiding recomputation of compute-intensive operations. This achieves a balance between memory saving and computational overhead. Thirdly, we adjust the DualPipe 1F1B overlapping scheme to accommodate the increased pipeline communication and enable concurrent execution of some operations in mHC.

首先，我们精心设计并实现了适用于训练和推理的 mHC 融合内核。其次，我们引入了一种选择性检查点中间张量的重计算策略。具体而言，我们重新计算层之间的大部分隐藏状态以及所有归一化层的输入，同时避免重新计算计算密集型操作。这在节省内存和计算开销之间取得了平衡。第三，我们调整了 DualPipe 1F1B 重叠方案，以适应增加的流水线通信，并使 mHC 中的一些操作能够并发执行。

Collectively, these optimizations constrain the wall-time overhead of mHC to only 6.7% of the overlapped 1F1B pipeline stage. More details of the engineering optimization can be found in the dedicated mHC paper (Xie et al., 2026).

总体而言，这些优化将 mHC 的墙时开销限制在重叠 1F1B 流水线阶段的仅 6.7%。有关工程优化的更多细节，可以参考专门的 mHC 论文（Xie 等，2026）。

3.4.3. Contextual Parallelism for Long-Context Attention

3.4.3. 长上下文注意力的上下文并行性

Conventional Context Parallelism (CP) partitions the sequence dimension, with each rank maintaining contiguous s tokens. This introduces two challenges to our compressed attention mechanisms (i.e., CSA and HCA). On the one hand, training samples are packed from multiple sequences, and each sequence is compressed independently by a factor of m (or m'), with any trailing tokens fewer than m being discarded. Consequently, the compressed KV lengths are typically less than $\frac{s}{m}$ and vary across ranks. On the other hand, the compression requires m consecutive KV entries, which may straddle the boundary between two neighboring CP ranks.

常规上下文并行（CP）将序列维度划分为多个部分，每个进程维护连续的 s 个 token。这给我们的压缩注意力机制（即 CSA 和 HCA）带来了两个挑战。一方面，训练样本是从多个序列中打包的，每个序列以因

子 m（或 m'）独立进行压缩，任何尾部 token 数少于 m 的部分都会被丢弃。因此，压缩后的 KV 长度通常小于 $\frac{s}{m}$，并且在不同进程中会有所不同。另一方面，压缩需要 m 个连续的 KV 条目，这些条目可能会跨越两个相邻 CP 进程之间的边界。

To address these challenges, we design a two-stage communication approach. In the first stage, each rank i sends its last m uncompressed KV entries to rank $i+1$. Then, rank $i+1$ compresses some of these received entries together with its local s uncompressed KV entries,

为了解决这些挑战，我们设计了一个两阶段的通信方法。在第一阶段，每个 rank i 将其最后的 m 个未压缩的 KV 条目发送到 rank $i+1$。然后，rank $i+1$ 将接收到的一些条目与本地的 s 个未压缩的 KV 条目一起进行压缩，

producing a fixed length of $\frac{s}{m}+1$ compressed entries, in which exist some padding entries. In the second stage, an all-gather operation across all CP ranks collects the locally compressed KV entries. Then, a fused select-and-pad operator reorganizes them into the full set of compressed KV entries with a total length of cp_size $\cdot \frac{s}{m}$. Any padding entries are placed at the tail. For HCA and the indexer in CSA, the visible range of compressed KV entries for each query token can be precomputed by rules. For the sparse attention in CSA, the top-k selector explicitly specifies the indices of visible compressed KV entries for each query.

生成固定长度的 $\frac{s}{m}+1$ 压缩条目，其中包含一些填充条目。在第二阶段，所有 CP 排列上的全部收集操作收集本地压缩的 KV 条目。然后，融合的选取和填充操作符将它们重新组织为完整的压缩 KV 条目集合，总长度为 cp_size $\cdot \frac{s}{m}$ 。任何填充条目都放置在尾部。对于 HCA 和 CSA 中的索引器，每个查询标记的压缩 KV 条目的可见范围可以通过规则预先计算。对于 CSA 中的稀疏注意力，顶部 k 选择器显式地指定每个查询的可见压缩 KV 条目的索引。

3.4.4. Extended Automatic Differentiation for Flexible Activation Checkpointing

3.4.4. 延伸的自动微分以实现灵活的激活检查点

Conventional activation checkpointing implementations operate at the granularity of an entire module, deciding whether to retain or recompute its output activations during the backward pass. This coarse granularity often leads to suboptimal trade-offs between recomputation cost and activation memory footprint. An alternative approach is to manually implement the forward and backward logic of an entire layer, explicitly managing tensor checkpointing states. While enabling fine-grained control, this method loses the convenience of the automatic differentiation framework, substantially increasing development complexity.

传统的激活检查点（activation checkpointing）实现方法以整个模块为粒度进行操作，在反向传播过程中决定是否保留或重新计算其输出激活值。这种粗粒度的方法通常会导致重新计算成本与激活内存占用之间的权衡不够理想。另一种方法是手动实现整个层的前向和反向逻辑，显式地管理张量检查点状态。虽然这种方法能够实现更细粒度的控制，但会失去自动微分框架的便利性，显著增加开发复杂度。

To achieve fine-grained control without sacrificing programming efficiency, we implement a tensor-level activation checkpointing mechanism with automatic differentiation support. With this mechanism, developers only need to implement the forward pass and selectively annotate individual tensors for automatic checkpointing and recomputation. Our framework leverages TorchFX (Reed et al., 2022) to trace the full computation graph. For each annotated tensor, it performs a backward traversal to identify the minimal subgraph required for its recomputation. We define these minimal subgraphs as recomputation graphs and insert them into the backward logic just before the corresponding gradient computation.

为了在不牺牲编程效率的前提下实现细粒度的控制，我们实现了一种支持自动微分的张量级激活检查点机制。通过这种机制，开发者只需实现前向传播，并选择性地对个别张量进行标注，以实现自动检查点和重新计算。我们的框架利用 TorchFX（Reed 等，2022）来追踪完整的计算图。对于每个标注的张量，它执

行反向遍历以识别其重新计算所需的最小子图。我们将这些最小子图定义为重新计算图，并在相应梯度计算之前将其插入到反向逻辑中。

Compared with the manual implementation, this design introduces no additional overhead during training. Recomputation in this framework is implemented by directly freeing the GPU memory of the annotated tensor and reusing the storage pointer from the recomputed tensor, without any GPU memory copy. Furthermore, since graph tracing executes the model concretely, we can track the underlying storage pointer of each tensor, which enables automatic deduplication of recomputation for tensors that share storage (e.g., the input and output of a reshape operation). This relieves developers from reasoning about low-level memory details when annotating recomputation.

与手动实现相比，该设计在训练过程中不会引入任何额外的开销。在这个框架中，重计算是通过直接释放标注张量的 GPU 内存，并重用重计算张量的存储指针来实现的，而无需进行任何 GPU 内存拷贝。此外，由于图追踪执行模型的具体操作，我们可以跟踪每个张量的底层存储指针，从而实现对共享存储张量（例如重塑操作的输入和输出）的重计算自动去重。这减轻了开发人员在标注重计算时对底层内存细节的思考负担。

3.5. Inference Framework

3.5. 推理框架

Our inference framework largely inherits from that of DeepSeek-V3, with some differences in KV Cache management.

我们的推理框架在很大程度上继承自 DeepSeek-V3，仅在 KV Cache 管理方面存在一些差异。

3.5.1. KV Cache Structure and Management

3.5.1. KV Cache 结构与管理

To efficiently manage the heterogeneous KV caches arising from the hybrid attention mechanism in DeepSeek-V4, we design a customized KV cache layout. The layout is illustrated in Figure 6, and we will elaborate on it in detail as follows.

为了高效管理 DeepSeek-V4 中混合注意力机制所产生的异构 KV 缓存，我们设计了一种定制的 KV 缓存布局。该布局如图 6 所示，我们将详细阐述如下。

Heterogeneous KV Entries in DeepSeek-V4. The hybrid attention mechanism in DeepSeek-V4 series introduces multiple types of KV entries with different Key-Value (KV) cache sizes and update rules. The lightning indexer for sparse selection introduces additional dimensions

DeepSeek-V4 中的异构 KV 条目。DeepSeek-V4 系列中的混合注意力机制引入了多种类型的 KV 条目，其 Key-Value（KV）缓存大小和更新规则各不相同。闪电索引器用于稀疏选择，引入了额外的维度

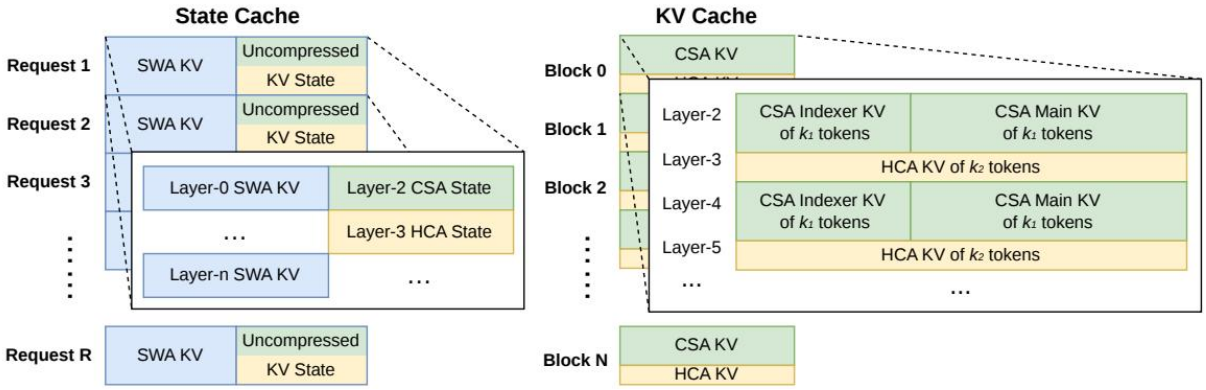

Figure 6 | Illustration of the KV cache Layout for DeepSeek-V4. The KV cache is organized into two primary components: a classical KV cache for CSA/HCA, and a state cache for SWA and unready-for-compression tokens in CSA/HCA. In the state cache, each request is assigned a fixed-size cache block. Within this block, the SWA segment stores the KV entries corresponding to the most recent n_{win} tokens, while the CSA/HCA segment stores uncompressed tail states that are not yet ready for compression. In the classical KV cache, we allocate multiple blocks per request. Each cache block covers $\text{lcm}(m, m')$ original tokens, producing $k_1 = \frac{\text{lcm}(m,m')}{m}$ CSA compressed tokens and $k_2 = \frac{\text{lcm}(m,m')}{m}'$ HCA compressed tokens.

图 6 | DeepSeek-V4 的 KV 缓存布局示意图。KV 缓存分为两个主要组件：用于 CSA/HCA 的经典 KV 缓存，以及用于 SWA 和尚未准备好压缩的 CSA/HCA 令牌的状态缓存。在状态缓存中，每个请求被分配一个固定大小的缓存块。在此块中，SWA 段存储与最近的 n_{win} 个令牌对应的 KV 条目，而 CSA/HCA 段存储尚未准备好压缩的未压缩尾部状态。在经典 KV 缓存中，我们为每个请求分配多个缓存块。每个缓存块覆盖 $\text{lcm}(m, m')$ 个原始令牌，生成 $k_1 = \frac{\text{lcm}(m,m')}{m}$ 个 CSA 压缩令牌和 $k_2 = \frac{\text{lcm}(m,m')}{m}'$ 个 HCA 压缩令牌。

into the KV cache that possess embedding sizes distinct from those in the primary attention. The compression techniques employed in CSA and HCA reduce the sequence length by factors of $\frac{1}{m}$ and $\frac{1}{m'}$, respectively, thereby decreasing the overall KV cache size. As a result, KV cache sizes vary across different layers. Furthermore, Sliding Window Attention (SWA) layers also operate with distinct KV cache sizes, as well as separate cache hit and eviction policies. In the compression branch, one KV entry is generated for every m tokens. When the number of remaining tokens is insufficient for compression, all pending tokens and their associated hidden states must be retained in a buffer until the compression operation can be executed. These buffered tokens represent a sequence state determined by positional context and are also managed within the KV cache framework.

进入具有与主注意力层不同嵌入尺寸的 KV 缓存中。CSA 和 HCA 中采用的压缩技术分别将序列长度压缩到 $\frac{1}{m}$ 和 $\frac{1}{m'}$ 倍，从而减少了整体的 KV 缓存大小。因此，不同层的 KV 缓存大小会有所不同。此外，滑动窗口注意力（SWA）层也使用不同的 KV 缓存大小，以及独立的缓存命中和驱逐策略。在压缩分支中，每 m 个标记生成一个 KV 条目。当剩余标记数量不足以进行压缩时，所有待处理的标记及其相关的隐藏状态都必须保留在缓冲区中，直到可以执行压缩操作。这些缓冲的标记代表由位置上下文决定的序列状态，并且也由 KV 缓存框架进行管理。

Challenges in Managing Hybrid Attention KV Cache. The hybrid attention mechanism violates fundamental assumptions behind PagedAttention and its variants. Although recent hybrid KV cache managing algorithms (e.g., Jenga (Zhang et al., 2025a), Hymba (Dong et al., 2025)) target general hybrid attention models or specific structures, two principal obstacles prevent consolidating KV caches across all layers under the PagedAttention framework:

混合注意力 KV 缓存管理的挑战。混合注意力机制违反了 PagedAttention 及其变体背后的基本假设。尽管最近的混合 KV 缓存管理算法（例如 Jenga（Zhang 等，2025a），Hymba（Dong 等，2025））旨在针对通用的混合注意力模型或特定结构，但在 PagedAttention 框架下，有两个主要障碍阻碍了在所有层之

间整合 KV 缓存：

- Diverse cache policies, such as those used in Sliding Window Attention.
- Constraints imposed by high-performance attention kernels, including alignment requirements.
- 多样的缓存策略，例如滑动窗口注意力中使用的策略。
- 高性能注意力内核所带来的限制，包括对齐要求等。

For efficient KV cache management of DeepSeek-V4, we design corresponding strategies to overcome these two challenges.

为了高效管理 DeepSeek-V4 的键值缓存，我们设计了相应的策略来克服这两个挑战。

State Cache for SWA and Uncompressed Tail Tokens. To address the first obstacle, we adopt an alternative cache management mechanism. Since SWA is designed to enhance performance under a limited KV cache size, it is reasonable to treat it, along with the uncompressed tail tokens from the compression branch, as a state-space model. The corresponding KV cache can thus be

SWA 的状态缓存和未压缩的尾部标记。为了解决第一个障碍，我们采用了一种替代的缓存管理机制。由于 SWA 是为了在有限的 KV 缓存大小下提升性能而设计的，因此将其与压缩分支中的未压缩尾部标记一起视为状态空间模型是合理的。因此，相应的 KV 缓存可以

regarded as a sequence-specific state that depends solely on the current position. Accordingly, we pre-allocate a fixed- and limited-size pool of state caches, and dynamically assign it to each sequence.

被视为一种与当前位置相关的特定位点状态。因此，我们预先分配一个固定且有限大小的状态缓存池，并动态地将其分配给每个序列。

Sparse Attention Kernel Co-Design. Regarding the second obstacle, conventional high-performance attention kernels typically assume a fixed number B of tokens per block to optimize performance, corresponding to $B \cdot m$ original tokens in CSA and $B \cdot m'$ in HCA. Through employing a high-performance sparse-attention kernel, different layers can accommodate variable tokens per block without performance degradation. Achieving this requires co-designing the KV cache layout and the sparse attention kernel. For instance, padding blocks to align with cache lines can improve performance. Thus, for CSA with compression ratio m and HCA with ratio m' , the number of original tokens per block can be any multiple of lcm(m, m') , the least common multiple of these two compression ratios.

稀疏注意力核协同设计。针对第二个障碍，传统的高性能注意力核通常假设每个块有固定数量 B 的 token 以优化性能，对应于 CSA 中的 $B \cdot m$ 个原始 token 和 HCA 中的 $B \cdot m'$ 个原始 token。通过采用高性能的稀疏注意力核，不同的层可以在不导致性能下降的情况下，每个块容纳不同数量的 token。实现这一点需要协同设计 KV 缓存布局和稀疏注意力核。例如，通过填充块以与缓存行对齐可以提高性能。因此，对于压缩比为 m 的 CSA 和压缩比为 m' 的 HCA，每个块中的原始 token 数量可以是 lcm(m, m') 的任意倍数，即这两个压缩比的最小公倍数。

3.5.2. On-Disk KV Cache Storage

3.5.2. 磁盘上的键值缓存存储

When serving DeepSeek-V4, we leverage an on-disk KV cache storage mechanism to eliminate repeated prefilling for shared-prefix requests. For the compressed KV entries in CSA/HCA and

the uncompressed KV entries in Sliding Window Attention (SWA), we design separate solutions for storage management.

在服务 DeepSeek-V4 时，我们利用一种基于磁盘的 KV 缓存存储机制，以消除共享前缀请求的重复预填充。对于 CSA/HCA 中的压缩 KV 条目和 Sliding Window Attention（SWA）中的未压缩 KV 条目，我们为存储管理设计了不同的解决方案。

For CSA and HCA, we simply store all of the compressed KV entries to the disk. When a request hits a stored prefix, we read and reuse the compressed KV entries corresponding to the prefix, until the last complete compression block. Specially, for prefix tokens in the tail incomplete block, we still need to recompute them to restore the uncompressed KV entries, as uncompressed KV entries in CSA and HCA are not stored.

对于 CSA 和 HCA，我们只需将所有的压缩 KV 条目存储到磁盘上。当请求命中一个存储的前缀时，我们读取并重用与该前缀对应的压缩 KV 条目，直到最后一个完整的压缩块。特别地，对于尾部不完整块中的前缀标记，我们仍需要重新计算它们以恢复未压缩的 KV 条目，因为 CSA 和 HCA 中未存储未压缩的 KV 条目。

For the SWA KV entries, since they are not compressed and exist in every layer, their volume is approximately 8 times larger than the compressed CSA and HCA KV entries. To handle these large SWA KV entries efficiently, we propose and implement three distinct strategies for managing on-disk SWA KV entries, each offering a different trade-off between storage overhead and computational redundancy:

对于 SWA KV 条目，由于它们未被压缩且存在于每一层，其体积大约是压缩后的 CSA 和 HCA KV 条目的 8 倍。为了高效地处理这些大型 SWA KV 条目，我们提出了并实现了三种不同的策略来管理磁盘上的 SWA KV 条目，每种策略在存储开销和计算冗余之间提供了不同的权衡：

- Full SWA Caching. This strategy stores the complete SWA KV entries for all tokens, ensuring computational zero-redundancy. Under this strategy, the SWA KV entries of the hitting prefix can be reconstructed by just reading the on-disk cache of the last n_{win} tokens within that prefix. Despite computational zero-redundancy, this strategy is inefficient for modern SSD-based storage systems — only a small subset of the stored SWA KV cache will be accessed for each hitting request, which leads to an unbalanced write-intensive access pattern.

- Periodic Checkpointing. This strategy checkpoints SWA KV entries of the last n_{win} tokens within every p tokens, where p is a tunable parameter. For a hitting prefix, we load the most recent checkpointed state, and then recompute the remaining tail tokens. Through tuning p , this strategy enables an on-demand trade-off between storage and computation.

- Zero SWA Caching. This strategy does not store any SWA KV entries. For a hitting prefix, we need to perform more recomputation to restore the SWA KV entries. To be specific, in each attention layer, the SWA KV entry of each token depends on the SWA KV entries of only the most recent n_{win} tokens from the previous layer. Therefore, leveraging cached CSA and HCA KV entries, recomputing the last $n_{\text{win}} \cdot L$ tokens is enough to restore the last n_{win} SWA KV entries for an L -layer model.

- 完整的 SWA 缓存。该策略存储所有 token 的完整 SWA KV 条目，确保计算零冗余。在此策略下，只需读取该前缀中最后 n_{win} 个 token 的磁盘缓存，就可以重建命中前缀的 SWA KV 条目。尽管计算零冗余，但该策略对于现代基于 SSD 的存储系统来说效率较低——每个命中请求只会访问存储的 SWA KV 缓存中的一小部分，导致写密集型访问模式不平衡。

- 定期检查点。该策略在每 p 个 token 中检查点存储最后 n_{win} 个 token 的 SWA KV 条目，其中 p 是一个可调参数。对于一个命中前缀，我们加载最新的检查点状态，然后重新计算剩余的尾部 token。通过调整 p ，该策略能够在存储和计算之间实现按需的权衡。

- 零 SWA 缓存。该策略不存储任何 SWA KV 条目。对于一个命中前缀，我们需要进行更多的重新计算来恢复 SWA KV 条目。具体来说，在每一层注意力中，每个 token 的 SWA KV 条目仅依赖于前一层中最近的 n_{win} 个 token 的 SWA KV 条目。因此，利用缓存的 CSA 和 HCA KV 条目，重新计算最后 $n_{\text{win}} \cdot L$ 个 token 就足以恢复一个 L 层模型的最后 n_{win} 个 SWA KV 条目。

Depending on specific deployment scenarios, we select the most suitable strategy to achieve the desired trade-off between storage and computation.

根据具体的部署场景，我们选择最合适的策略，以实现存储与计算之间的最佳平衡。

4. Pre-Training

4. 预训练

4.1. Data Construction

4.1. 数据构建

On top of the pre-training data of DeepSeek-V3, we endeavor to construct a more diverse and higher-quality training corpus with longer effective contexts. We continually refine our data construction pipelines. For web-sourced data, we implement filtering strategies to remove batched auto-generated and templated content, thereby mitigating the risk of model collapse (Zhu et al., 2024). Mathematical and programming corpora still remain core components of our training data, and we further enhance the coding capabilities of DeepSeek-V4 series by incorporating agentic data during the mid-training phase. For multilingual data, we build a larger corpus for DeepSeek-V4, improving its capture of long-tail knowledge across different cultures. For DeepSeek-V4, we place a particular emphasis on long-document data curation, prioritizing scientific papers, technical reports, and other materials that reflect unique academic values. Combining all the above, our pre-training corpus comprises more than 32T tokens, containing mathematical contents, codes, web pages, long documents, and other high-quality categories.

在 DeepSeek-V3 的预训练数据基础上，我们努力构建一个更加多样化和高质量的训练语料库，具有更长的有效上下文。我们持续优化我们的数据构建流程。对于网络来源的数据，我们实施过滤策略，以去除批量自动生成和模板化内容，从而降低模型崩溃的风险（Zhu 等，2024）。数学和编程语料库仍然是我们训练数据的核心组成部分，我们通过在中期训练阶段引入代理数据，进一步增强 DeepSeek-V4 系列的编码能力。对于多语言数据，我们为 DeepSeek-V4 构建了一个更大的语料库，提高了其在不同文化中捕捉长尾知识的能力。对于 DeepSeek-V4，我们特别重视长文档数据的整理，优先选择反映独特学术价值的科学论文、技术报告和其他材料。综合以上所有内容，我们的预训练语料库包含超过 32T 个 token，涵盖数学内容、代码、网页、长文档等高质量类别。

For pre-training data, we largely follow the same pre-processing strategies of DeepSeek-V3. For tokenization, on top of the DeepSeek-V3 tokenizer, we introduce a few special tokens for context construction, and still remain the vocabulary size to be 128K. We also inherit the token-splitting (DeepSeek-AI, 2024) and Fill-in-Middle (FIM) (DeepSeek-AI, 2024) strategies from DeepSeek-V3. Inspired by Ding et al. (2024), we pack documents from different sources into appropriate sequences to minimize sample truncation. Different from DeepSeek-V3, we employ sample-level attention masking during pre-training.

对于预训练数据，我们基本上遵循 DeepSeek-V3 相同的预处理策略。在分词方面，在 DeepSeek-V3 分词器的基础上，我们引入了一些用于上下文构建的特殊标记，并仍将词汇表大小保持为 128K。我们还继承了 DeepSeek-V3 的 token-splitting（DeepSeek-AI, 2024）和 Fill-in-Middle（FIM）（DeepSeek-AI, 2024）策略。受 Ding 等人（2024）的启发，我们将来自不同来源的文档打包到适当的序列中，以最小化样本截断。与 DeepSeek-V3 不同的是，我们在预训练过程中采用了样本级的注意力掩码。

4.2. Pre-Training Setups

4.2. 预训练设置

4.2.1. Model Setups

4.2.1. 模型设置

DeepSeek-V4-Flash. We set the number of Transformer layers to 43 and the hidden dimension d to 4096. For the first two layers, we use pure sliding window attention. For the subsequent layers, CSA and HCA are used in an interleaved manner. For CSA, we set the compression rate m to 4, the number of indexer query heads n_h^I to 64, the indexer head dimension c^I to 128, and the number of KV entries selected for sparse attention (i.e., attention top-k) to 512. For HCA, we set the compression rate m' to 128. For both CSA and HCA, we set the number of query heads n_h to 64, the head dimension c to 512, and the query compression dimension d_c to 1024. The number of output projection groups g is set to 8, and the dimension of each intermediate attention output d_g is set to 1024. For the additional branch of sliding window attention, the window size n_{win} is set to 128. We employ MoE layers in all Transformer blocks, but use the Hash routing strategy for the first 3 MoE layers. Each MoE layer consists of 1 shared expert and 256 routed experts, where the intermediate hidden dimension of each expert is 2048. Among the routed experts, 6 experts will be activated for each token. The multi-token prediction depth is set to 1. As for mHC, the expansion factor n_{hc} is set to 4, and the number of Sinkhorn-Knopp iterations t_{max} is set to 20. Under this configuration, DeepSeek-V4-Flash comprises 284B total parameters, of which 13B are activated for each token.

DeepSeek-V4-Flash。我们将 Transformer 层的数量设置为 43，隐藏维度 d 设置为 4096。对于前两层，我们使用纯滑动窗口注意力。对于后续的层，CSA 和 HCA 以交错的方式使用。对于 CSA，我们将压缩率 m 设置为 4，索引器查询头的数量 n_h^I 设置为 64，索引器头维度 c^I 设置为 128，并将用于稀疏注意力（即注意力 top-k）选择的 KV 条目数量设置为 512。对于 HCA，我们将压缩率 m' 设置为 128。对于 CSA 和 HCA，我们将查询头的数量 n_h 设置为 64，头维度 c 设置为 512，并将查询压缩维度 d_c 设置为 1024。输出投影组的数量 g 设置为 8，每个中间注意力输出 d_g 的维度设置为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_{win} 设置为 128。我们在所有 Transformer 块中使用 MoE 层，但前 3 个 MoE 层使用 Hash 路由策略。每个 MoE 层由 1 个共享专家和 256 个路由专家组成，其中每个专家的中间隐藏维度为 2048。在路由专家中，每个 token 将激活 6 个专家。多 token 预测深度设置为 1。至于 mHC，扩展因子 n_{hc} 设置为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设置为 20。在这一配置下，DeepSeek-V4-Flash 总参数数量为 284B，每个 token 激活的参数数量为 13B。

DeepSeek-V4-Pro. We set the number of Transformer layers to 61 and the hidden dimension d to 7168. For the first two layers, we use HCA. For the subsequent layers, CSA and HCA are used in an interleaved manner. For CSA, we set the compression rate m to 4, the number of indexer query heads n_h^I to 64, the indexer head dimension c^I to 128, and the number of KV entries selected for sparse attention (i.e., attention top-k) to 1024. For HCA, we set the compression rate m' to 128. For both CSA and HCA, we set the number of query heads n_h to 128, the head dimension c to 512, and the query compression dimension d_c to 1536. The number of output projection groups g is set to 16, and the dimension of each intermediate attention output d_g is set to 1024. For the additional branch of sliding window attention, the window size n_{win} is set to 128. We employ MoE layers in all Transformer blocks, but use the Hash routing strategy for the first 3 MoE layers. Each MoE layer consists of 1 shared expert and 384 routed experts, where the intermediate hidden dimension of each expert is 3072. Among the routed experts, 6 experts will be activated for each token. The multi-token prediction depth is set to 1. As for mHC, the expansion factor n_{hc} is set to 4, and the number of Sinkhorn-Knopp iterations t_{max} is set to 20. Under this configuration, DeepSeek-V4-Pro comprises 1.6T total parameters, of which 49B are activated for each token.

DeepSeek-V4-Pro。我们将 Transformer 层的数量设置为 61，隐藏维度 d 设置为 7168。对于前两层，我们使用 HCA。对于后续的层，CSA 和 HCA 以交错的方式使用。对于 CSA，我们将压缩率 m 设置为 4，索引器查询头数量 n_h^I 设置为 64，索引器头维度 c^I 设置为 128，并将用于稀疏注意力（即注意力 top-k）

的 KV 条目数量设置为 1024。对于 HCA，我们将压缩率 m' 设置为 128。对于 CSA 和 HCA，我们将查询头数量 n_h 设置为 128，头维度 c 设置为 512，并将查询压缩维度 d_c 设置为 1536。输出投影组数 g 设置为 16，每个中间注意力输出的维度 d_g 设置为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_{win} 设置为 128。我们在所有的 Transformer 块中使用 MoE 层，但前 3 个 MoE 层使用 Hash 路由策略。每个 MoE 层包含 1 个共享专家和 384 个路由专家，其中每个专家的中间隐藏维度为 3072。在路由专家中，每个 token 会激活 6 个专家。多 token 预测深度设置为 1。至于 mHC，扩展因子 n_{hc} 设置为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设置为 20。在此配置下，DeepSeek-V4-Pro 总共包含 1.6T 个参数，每个 token 激活 49B 个参数。

4.2.2. Training Setups

4.2.2. 训练设置

DeepSeek-V4-Flash. We employ the Muon optimizer (Jordan et al., 2024; Liu et al., 2025) for the majority of parameters, but use the AdamW optimizer (Loshchilov and Hutter, 2017) for the embedding module, the prediction head module, and the weights of all RMSNorm modules. For AdamW, we set its hyper-parameters to $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\varepsilon = 10^{-20}$, and weight_decay = 0.1. For Muon, we set the momentum to 0.95 and the weight decay to 0.1, and rescale the RMS of each update matrix to 0.18 for reutilization of the AdamW learning rate. We train DeepSeek-V4-Flash on 32T tokens, and as in DeepSeek-V3, we also employ a batch size scheduling strategy that increases the batch size (in tokens) from a small size to 75.5M and then keeps it at 75.5M during most of the training. The learning rate is linearly warmed up in the first 2000 steps, maintained at 2.7×10^{-4} for most of the training. Near the end of the training, we finally decay the learning rate to 2.7×10^{-5} following a cosine schedule. The training starts with a sequence length of 4K, and we gradually extend the training sequence length to 16K, 64K, and 1M. As for the setups of sparse attention, we first warmup the model with dense attention for the first 1T tokens, and introduce sparse attention at the sequence length of 64K and keep sparse attention during the rest of the training. When introducing attention sparsity, we first set a short stage to warm up the lightning indexer in CSA, and then train the model with sparse attention for most of the training. For auxiliary-loss-free load balancing, we set the bias update speed to 0.001. For the balance loss, we set its loss weight to 0.0001 to avoid extreme imbalance within single sequences. The MTP loss weight is set to 0.3 for most of the training, and to 0.1 upon the start of learning rate decay.

DeepSeek-V4-Flash。我们对大部分参数使用 Muon 优化器（Jordan 等，2024；Liu 等，2025），但对于嵌入模块、预测头模块以及所有 RMSNorm 模块的权重，我们使用 AdamW 优化器（Loshchilov 和 Hutter，2017）。对于 AdamW，我们将其超参数设置为 $\beta_1 = 0.9$、$\beta_2 = 0.95$、$\varepsilon = 10^{-20}$，并设置 weight_decay = 0.1。对于 Muon，我们将动量设置为 0.95，权重衰减设置为 0.1，并将每个更新矩阵的 RMS 重新缩放至 0.18，以便重用 AdamW 的学习率。我们在 32T 个 token 上训练 DeepSeek-V4-Flash，与 DeepSeek-V3 类似，我们也采用了一种批量大小调度策略，将批量大小（以 token 为单位）从一个小的值逐步增加到 75.5M，然后在大部分训练过程中保持在 75.5M。在前 2000 步中，学习率线性增加，大部分训练过程中保持在 2.7×10^{-4}。在训练接近结束时，我们按照余弦调度方案最终将学习率衰减至 2.7×10^{-5}。训练开始时的序列长度为 4K，并逐步将训练序列长度扩展到 16K、64K 和 1M。对于稀疏注意力的设置，我们首先使用密集注意力对前 1T 个 token 进行预热，然后在序列长度为 64K 时引入稀疏注意力，并在其余训练过程中保持稀疏注意力。在引入注意力稀疏性时，我们首先设置一个较短的阶段以在 CSA 中预热闪电索引器，然后在大部分训练过程中使用稀疏注意力进行训练。对于无辅助损失的负载均衡，我们将偏置更新速度设置为 0.001。对于平衡损失，我们将其损失权重设置为 0.0001，以避免单个序列内的极端不平衡。在大部分训练过程中，MTP 损失权重设置为 0.3，在学习率衰减开始时设置为 0.1。

DeepSeek-V4-Pro. Except for specific values of hyper-parameters, the training setup of DeepSeek-V4-Pro is largely consistent with that of DeepSeek-V4-Flash. We employ the Muon optimizer for the majority of parameters, but use the AdamW optimizer for the embedding module, the prediction head module, and the weights of all RMSNorm modules. The hyper-parameters of AdamW and Muon are the same as those of DeepSeek-V4-Flash. We train DeepSeek-V4-Pro on 33T tokens, and also employ a batch size scheduling strategy, with the maximum batch size being 94.4M tokens. The learning rate scheduling strategy is largely the same as that of DeepSeek-V4-Flash, but the peak learning rate is set to 2.0×10^{-4} and the end learning rate is set to 2.0×10^{-5} . The

training also starts with a sequence length of 4K, and the length is gradually

DeepSeek-V4-Pro。除了某些超参数的特定值外，DeepSeek-V4-Pro 的训练设置与 DeepSeek-V4-Flash 基本一致。我们对大部分参数使用 Muon 优化器，但对于嵌入模块、预测头模块以及所有 RMSNorm 模块的权重，我们使用 AdamW 优化器。AdamW 和 Muon 的超参数与 DeepSeek-V4-Flash 相同。我们使用 33T tokens 对 DeepSeek-V4-Pro 进行训练，并采用了批量大小调度策略，最大批量大小为 94.4M tokens。学习率调度策略与 DeepSeek-V4-Flash 基本相同，但峰值学习率设置为 2.0×10^{-4}，最终学习率设置为 2.0×10^{-5}。训练还从 4K 的序列长度开始，长度逐渐

extended to 16K, 64K, and 1M. Compared with DeepSeek-V4-Flash, DeepSeek-V4-Pro starts with a longer stage of dense attention, and the strategy of introducing sparse attention is the same as DeepSeek-V4-Flash, following a two-stage training method. For auxiliary-loss-free load balancing, we set the bias update speed to 0.001. For the balance loss, we set its loss weight to 0.0001 to avoid extreme imbalance within single sequences. The MTP loss weight is set to 0.3 for most of the training, and to 0.1 upon the start of learning rate decay.

扩展到 16K、64K 和 1M。与 DeepSeek-V4-Flash 相比，DeepSeek-V4-Pro 从更长的密集注意力阶段开始，引入稀疏注意力的策略与 DeepSeek-V4-Flash 相同，采用两阶段训练方法。对于无辅助损失的负载均衡，我们将偏置更新速度设置为 0.001。对于平衡损失，我们将其损失权重设置为 0.0001，以避免单个序列内部出现极端不平衡。大多数训练过程中，MTP 损失权重设置为 0.3，而在学习率衰减开始时设置为 0.1。

4.2.3. Mitigating Training Instability

4.2.3. 缓解训练不稳定

Training trillion-parameter MoE models presents significant stability challenges, and DeepSeek-V4 series are no exception. We encountered notable instability challenges during training. While simple rollbacks could temporarily restore the training state, they proved inadequate as a long-term solution because they do not prevent the recurrence of loss spikes. Empirically, we identified that the occurrence of spikes is consistently tied to outliers in the MoE layers, and the routing mechanism itself appears to exacerbate the emergence of these outliers. Therefore, we sought to tackle this issue from two dimensions: breaking the vicious cycle induced by routing, and directly suppressing anomalous values. Fortunately, we discovered two practical techniques that effectively maintain training stability. Although a comprehensive theoretical understanding of their underlying mechanisms remains an open question for now, we are sharing them openly to foster further exploration by the community.

训练万亿参数的 MoE 模型带来了显著的稳定性挑战，DeepSeek-V4 系列也不例外。我们在训练过程中遇到了明显的不稳定性问题。虽然简单的回滚操作可以暂时恢复训练状态，但它们作为长期解决方案是不够的，因为它们无法防止损失激增的再次发生。通过实证分析，我们发现损失激增的发生始终与 MoE 层中的异常值有关，而路由机制本身似乎加剧了这些异常值的出现。因此，我们从两个方面着手解决这个问题：打破由路由引起的恶性循环，以及直接抑制异常值。幸运的是，我们发现了两种实用的技术，可以有效维持训练的稳定性。尽管目前对这些技术底层机制的全面理论理解仍是一个开放性问题，但我们仍然公开分享这些技术，以促进社区的进一步探索。

Anticipatory Routing. We found that decoupling the synchronous updates of the backbone network and the routing network significantly improves training stability. Consequently, at step t, we use the current network parameters θ_t for feature computation, but the routing indices are computed and applied using the historical network parameters $\theta_{t-\Delta t}$. In practice, to circumvent the overhead of loading model parameters twice, we fetch the data for step t in advance at step $t - \Delta t$. We "anticipatorily" compute and cache the routing indices to be used later at step t, which is why we name this approach Anticipatory Routing. We also heavily optimized this at the infrastructure level. First, given that pre-computing the routing indices only requires a single forward pass over the data, we carefully orchestrated the pipeline execution and the overlapping of computation with Expert Parallelism (EP) communication, successfully bounding the additional wall-clock time overhead of Anticipatory Routing to approximately 20%. Second, we

introduced an automatic detection mechanism that triggers a short rollback and activates Anticipatory Routing exclusively when a loss spike occurs; after operating in this mode for a certain period, the system reverts to standard training. Ultimately, this dynamic application allows us to avert loss spikes with negligible overall additional training overhead, all without compromising model performance.

预判路由。我们发现将主干网络和路由网络的同步更新解耦，显著提高了训练的稳定性。因此，在第 t 步，我们使用当前网络参数 θ_t 进行特征计算，但路由索引的计算和应用则使用历史网络参数 $\theta_{t-\Delta t}$ 进行。在实践中，为了避免两次加载模型参数的开销，我们在第 $t - \Delta t$ 步提前获取第 t 步所需的数据。我们 “预判性” 地计算并缓存将在第 t 步使用的路由索引，因此我们将这种方法命名为预判路由。我们还在基础设施层面对该方法进行了大量优化。首先，由于预计算路由索引只需要对数据进行一次前向传播，我们精心安排了流水线执行和计算与专家并行（EP）通信的重叠，成功将预判路由的额外实际时间开销控制在大约 20%。其次，我们引入了一种自动检测机制，当损失出现激增时，会触发短暂回滚并仅激活预判路由；在该模式下运行一段时间后，系统会恢复到标准训练模式。最终，这种动态应用使我们能够在几乎不增加总体额外训练开销的情况下避免损失激增，同时不损害模型性能。

SwiGLU Clamping. In previous literature (Bello et al., 2017; Riviere et al., 2024), clamping has been explicitly utilized to constrain numerical ranges, thereby enhancing training stability. In our actual training runs, we empirically found that applying SwiGLU clamping (OpenAI, 2025) effectively eliminates outliers and substantially aids in stabilizing the training process, without compromising performance. Throughout the training of both DeepSeek-V4-Flash and DeepSeek-V4-Pro, we clamped the linear component of SwiGLU to the range of $[-10, 10]$, while capping the upper bound of the gate component at 10.

SwiGLU 拉伸。在之前的文献（Bello 等，2017；Riviere 等，2024）中，拉伸已被明确用于限制数值范围，从而提高训练的稳定性。在我们实际的训练过程中，我们通过实证发现应用 SwiGLU 拉伸（OpenAI，2025）可以有效消除异常值，并显著有助于稳定训练过程，而不会影响性能。在整个 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的训练过程中，我们将 SwiGLU 的线性部分限制在 $[-10, 10]$ 范围内，同时将门控部分的上限限制在 10。

4.3. Evaluations

4.3. 评估

4.3.1. Evaluation Benchmarks

4.3.1. 评估基准

For the evaluation of the base models, we consider benchmarks spanning four key dimensions: world knowledge, language understanding and reasoning, coding and mathematics, and long-context processing.

对于基础模型的评估，我们考虑涵盖四个关键维度的基准测试：世界知识、语言理解和推理、编程与数学，以及长上下文处理。

World knowledge benchmarks include AGIEval (Zhong et al., 2023), C-Eval (Huang et al., 2023), CMMLU (Li et al., 2023) MMLU (Hendrycks et al., 2020), MMLU-Redux (Gema et al., 2024), MMLU-Pro (Wang et al., 2024b), MMMLU (OpenAI, 2024a), MultiLoKo (Hupkes and Bogoychev, 2025), Simple-QA verified (Haas et al., 2025), SuperGPQA (Du et al., 2025), FACTS Parametric (Cheng et al., 2025), and TriviaQA (Joshi et al., 2017).

世界知识基准包括 AGIEval (Zhong 等人，2023)，C-Eval (Huang 等人，2023)，CMMLU (Li 等人，2023)，MMLU (Hendrycks 等人，2020)，MMLU-Redux (Gema 等人，2024)，MMLU-Pro (Wang 等人，2024b)，MMMLU (OpenAI，2024a)，MultiLoKo (Hupkes 和 Bogoychev，2025)，Simple-QA verified (Haas 等人，2025)，SuperGPQA (Du 等人，2025)，FACTS Parametric (Cheng 等人，2025)，以及 TriviaQA (Joshi 等人，2017)。

Language understanding and reasoning benchmarks include BigBench Hard (BBH) (Suzgun et al., 2022), DROP (Dua et al., 2019), HellaSwag (Zellers et al., 2019), CLUEWSC (Xu et al., 2020), and WinoGrande (Sakaguchi et al., 2019).

语言理解和推理基准测试包括 BigBench Hard (BBH) (Suzgun 等人，2022)，DROP (Dua 等人，2019)，HellaSwag (Zellers 等人，2019)，CLUEWSC (Xu 等人，2020)，以及 WinoGrande (Sakaguchi 等人，2019)。

Coding and mathematical benchmarks include BigCodeBench (Zhuo et al., 2025), HumanEval (Chen et al., 2021), GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), MGSM (Shi et al., 2023), and CMath (Wei et al., 2023).

编码和数学基准测试包括 BigCodeBench (Zhuo 等人，2025)，HumanEval (Chen 等人，2021)，GSM8K (Cobbe 等人，2021)，MATH (Hendrycks 等人，2021)，MGSM (Shi 等人，2023)，以及 CMath (Wei 等人，2023)。

Long context benchmarks include LongBench-V2 (Bai et al., 2025b).

长上下文基准测试包括 LongBench-V2 (Bai et al., 2025b)。

4.3.2. Evaluation Results

4.3.2. 评估结果

In Table 1, we provide a detailed comparison of the base models for DeepSeek-V3.2, DeepSeek-V4-Flash, and DeepSeek-V4-Pro, all evaluated under a unified internal framework with strictly consistent settings.

在表 1 中，我们提供了对 DeepSeek-V3.2、DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 基础模型的详细比较，所有模型均在统一的内部框架下进行评估，并且设置严格一致。

Comparing DeepSeek-V4-Flash-Base with DeepSeek-V3.2-Base reveals a compelling efficiency story. Despite utilizing a substantially smaller number of both activated and total parameters, DeepSeek-V4-Flash-Base outperforms DeepSeek-V3.2-Base across a wide array of benchmarks. This advantage is especially evident in world knowledge tasks and challenging long-context scenarios. These results underscore that architectural improvements, refined data quality, and training optimizations in DeepSeek-V4-Flash-Base yield superior performance even with a more compact parameter budget, effectively surpassing the larger DeepSeek-V3.2-Base on the majority of evaluations.

比较 DeepSeek-V4-Flash-Base 与 DeepSeek-V3.2-Base 可以发现一个引人注目的效率故事。尽管 DeepSeek-V4-Flash-Base 使用的激活参数和总参数数量明显更少，但在一系列基准测试中，其表现优于 DeepSeek-V3.2-Base。这种优势在世界知识任务和具有挑战性的长上下文场景中尤为明显。这些结果表明，DeepSeek-V4-Flash-Base 在架构改进、数据质量优化和训练优化方面的提升，即使在更紧凑的参数预算下也能实现卓越的性能，从而在大多数评估中有效超越了更大的 DeepSeek-V3.2-Base。

Furthermore, DeepSeek-V4-Pro-Base demonstrates a further, decisive leap in capability, establishing near-universal dominance over both DeepSeek-V3.2-Base and DeepSeek-V4-Flash-Base. With improvements across almost all categories, DeepSeek-V4-Pro-Base reaches new performance highs among DeepSeek base models on the most demanding benchmarks. On knowledge-intensive evaluations, it delivers dramatic gains, while also substantially advancing long-context understanding. On most reasoning and code benchmarks, DeepSeek-V4-Pro-Base also exceeds both previous models. This comprehensive uplift confirms DeepSeek-V4-Pro-Base as the strongest foundation model in the DeepSeek series, outperforming its predecessors across the spectrum of knowledge, reasoning, coding, and long-context capabilities.

此外，DeepSeek-V4-Pro-Base 在能力上实现了进一步的、决定性的飞跃，几乎在所有方面都超越了

DeepSeek-V3.2-Base 和 DeepSeek-V4-Flash-Base，确立了近似全面的主导地位。在几乎所有类别中都有改进，DeepSeek-V4-Pro-Base 在最具挑战性的基准测试中达到了 DeepSeek 基础模型的新性能高峰。在知识密集型评估中，它实现了显著的提升，同时在长上下文理解方面也取得了重大进展。在大多数推理和代码基准测试中，DeepSeek-V4-Pro-Base 也超过了之前的模型。这种全面的提升确认了 DeepSeek-V4-Pro-Base 是 DeepSeek 系列中最强的基础模型，在知识、推理、编程和长上下文能力等方面均优于其前身。

Table 1 | Comparison among DeepSeek-V3.2-Base, DeepSeek-V4-Flash-Base, and DeepSeek-V4-Pro-Base. All models are evaluated in our internal framework and share the same evaluation setting. Scores with a gap not exceeding 0.3 are considered to be at the same level. The highest score in each row is in bold font, and the second is underlined.

表 1 | DeepSeek-V3.2-Base、DeepSeek-V4-Flash-Base 和 DeepSeek-V4-Pro-Base 的对比。所有模型均在我们的内部框架中进行评估，并且共享相同的评估设置。得分差距不超过 0.3 的视为同一水平。每行中得分最高的以粗体显示，次高的则用下划线标出。

	Benchmark (Metric)	# Shots	DeepSeek-V3.2 Base	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base
	Architecture	-	MoE	MoE	MoE
# Activated Params	-	37B	13B	49B	
# Total Params	-	671B	284B	1.6T	
World Knowl.	AGIEval (EM)	0-shot	80.1	82.6	83.1
MMLU (EM)	5-shot	87.8	88.7	90.1	
MMLU-Redux (EM)	5-shot	87.5	89.4	90.8	
MMLU-Pro (EM)	5-shot	65.5	68.3	73.5	
MMMLU (EM)	5-shot	87.9	88.8	90.3	
C-Eval (EM)	5-shot	90.4	92.1	93.1	
CMMLU (EM)	5-shot	88.9	90.4	90.8	
MultiLoKo (EM)	5-shot	38.7	42.2	51.1	
Simple-QA verified (EM)	25-shot	28.3	30.1	55.2	
SuperGPQA (EM)	5-shot	45.0	46.5	53.9	
FACTS Parametric (EM)	25-shot	27.1	33.9	62.6	
TriviaQA (EM)	5-shot	83.3	82.8	85.6	
Lang. & Reas.	BBH (EM)	3-shot	87.6	86.9	87.5
DROP (F1)	1-shot	88.2	88.6	88.7	
HellaSwag (EM)	0-shot	86.4	85.7	88.0	
WinoGrande (EM)	0-shot	78.9	79.5	81.5	

CLUEWSC (EM)	5-shot	83.5	82.2	85.2	
Code & Math	BigCodeBench (Pass@1)	3-shot	63.9	56.8	59.2
HumanEval (Pass@1)	0-shot	62.8	69.5	76.8	
GSM8K (EM)	8-shot	91.1	90.8	92.6	
MATH (EM)	4-shot	60.5	57.4	64.5	
MGSM (EM)	8-shot	81.3	85.7	84.4	
CMath (EM)	3-shot	92.6	93.6	90.9	
Long Context	LongBench-V2 (EM)	1-shot	40.2	44.7	51.5

	基准（指标）	# 照片	DeepSeek-V3.2 Base	DeepSeek-V4-Flash B
	架构	-	MoE	MoE
# 激活参数	-	37B	13B	49B
# 总参数量	-	671B	284B	1.6T
世界知识	AGIEval (EM)	零样本	80.1	82.6
MMLU (EM)	5-shot	87.8	88.7	90.1
MMLU-Redux (EM)	5-shot	87.5	89.4	90.8
MMLU-Pro (EM)	5-shot	65.5	68.3	73.5
MMMLU (EM)	五例示例	87.9	88.8	90.3
C-Eval (EM)	五例示例	90.4	92.1	93.1
CMMLU (EM)	5-shot	88.9	90.4	90.8
MultiLoKo (EM)	5-shot	38.7	42.2	51.1
简单问答验证（EM）	25-shot	28.3	30.1	55.2
SuperGPQA (EM)	5-shot	45.0	46.5	53.9
FACTS 参数化 (EM)	25-shot	27.1	33.9	62.6
TriviaQA (EM)	5-shot	83.3	82.8	85.6
语言 & 推理	BBH (EM)	三例示例	87.6	86.9
DROP (F1)	1-shot	88.2	88.6	88.7
HellaSwag (EM)	零样本	86.4	85.7	88.0
WinoGrande (EM)	零样本	78.9	79.5	81.5
CLUEWSC (EM)	5-shot	83.5	82.2	85.2
代码与数学	BigCodeBench (Pass@1)	三例示例	63.9	56.8
HumanEval (Pass@1)	零样本	62.8	69.5	76.8
GSM8K (EM)	8-shot	91.1	90.8	92.6
MATH (EM)	四例示例	60.5	57.4	64.5
MGSM (EM)	8-shot	81.3	85.7	84.4
CMath (EM)	三例示例	92.6	93.6	90.9
长上下文	LongBench-V2 (EM)	单样本（1-shot）	40.2	44.7

5. Post-Training

5. 模型微调后

5.1. Post-Training Pipeline

5.1. 训练后流水线

Following pre-training, we conducted a post-training phase to yield the final models of DeepSeek-V4 series. Although the training pipeline largely mirrored that of DeepSeek-V3.2, a critical methodological substitution was made: the mixed Reinforcement Learning (RL) stage was entirely replaced by On-Policy Distillation (OPD; Gu et al., 2024; Lu and Lab, 2025).

在预训练之后，我们进行了一个后训练阶段，以产生 DeepSeek-V4 系列的最终模型。尽管训练流程在很大程度上与 DeepSeek-V3.2 相似，但关键的方法上进行了替换：混合强化学习（RL）阶段完全被基于策略的蒸馏（OPD；Gu 等，2024；Lu 和 Lab，2025）所取代。

5.1.1. Specialist Training

5.1.1. 专业培训

The development of domain specialists was conducted by adapting the DeepSeek-V3.2 training pipeline. Specifically, each model was sequentially optimized through an initial fine-tuning phase and subsequent Reinforcement Learning (RL) guided by domain-specific prompts and reward signals. For the RL stage, we implemented the Group Relative Policy Optimization (GRPO) algorithm, maintaining hyper-parameters closely aligned with our prior research (DeepSeek-AI, 2025; DeepSeek-AI, 2025).

领域专家的开发是通过适应 DeepSeek-V3.2 训练流程进行的。具体而言，每个模型都依次通过初始微调阶段进行优化，并随后通过基于领域特定提示和奖励信号的强化学习（RL）进行指导。在强化学习阶段，我们实现了组相对策略优化（GRPO）算法，保持超参数与我们之前的研究所保持一致（DeepSeek-AI, 2025; DeepSeek-AI, 2025）。

Reasoning Efforts. It is widely recognized that a model's performance on reasoning tasks is fundamentally governed by the computational effort expended. Consequently, we trained distinct specialist models under divergent RL configurations to facilitate the development of models optimized for varying reasoning capacities. As detailed in Table 2, DeepSeek-V4-Pro and DeepSeek-V4-Flash both support three specific reasoning effort modes. For each mode, we apply distinct length penalties and context windows during RL training, which results in varying output token lengths for reasoning. To integrate these distinct reasoning modes, we utilize specialized response formats demarcated by the and tokens. Furthermore, for the "Think Max" mode, we prepend a specific instruction to the beginning of the system prompt to guide the model's reasoning process, as shown in Table 3.

推理努力。众所周知，模型在推理任务上的表现本质上由其计算努力程度所决定。因此，我们采用不同的强化学习（RL）配置训练了不同的专业模型，以促进开发适用于不同推理能力的模型。如表 2 所示，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 均支持三种特定的推理努力模式。对于每种模式，我们在 RL 训练过程中应用了不同的长度惩罚和上下文窗口，从而导致推理输出的 token 长度不同。为了整合这些不同的推理模式，我们利用由和标记界定的专用响应格式。此外，对于 "Think Max" 模式，我们在系统提示的开头添加了特定的指令，以引导模型的推理过程，如表 3 所示。

Table 2 | Comparison of three reasoning modes

表 2 | 三种推理模式的比较

Reasoning Mode	Characteristics	Typical Use Cases	Response Format
Non-think	Fast, intuitive responses based on habits or simple rules.	Routine daily tasks, emergency reactions, low-risk decisions.	summary
Think High	Conscious logical analysis, slower but more accurate.	Complex problem-solving, planning, medium-risk decisions.	<thinking tokens summary
Think Max	Push reasoning to its fullest extent. Slow but powerful.	Exploring the boundary of model reasoning capability.	1. A special system prompt at the beginning.2.thinking tokens summary

推理模式	特性	典型用例	响应格式
非思考	基于习惯或简单规则的快速、直观响应。	日常例行任务、应急反应、低风险决策。	摘要
高瞻远瞩	有意识的逻辑分析，速度较慢但更准确。	复杂问题解决、计划、中等风险决策。	< 思考标记摘要
Think Max	将推理推至极致。缓慢却强大。	探索模型推理能力的边界。	1. 开始时的一个特殊系

Table 3 | Instruction injected into the system prompt for the “Think Max” mode.

表 3 | 注入到系统提示中的 “Think Max” 模式指令。

Injected Instruction

注入指令

Reasoning Effort: Absolute maximum with no shortcuts permitted.

推理努力：绝对最大值，不允许有任何捷径。

You MUST be very thorough in your thinking and comprehensively decompose the problem to resolve the root cause, rigorously stress-testing your logic against all potential paths, edge cases, and adversarial scenarios.

你必须非常细致地进行思考，并全面地分解问题以解决根本原因，严格地将你的逻辑与所有潜在路径、边界情况和对抗性场景进行反复测试。

Explicitly write out your entire deliberation process, documenting every intermediate step, considered alternative, and rejected hypothesis to ensure absolutely no assumption is left unchecked.

明确写出你整个思考过程，记录每一个中间步骤、考虑过的替代方案和被拒绝的假设，以确保没有任何假设被遗漏。

Generative Reward Model. Typically, easy-to-verify tasks can be effectively optimized using simple rule-based verifiers or test cases. In contrast, hard-to-verify tasks traditionally rely on Reinforcement Learning from Human Feedback (RLHF), which necessitates extensive human annotation to train a scalar reward model. In the post-training phase of DeepSeek-V4 series,

生成式奖励模型。通常，易于验证的任务可以使用简单的基于规则的验证器或测试用例有效优化。相比之

下，难以验证的任务传统上依赖于从人类反馈中进行强化学习（RLHF），这需要大量的人工标注来训练标量奖励模型。在 DeepSeek-V4 系列的微调阶段后，

however, we dispense with these conventional scalar-based reward models. Instead, to address hard-to-verify tasks, we curate rubric-guided RL data and employ a Generative Reward Model (GRM) to evaluate policy trajectories. Crucially, we apply RL optimization directly to the GRM itself. In this paradigm, the actor network natively functions as the GRM, enabling the joint optimization of the model's evaluative (judging) proficiency alongside its standard generative capabilities. By unifying these roles, the model's internal reasoning capabilities are inherently fused into its evaluative process, resulting in highly robust scoring. Furthermore, this approach achieves superior performance with only a minimal set of diverse human annotations, as the model leverages its own logic to generalize across complex tasks.

然而，我们摒弃了这些传统的基于标量的奖励模型。相反，为了解决难以验证的任务，我们整理了由评分标准引导的强化学习数据，并采用生成式奖励模型（GRM）来评估策略轨迹。关键的是，我们将强化学习优化直接应用于 GRM 本身。在这一范式中，演员网络（actor network）原生地作为 GRM 运作，使模型的评估（判断）能力与其标准生成能力能够联合优化。通过统一这些角色，模型的内部推理能力被内化到其评估过程中，从而实现高度稳健的评分。此外，这种方法仅需极少量多样化的标注数据即可实现卓越的性能，因为模型能够利用自身的逻辑，在复杂任务中实现泛化。

Table 4 | Tool-call schema for DeepSeek-V4 series.
Tool Call Schema

表 4 | DeepSeek-V4 系列工具调用模式。
工具调用模式

```
## Tools

You have access to a set of tools to help answer the user's question. You can invoke too

<|DSML|tool_calls>
<|DSML|invoke name="$TOOL_NAME">
<|DSML|parameter name="$PARAMETER_NAME" string="true|false"$PARAMETER_VALUE
    </|DSML|parameter>
...
</|DSML|invoke>
<|DSML|invoke name="$TOOL_NAME2">
...
</|DSML|invoke>
</|DSML|tool_calls>

String parameters should be specified as is and set 'string="true"'. For all other types

If thinking_mode is enabled (triggered by <think>), you MUST output your complete reason

Otherwise, output directly after </think> with tool calls or final response.

### Available Tool Schemas

{Tool Definition...}

You MUST strictly follow the above definedtool name and parameter schemas to invoke tool
```

Tool-Call Schema and Special Token. Consistent with our previous version, we utilize a dedicated tag to delineate the reasoning path. In DeepSeek-V4 series, we introduce a new tool-call schema that employs a special "|DSML|" token and utilizes an XML-based format for tool invocations, as demonstrated in Table 4. Our experiments demonstrate that the XML format effectively mitigates escaping failures and reduces tool-call errors, providing a more robust interface for model-tool interactions.

工具调用模式和特殊标记。与我们之前的版本保持一致，我们使用一个专用的标签来界定推理路径。在 DeepSeek-V4 系列中，我们引入了一种新的工具调用模式，该模式使用一个特殊的“|DSML|”标记，并采用基于 XML 的格式来进行工具调用，如表 4 所示。我们的实验表明，XML 格式有效地缓解了转义失败的问题，并减少了工具调用错误，为模型与工具之间的交互提供了更加稳健的接口。

Figure 7: images/496e910b13ce60ca4997759327f783852f88a5e776c2fb1ed7e9ba5ae9867686.jpg

a) Thinking with tools

a) 用工具思考

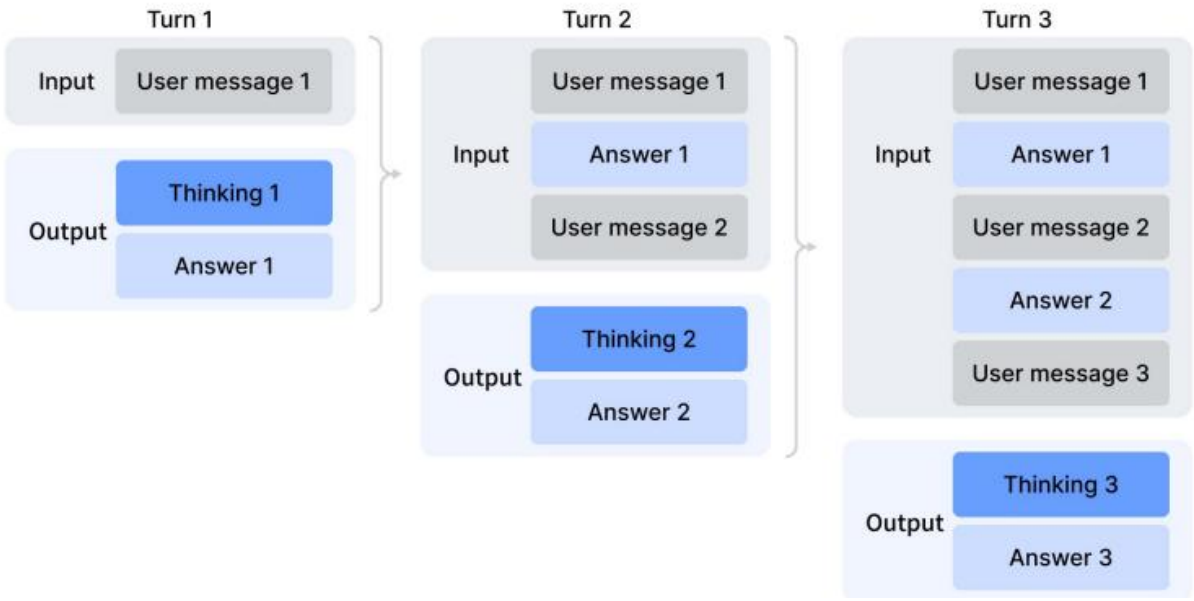

Figure 8: images/1f55a07b40421bed1e5aa0a6fbfea5bc565355545e28507cd5e51f6976e56539.jpg

b) Thinking without tools
Figure 7 | Thinking management of DeepSeek-V4 series.

b) 不使用工具的思考
图 7 | DeepSeek-V4 系列的思考管理。

Interleaved Thinking. DeepSeek-V3.2 introduced a context management strategy that retains reasoning traces across tool-result rounds but discards them upon the arrival of new user messages. While effective, this still caused unnecessary token waste in complex agentic workflows — each new user turn would flush all accumulated reasoning content, forcing the model to reconstruct its problem-solving state from scratch. Leveraging the expanded 1M-token context window of DeepSeek-V4 series, we further refine this mechanism to maximize the effectiveness of interleaved thinking in agentic environments:

交错思考。DeepSeek-V3.2 引入了一种上下文管理策略，该策略在工具结果轮次之间保留推理痕迹，但在接收到新用户消息时会丢弃这些痕迹。虽然这种策略是有效的，但在复杂的代理工作流中，它仍然导致了不必要的 token 浪费——每次新的用户回合都会清除所有累积的推理内容，迫使模型必须从头开始重建其解决问题的状态。借助 DeepSeek-V4 系列扩展到 1M token 的上下文窗口，我们进一步优化了这一机制，以最大化交错思考在代理环境中的效果：

- Tool-Calling Scenarios. As illustrated in Figure 7(a), all reasoning content is fully preserved throughout the entire conversation. Unlike DeepSeek-V3.2, which discarded thinking traces upon each new user turn, DeepSeek-V4 series retain the complete reasoning history across all rounds, including across user message boundaries. This allows the model to maintain a coherent, cumulative chain of thought over long-horizon agent tasks.

- General Conversational Scenarios. As illustrated in Figure 7(b), the original strategy is preserved: reasoning content from previous turns is discarded when a new user message arrives, keeping the context concise for settings where persistent reasoning traces provide limited benefit.

- 工具调用场景。如图 7(a) 所示，整个对话过程中所有的推理内容都会被完整保留。与 DeepSeek-V3.2 不同，后者在每次用户的新回合中都会丢弃推理痕迹，而 DeepSeek-V4 系列则在所有回合中保留完

整的推理历史，包括跨越用户消息边界的情况。这使得模型能够在长周期代理任务中保持连贯、累积的推理链。

- 通用对话场景。如图 7(b) 所示，原始策略得以保留：当新用户消息到达时，之前的推理内容会被丢弃，以保持上下文的简洁性，适用于持久推理痕迹提供有限益处的场景。

As with DeepSeek-V3.2, agent frameworks that simulate tool interactions via user messages (e.g., Terminus) may not trigger the tool-calling context path and thus may not benefit from enhanced

与 DeepSeek-V3.2 类似，通过用户消息模拟工具交互的代理框架（例如 Terminus）可能不会触发工具调用上下文路径，因此可能无法受益于增强功能

reasoning persistence. We continue to recommend non-think models for such architectures.

推理持久性。我们继续推荐在这样的架构中使用非思考模型。

Table 5 | Quick Instruction special tokens for auxiliary tasks.

表 5 | 辅助任务的快速指令特殊标记。

Special Token	Description	Format
	Determines whether the user prompt requires a web search or can be answered directly.	...{prompt}
	Generates a concise conversation title after the first assistant response.	...{response}
	Generates search queries for the user prompt.	...{prompt}
	Classifies the user prompt's demand for source authoritativeness.	...{prompt}
	Identifies the domain of the user prompt.	...{prompt}
	Determines whether each URL in the user prompt should be fetched and read.	...{prompt}

特殊标记	描述	格式
	确定用户提示是否需要进行网络搜索或可以直接回答。	... {提示}
	在第一次助手回复后生成一个简洁的对话标题。	...{response}
	为用户提示生成搜索查询。	... {prompt}
	对用户提示中对来源权威性的需求进行分类。	... {提示}
	识别用户提示的领域。	...{prompt}
	确定是否应获取和读取用户提示中的每个 URL。	... {prompt}

Quick Instruction. In chatbot scenarios, a number of auxiliary tasks (e.g., determining whether to trigger a web search, intent recognition, etc.) must be executed before generating the response.

Conventionally, these tasks are handled by a separate small model, requiring redundant prefilling since it cannot reuse the existing KV cache. To overcome this limitation, we introduce Quick Instruction. We append a set of dedicated special tokens directly to the input sequence, where each token corresponds to a specific auxiliary task. By directly reusing the already-computed KV cache, this mechanism completely avoids redundant prefilling and allows certain tasks, such as generating search queries and determining authority and domain, to be executed in parallel. Consequently, this approach significantly reduces the user-perceived time-to-first-token (TTFT) and eliminates the engineering overhead of maintaining and iterating an extra small model. The supported Quick Instruction tokens are summarized in Table 5.

快速指令。在聊天机器人场景中，在生成响应之前，需要执行一系列辅助任务（例如，确定是否触发网络搜索、意图识别等）。通常，这些任务由一个独立的小模型来处理，由于无法复用现有的 KV 缓存，因此需要重复进行预填充。为了解决这一限制，我们引入了快速指令。我们直接将一组专用的特殊标记添加到输入序列中，其中每个标记对应一个特定的辅助任务。通过直接复用已经计算出的 KV 缓存，这种机制完全避免了重复的预填充，并允许某些任务（如生成搜索查询和确定权威性和领域）并行执行。因此，这种方法显著减少了用户感知到的首次令牌生成时间（TTFT），并消除了维护和迭代额外小模型的工程开销。支持的快速指令标记汇总在表 5 中。

5.1.2. On-Policy Distillation

5.1.2. 在策略蒸馏

After training multiple domain-specific experts via specialized fine-tuning and reinforcement learning, we employ multi-teacher On-Policy Distillation (OPD; Gu et al. 2024; Lu and Lab 2025) as the primary technique for merging expert capabilities into the final model. OPD has emerged as an effective post-training paradigm for efficiently transferring the knowledge and capabilities of domain experts to a single, unified model. This is achieved by having the student learn from the output distributions of teacher models on its own generated trajectories. Formally, given a set of N expert models $\left\{\pi_{E_1}, \pi_{E_2}, \ldots, \pi_{E_N}\right\}$, the OPD objective function is defined as:

在通过专门的微调和强化学习训练了多个领域特定专家之后，我们采用多教师在线策略蒸馏（OPD；Gu 等人，2024；Lu 和 Lab，2025）作为将专家能力整合到最终模型的主要技术。OPD 已经成为一种有效的后训练范式，能够高效地将领域专家的知识和能力转移到单一的统一模型中。这是通过让学生模型从其自身生成轨迹上教师模型的输出分布中学习来实现的。形式上，给定一组 N 个专家模型 $\left\{\pi_{E_1}, \pi_{E_2}, \ldots, \pi_{E_N}\right\}$，OPD 的目标函数定义为：

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{i=1}^{N} w_i \cdot \mathrm{D}_{\text{KL}}\left(\pi_\theta \| \pi_{E_i}\right). \tag{29}$$

In this formulation, w_i represents the assigned weight for each expert, typically determined by the relative importance of the expert. Computing the reverse KL loss $D_{\text{KL}}\left(\pi_\theta \parallel \pi_{E_i}\right)$ requires

在这个公式中，w_i 表示每个专家的权重，通常由专家的重要性相对程度决定。计算反向 KL 损失 $D_{\text{KL}}\left(\pi_\theta \parallel \pi_{E_i}\right)$ 需要

sampling training trajectories from the student π_θ to maintain on-policy learning. The underlying logic ensures that the unified policy π_θ selectively learns from the specialized expert relevant to the current task context (e.g., aligning with the mathematics expert for math reasoning tasks and the coding expert for programming tasks). Through this mechanism, the knowledge from physically distinct expert weights is consolidated into a unified parameter space via logits-level alignment, practically circumventing the performance degradation often encountered in traditional weight-merging or mixed RL techniques. In this stage, more than ten teacher models covering various domains are employed to distill a single student model.

从学生 π_θ 中采样训练轨迹以维持在线策略学习。其底层逻辑确保统一策略 π_θ 能够有选择地从与当前任务上下文相关的专业专家中学习（例如，在数学推理任务中对齐数学专家，在编程任务中对齐编码专家）。通过这种机制，物理上不同的专家权重知识通过 logits 层级对齐被整合到统一的参数空间中，实际上绕过了传统权重合并或混合强化学习技术中常遇到的性能下降问题。在这一阶段，使用了涵盖各种领域的十余个教师模型来蒸馏出一个学生模型。

In handling the above OPD objective, prior works usually simplify the full-vocabulary KL loss into a token-level KL estimate at each token position, and reuse RL framework by replacing $\text{sg}\left[\log \frac{\pi_{E_i}(y_t|x,y_{<t})}{\pi_\theta(y_t|x,y_{<t})}\right]$ (sg represents the stop gradient operation) as the per-token advantage estimate in the policy loss calculation. Although this approach is resource-efficient, it leads to high variance in gradient estimation and often causes training instability. Therefore, we adopt full-vocabulary logit distillation in our OPD. Preserving the complete logit distribution in calculating reverse KL loss yields more stable gradient estimates and ensures faithful distillation of the teachers' knowledge. In the following subsection, we describe the engineering efforts that make full-vocabulary OPD feasible at scale.

在处理上述 OPD 目标时，以往的工作通常将全词汇 KL 损失简化为在每个 token 位置上的 token 级 KL 估计，并通过将 $\text{sg}\left[\log \frac{\pi_{E_i}(y_t|x,y_{<t})}{\pi_\theta(y_t|x,y_{<t})}\right]$（sg 表示停止梯度操作）替换为策略损失计算中的每个 token 的优势估计，重用 RL 框架。虽然这种方法资源效率较高，但会导致梯度估计的方差较高，常常引起训练不稳定。因此，我们在 OPD 中采用了全词汇 logit 蒸馏。在计算反向 KL 损失时保留完整的 logit 分布，可以得到更加稳定的梯度估计，并确保教师知识的忠实蒸馏。在以下子节中，我们将描述使全词汇 OPD 在大规模上可行的工程努力。

5.2. Post-Training Infrastructures

5.2. 训练后基础设施

Our post-training infrastructure is built upon the scalable framework developed for DeepSeek-V3.2. Specifically, we integrate the same distributed training stack described in Section 3.4 and the rollout engine introduced earlier for efficient auto-regressive sampling. Building on this foundation, we introduce the following principal enhancements in the present work. These designs enable efficient execution of ultra-long-context RL and OPD merging tasks involving over ten distinct teacher models, thereby substantially accelerating the iteration cycle for model releases.

我们的训练后基础设施是基于为 DeepSeek-V3.2 开发的可扩展框架构建的。具体来说，我们集成了第 3.4 节中描述的相同分布式训练堆栈，以及之前引入的 rollout 引擎，以实现高效的自回归采样。在此基础上，我们在本工作中引入了以下主要改进。这些设计使得执行涉及超过十个不同教师模型的超长上下文强化学习和 OPD 合并任务变得高效，从而显著加快模型发布的迭代周期。

5.2.1. FP4 Quantization-Aware Training

5.2.1. FP4 量化感知训练

To achieve inference acceleration and reducing memory traffic at deployment, we introduce Quantization-Aware Training (QAT) (Jacob et al., 2018) during the post-training stage, enabling the model, including those of teacher and reference models, to adapt to the precision degradation introduced by quantization. We apply FP4 (MXFP4) quantization (Rouhani et al., 2023) to two components: (1) MoE expert weights, which are a major source of GPU memory occupancy (OpenAI, 2025), and (2) the Query-Key (QK) path in the indexer of CSA, where QK activations are cached, loaded, and multiplied entirely in FP4, accelerating attention score computation in long-context scenarios. In addition, we further quantize the index scores $I_{:,:}$ from FP32 to BF16 during this QAT process. This optimization achieves a $2\times$ speedup for the top-k selector, while preserving a 99.7% recall rate of KV entries.

为了实现推理加速并减少部署时的内存流量，我们在训练后阶段引入量化感知训练（QAT）（Jacob 等，2018），使模型（包括教师模型和参考模型）能够适应量化带来的精度下降。我们对两个组件应用 FP4（MXFP4）量化（Rouhani 等，2023）：（1）MoE 专家权重，这是 GPU 内存占用的主要来源（OpenAI，2025）；（2）CSA 索引器中的查询-键（QK）路径，其中 QK 激活值在 FP4 中完全缓存、加载和相乘，从而加速长上下文场景下的注意力得分计算。此外，在此 QAT 过程中，我们进一步将索引得分 $I_{:}$ 从 FP32 量化到 BF16。这种优化实现了对 top-k 选择器的 $2\times$ 加速，同时保持了 KV 条目 99.7% 的召回率。

For MoE expert weights, following the common practice of QAT, the FP32 master weights maintained by the optimizer are first quantized to FP4, then dequantized back to FP8 for computation. Notably, our FP4-to-FP8 dequantization is lossless. This is because FP8 (E4M3) has 2 additional exponent bits compared with FP4 (E2M1), offering a larger dynamic range. Consequently, as long as the ratio between the maximum and minimum scale factors of the FP4 sub-blocks (1×32 tiles) within each FP8 quantization block (128×128 tiles) does not exceed a certain threshold, the fine-grained scale information can be fully absorbed by the extended dynamic range of FP8. We empirically verify that current weights satisfy this condition. This

对于 MoE 专家权重，遵循 QAT 的常见做法，由优化器维护的 FP32 主权重首先被量化为 FP4，然后解量化为 FP8 进行计算。值得注意的是，我们的 FP4 到 FP8 的解量化过程是无损的。这是因为 FP8（E4M3）相比 FP4（E2M1）多出 2 个指数位，从而提供了更大的动态范围。因此，只要每个 FP8 量化块（128×128 tiles）内 FP4 子块（1×32 tiles）的最大和最小缩放因子之间的比例不超过某个阈值，细粒度的缩放信息就可以被 FP8 的扩展动态范围完全吸收。我们实证验证了当前权重满足这一条件。

allows the entire QAT pipeline to fully reuse the existing FP8 training framework without any modification. In the backward pass, gradients are computed with respect to the same FP8 weights in the forward pass and directly propagated back to the FP32 master weights, equivalent to applying the Straight-Through Estimator (STE) through the quantization operation. This also avoids the need to re-quantize transposed weights.

允许整个 QAT 流水线完全复用现有的 FP8 训练框架，无需任何修改。在反向传播过程中，梯度是根据前向传播中相同的 FP8 权重计算的，并直接传播回 FP32 主权重，等同于通过量化操作应用直通估计器（STE）。这也避免了重新量化转置权重的需要。

During the inference and rollout phases of RL training, which do not involve backward passes, we directly use native FP4 quantized weights instead of simulated quantization. This ensures that model behavior during sampling is fully consistent with online deployment, while also reducing kernel memory loading for actual speedup and significantly lowering memory consumption. We process the QK path in the indexer of CSA similarly.

在强化学习训练的推理和部署阶段，这些阶段不涉及反向传播，我们直接使用原生的 FP4 量化权重，而不是模拟量化。这确保了在采样期间模型的行为与在线部署完全一致，同时还能减少实际加速的内核内存加载，并显著降低内存消耗。我们同样在 CSA 的索引器中以类似的方式处理 QK 路径。

5.2.2. Efficient Teacher Scheduling for Full-Vocabulary OPD

5.2.2. 针对全词汇 OPD 的高效教师排课

Our framework supports full-vocabulary On-Policy Distillation (OPD) with an effectively unbounded number of teachers, each potentially comprising trillions of parameters. To enable this, all teacher weights are offloaded to a centralized distributed storage and are loaded on demand during the teacher forward pass with ZeRO-like parameter sharding to alleviate both I/O and DRAM pressure. Furthermore, naively materializing logits for a vocabulary size $|V| > 100k$ across all teachers is prohibitive, even when spooled to disk. We address this by caching only the last-layer teacher hidden states in a centralized buffer during the forward pass. At training time, these cached states are retrieved and passed through the corresponding prediction head module to reconstruct the full logits on the fly. This design incurs negligible recomputation overhead while completely circumventing the memory burden associated with explicit logits materialization. To mitigate the GPU memory footprint of the teacher prediction head, we order

training samples by teacher index during data dispatching. This arrangement ensures that each distinct teacher head is loaded only once per mini-batch and that at most one teacher head resides in device memory at any given time. All parameters and hidden state loading/offloading operations proceed asynchronously in the background, without blocking computation on the critical path. Finally, the exact KL divergences between teacher and student logits are computed using a specialized TileLang kernel, which accelerates the computation and curtails dynamic memory allocation.

我们的框架支持使用有效无限制数量的教师（每个教师可能包含数万亿个参数）进行全词汇量的 On-Policy Distillation（OPD）。为了实现这一点，所有教师的权重都被卸载到一个集中的分布式存储中，并在教师前向传播过程中按需加载，采用类似 ZeRO 的参数分片技术以减轻 I/O 和 DRAM 的压力。此外，直接为词汇量 $|V| > 100k$ 的所有教师生成 logits 是不可行的，即使将这些 logits 存储到磁盘上也是如此。为了解决这个问题，我们在前向传播过程中仅缓存最后一个层的教师隐藏状态到一个集中的缓冲区中。在训练过程中，这些缓存的状态会被检索并通过相应的预测头模块传递，从而实时重建完整的 logits。这种设计几乎不增加重新计算的开销，同时完全避免了显式生成 logits 所带来的内存负担。为了减轻教师预测头的 GPU 内存占用，我们在数据分发过程中按教师索引对训练样本进行排序。这种安排确保了每个不同的教师头在每个小批量中仅被加载一次，并且在任何时刻，设备内存中最多只存在一个教师头。所有参数和隐藏状态的加载/卸载操作在后台异步进行，不会阻塞关键路径上的计算。最后，使用专门的 TileLang 内核精确计算教师与学生 logits 之间的 KL 散度，这加速了计算并减少了动态内存分配。

5.2.3. Preemptible and Fault-Tolerant Rollout Service

5.2.3. 可抢占和容错的发布服务

To maximize GPU resource utilization while enabling rapid hardware provisioning for high-priority tasks, our GPU cluster employs a cluster-wide preemptive task scheduler, where any running task may be preempted at any time. Also, hardware failures are prevalent in large-scale GPU clusters. To this end, we implement a preemptible and fault-tolerant LLM generation service for RL/OPD rollout.

为了在提高 GPU 资源利用率的同时，能够快速为高优先级任务提供硬件资源，我们的 GPU 集群采用了一个集群范围的抢占式任务调度器，任何正在运行的任务都可能在任何时候被抢占。此外，大规模 GPU 集群中硬件故障非常常见。为此，我们实现了一个可抢占且具有容错能力的 LLM 生成服务，用于 RL/OPD 的部署。

Specifically, we implement a token-granular Write-Ahead Log (WAL) for each generation request. Whenever a new token is generated for a request, we immediately append it to that request's WAL. During preemption, we pause the inference engine and save the KV cache of unfinished requests. Upon resumption, we use the persisted WALs and saved KV cache to continue decoding. Even when a fatal hardware error occurs, we can re-run the prefill phase using the persisted tokens in WAL to reconstruct the KV cache.

具体来说，我们为每个生成请求实现了一个基于令牌的预写日志（Write-Ahead Log，WAL）。每当为一个请求生成新的令牌时，我们立即将其追加到该请求的 WAL 中。在抢占发生时，我们暂停推理引擎，并保存未完成请求的 KV 缓存。在恢复时，我们使用已保存的 WAL 和 KV 缓存继续解码。即使发生致命的硬件错误，我们也可以使用 WAL 中保存的令牌重新运行预填充阶段，以重建 KV 缓存。

Importantly, it is mathematically incorrect to regenerate unfinished requests from scratch, as this introduces length bias. Because shorter responses are more likely to survive interruption, regenerating from scratch makes the model more prone to producing shorter sequences

重要的是，从头开始重新生成未完成的请求在数学上是不正确的，因为这会引入长度偏差。由于较短的响应更容易在中断中幸存下来，从头开始重新生成会使模型更容易产生较短的序列。

whenever an interruption occurs. If the inference stack is batch-invariant and deterministic, this correctness issue could also be addressed by regenerating with a consistent seed for the pseudorandom number generator used in the sampler. However, this approach still incurs the

extra cost of re-running the decoding phase, making it far less efficient than our token-granular WAL method.

每当发生中断时。如果推理栈是批处理无关且确定性的，这个问题也可以通过为采样器中使用的伪随机数生成器使用一致的种子来重新生成来解决。然而，这种方法仍然需要重新运行解码阶段，因此效率远低于我们的基于令牌的 WAL 方法。

5.2.4. Scaling RL Framework for Million-Token Context

5.2.4. 用于百万标记上下文的强化学习框架扩展

We introduce targeted optimizations for efficient RL and OPD on million-token sequences. During the rollout phase, we adopt a preemptible and fault-tolerant rollout service, detailed in Section 5.2.3. For the inference and training phase, we decompose the rollout data format into lightweight metadata and heavy per-token fields. During data dispatching, the metadata for the entire rollout data can be loaded to perform global shuffling and packing layout computation. Heavy per-token fields are loaded via a shared-memory data loader to eliminate intra-node data redundancy and are released immediately upon consumption at the mini-batch granularity, substantially reducing both CPU and GPU memory pressure. The number of on-device mini-batches is dynamically determined based on workload, allowing an efficient trade-off between computational throughput and I/O overlap.

我们引入了针对在百万标记序列上实现高效强化学习（RL）和操作（OPD）的定向优化。在 rollout 阶段，我们采用了一种可抢占且具有容错能力的 rollout 服务，详见第 5.2.3 节。对于推理和训练阶段，我们将 rollout 数据格式分解为轻量级的元数据和每个标记的重型字段。在数据分发过程中，可以将整个 rollout 数据的元数据加载进来，以执行全局洗牌和打包布局计算。重型的每个标记字段通过共享内存数据加载器进行加载，以消除节点内部的数据冗余，并在以小批量粒度消费时立即释放，从而显著降低 CPU 和 GPU 内存的压力。设备上的小批量数量根据工作负载动态确定，从而在计算吞吐量和 I/O 重叠之间实现高效的权衡。

5.2.5. Sandbox Infrastructure for Agentic AI

5.2.5. 代理型人工智能的沙盒基础设施

To meet the diverse execution demands of agentic AI during post-training and evaluation, we build a production-grade sandbox platform, DeepSeek Elastic Compute (DSec). DSec comprises three Rust components — the API gateway (Apiserver), per-host agent (Edge), and the cluster monitor (Watcher) — that are interconnected by a custom RPC protocol and scale horizontally atop the 3FS distributed filesystem (DeepSeek-AI, 2025). In production, a single DSec cluster manages hundreds of thousands of concurrent sandbox instances.

为满足代理型 AI 在训练后和评估阶段多样化的执行需求，我们构建了一个生产级的沙箱平台——DeepSeek 弹性计算（DSec）。DSec 包含三个 Rust 组件——API 网关（Apiserver）、每台主机代理（Edge）和集群监控器（Watcher），它们通过自定义的 RPC 协议相互连接，并基于 3FS 分布式文件系统（DeepSeek-AI，2025）进行横向扩展。在生产环境中，一个单一的 DSec 集群可以管理数以万计的并发沙箱实例。

The design of DSec is motivated by four observations: (1) agentic workloads are highly heterogeneous, spanning lightweight function calls to full software-engineering pipelines with diverse OS and security requirements; (2) environment images are numerous and large, yet must load quickly and support iterative customization; (3) high-density deployment demands efficient CPU and memory utilization; (4) sandbox lifecycles must coordinate with GPU training schedules, including preemption and checkpoint-based resumption. Based on these observations, we elaborate on the four core designs of DSec individually in the following.

DSec 的设计基于四个观察得出：(1) 代理型工作负载具有高度的异质性，涵盖从轻量级函数调用到具有多样化操作系统和安全需求的完整软件工程流水线；(2) 环境镜像数量众多且体积庞大，但必须能够快速加载并支持迭代定制；(3) 高密度部署要求高效利用 CPU 和内存；(4) 沙箱生命周期必须与 GPU 训练计划协调，包括抢占和基于检查点的恢复。基于这些观察，我们将在下文中分别阐述 DSec 的四个核心设计。

Four Execution Substrates Behind One Unified Interface. DSec exposes a single Python SDK (libdsec) that abstracts four execution substrates. Function Call dispatches stateless invocations to a pre-warmed container pool, eliminating cold-start overhead. Container is fully Docker-compatible and leverages EROFS (Gao et al., 2019) on-demand loading for efficient image assembly. microVM, built on Firecracker (Agache et al., 2020), adds VM-level isolation for security-sensitive, high-density deployments. fullVM, built on QEMU (Bellard, 2005), supports arbitrary guest operating systems. All four share a common API surface — command execution, file transfer, and TTY access — and switching between them requires only a parameter change.

四种执行基底，一个统一接口。DSec 提供了一个单一的 Python SDK（libdsec），它抽象了四种执行基底。函数调用将无状态调用分发到预热的容器池，从而消除冷启动的开销。容器完全兼容 Docker，并利用 EROFS（Gao 等人，2019）按需加载，以实现高效的镜像构建。microVM 基于 Firecracker（Agache 等人，2020）构建，为安全敏感、高密度部署提供 VM 级别的隔离。fullVM 基于 QEMU（Bellard，2005）构建，支持任意的客户操作系统。这四种基底共享一个共同的 API 接口 —— 命令执行、文件传输和 TTY 访问 —— 在它们之间切换只需更改一个参数。

Fast Image Loading via Layered Storage. DSec reconciles fast startup with a large and growing corpus of environment images through layered, on-demand loading. For containers, base images and filesystem commits are stored as 3FS-backed readonly EROFS layers mounted directly into overlay lowerdirs. We keep file metadata readily available on the local disk at mount time;

通过分层存储实现快速图像加载。DSec 通过分层、按需加载的方式，将快速启动与大量且不断增长的环境图像集合相结合。对于容器，基础镜像和文件系统提交被存储为基于 3FS 的只读 EROFS 层，直接挂载到 overlay 的 lowerdirs 中。我们在挂载时保持文件元数据在本地磁盘上随时可用；

meanwhile, data blocks are fetched from 3FS upon request. For microVMs, DSec uses the overlaybd (Li et al., 2020) disk format: the read-only base layer resides on 3FS for cross-instance sharing, while writes go to a local copy-on-write layer. Such snapshots are chainable, facilitating efficient versioning and millisecond-scale resumption.

同时，数据块在请求时从 3FS 中获取。对于 microVMs，DSec 使用 overlaybd（Li 等，2020）磁盘格式：只读的底层位于 3FS 上，用于跨实例共享，而写入操作则针对本地的写时复制层。此类快照可以链接在一起，从而实现高效的版本控制和毫秒级的恢复。

Density Optimizations Under Massive Concurrency. To accommodate hundreds of thousands of sandboxes per cluster, DSec tackles two resource bottlenecks. First, it mitigates duplicate page-cache footprints in virtualized environments and applies memory reclamation to enable safe overcommitment. Second, it alleviates spinlock contention in the container runtime and therefore, reduces per-sandbox CPU overhead, significantly increasing per-host packing density.

高并发下的密度优化。为了支持每个集群数以万计的沙箱，DSec 解决了两个资源瓶颈。首先，它减轻了虚拟化环境中页面缓存的重复占用，并通过内存回收来实现安全的过量提交。其次，它缓解了容器运行时中的自旋锁竞争，从而减少了每个沙箱的 CPU 开销，显著提高了每台主机的打包密度。

Trajectory Logging and Preemption-Safe Resumption. DSec maintains a globally ordered trajectory log for each sandbox, persistently recording every command invocation and its results. The trajectory serves three purposes: (1) client fast-forwarding — when a training task is preempted, sandbox resources are retained nonetheless; upon resumption, DSec replays cached results for previously completed commands, accelerating task recovery whilst also preventing errors from re-execution of non-idempotent operations; (2) fine-grained provenance — the origin and corresponding outcomes of each state change are traceable; (3) deterministic replay — any historical session can be faithfully reproduced from its trajectory.

轨迹记录与抢占安全恢复。DSec 为每个沙箱维护一个全局有序的轨迹日志，持久化记录每一个命令调用

及其结果。轨迹具有三个用途：(1) 客户端快进 —— 当训练任务被抢占时，沙箱资源仍然保留；恢复时，DSec 会回放之前完成命令的缓存结果，加快任务恢复，同时也能防止非幂等操作重复执行导致的错误；(2) 精细粒度的溯源 —— 每个状态变化的来源及其对应的结果均可追溯；(3) 确定性重放 —— 任何历史会话都可以从其轨迹中准确地重现。

5.3. Standard Benchmark Evaluation

5.3. 标准基准评估

5.3.1. Evaluation Setup

5.3.1. 评估设置

Knowledge and Reasoning. Knowledge and reasoning datasets include MMLU-Pro (Wang et al., 2024b), GPQA (Rein et al., 2023), Human Last Exam (Phan et al., 2025), Simple-QA Verified (Haas et al., 2025), Chinese-SimpleQA (He et al., 2024), LiveCodeBench-v6 (Jain et al., 2024), CodeForces (Internal Benchmark), HMMT 2026 Feb, Apex (Balunović et al., 2025), Apex Shortlist (Balunović et al., 2025), IMOAnswerBench (Luong et al., 2025), and PutnamBench (Tsoukalas et al., 2024).

知识与推理。知识与推理数据集包括 MMLU-Pro (Wang 等，2024b)，GPQA (Rein 等，2023)，Human Last Exam (Phan 等，2025)，Simple-QA Verified (Haas 等，2025)，Chinese-SimpleQA (He 等，2024)，LiveCodeBench-v6 (Jain 等，2024)，CodeForces (内部基准)，HMMT 2026 Feb，Apex (Balunović 等，2025)，Apex Shortlist (Balunović 等，2025)，IMOAnswerBench (Luong 等，2025)，以及 PutnamBench (Tsoukalas 等，2024)。

For code, we evaluate DeepSeek-V4 series on LiveCodeBench-v6 and an internal Codeforces benchmark. For Codeforces, we collect 14 Codeforces Division 1 contests comprising 114 problems (May 2025 - November 2025). The Elo rating is computed as follows. For each contest, we generate 32 candidate solutions per problem. For each problem independently, we sample 10 of these solutions without replacement and arrange them in a random order to form the submission sequence. Each submission is judged against a test suite constructed by domain experts. The score for a solved problem follows the penalty scheme of OpenAI (2025): the model receives the median score of human participants who solved the same problem with the same number of prior failed attempts. This yields a total contest score for each sampled submission sequence, which is then converted into a contest rank and subsequently into an estimated rating via the standard Codeforces rating system. The contest-level expected rating is defined as the expectation of this estimated rating over all possible random selections and orderings of the 10 submissions per problem. The model's overall rating is the average of these contest-level expected ratings across all 14 contests.

对于代码任务，我们在 LiveCodeBench-v6 和一个内部的 Codeforces 测试集上评估 DeepSeek-V4 系列模型。对于 Codeforces 测试集，我们收集了 14 个 Codeforces Division 1 比赛，共计 114 道题目（2025 年 5 月至 11 月）。Elo 等级的计算方式如下。对于每个比赛，我们为每道题目生成 32 个候选解法。对于每道题目独立地，我们从中随机抽取 10 个解法（不放回），并以随机顺序排列，形成提交序列。每个提交都会与由领域专家构建的测试集进行评估。解决一道题目的得分遵循 OpenAI（2025）的惩罚机制：模型获得的是那些以相同失败次数尝试后成功解决同一题目的人类参与者的中位数得分。这会为每个采样的提交序列生成一个总比赛得分，然后将其转换为比赛排名，再通过标准的 Codeforces 等级系统转换为估计等级。比赛级别的预期等级定义为在每道题目上所有可能的 10 个提交的随机选择和排序的期望估计等级。模型的整体等级是这些比赛级别预期等级在 14 场比赛上的平均值。

For reasoning and knowledge tasks, we set the temperature to 1.0 and the context window to 8K, 128K, and 384K tokens for the Non-think, High, and Max modes, respectively. For math tasks (e.g., HMMT, IMOAnswerBench, Apex, and HLE), we evaluate using the following template: "{question}\nPlease reason step by step, and put your final answer within

对于推理和知识任务，我们分别将温度设置为 1.0，并将上下文窗口设置为 8K、128K 和 384K 个标记，对

应 Non-think、High 和 Max 模式。对于数学任务（例如 HMMT、IMOAnswerBench、Apex 和 HLE），我们使用以下模板进行评估："{question}\n 请逐步推理，并将最终答案放在

\boxed{}. " For DeepSeek-V4-Pro-Max on math tasks, we use the following template to elicit deeper reasoning: "Solve the following problem. The problem may ask you to prove a statement, or ask for an answer. If finding an answer is required, you should come up with the answer, and your final solution should also be a rigorous proof of that answer being valid.\n\n{question}".

\boxed{}. "对于 DeepSeek-V4-Pro-Max 在数学任务上的应用，我们使用以下模板来激发更深层次的推理：" 解决以下问题。该问题可能要求你证明一个陈述，或要求你得出一个答案。如果需要找到答案，你应该提出答案，并且你的最终解决方案也应是该答案有效性的严谨证明。\n\n{question}"。

For formal math tasks, we evaluate in an agentic setting on Lean v4.28.0-rc1 (Moura and Ullrich, 2021), with access to the Lean compiler and a semantic tactic search engine, running up to 500 tool calls with max reasoning effort. In addition, we evaluate a more compute-intensive pipeline in which candidate natural-language solutions are first generated and filtered by self-verification(Shao et al., 2025), and the retained solutions are then provided as guidance to a formal agent for proving the corresponding Lean statement. This design uses informal reasoning to improve exploration while preserving strict correctness through formal verification. A submission is counted as correct only if the strict verifier Comparator accepts it for both settings.

对于正式的数学任务，我们在 Lean v4.28.0-rc1（Moura 和 Ullrich, 2021）中以代理设置进行评估，可以访问 Lean 编译器和语义策略搜索引擎，最多运行 500 次工具调用，并具有最大推理努力。此外，我们还评估了一个计算量更大的流水线，在该流水线中，候选的自然语言解决方案首先由自我验证（Shao 等人，2025）生成并过滤，然后保留的解决方案作为指导提供给正式代理以证明相应的 Lean 声明。这种设计通过形式化验证保持严格正确性，同时利用非正式推理来提高探索能力。只有当严格验证器 Comparator 在两种设置下都接受提交时，该提交才被视为正确。

We have left some entries blank for K2.6 and GLM-5.1, as their APIs were too busy to return responses to our queries.

我们在 K2.6 和 GLM-5.1 中留了一些条目为空，因为它们的 API 太忙了，无法回应我们的查询。

1M-Token Context. Since DeepSeek-V4 series supports 1M-token contexts, we evaluate model performance in a long context scenario by selecting OpenAI MRCR (OpenAI, 2024b) and CorpusQA (Lu et al., 2026) as the benchmarks. We re-evaluate Claude Opus 4.6 and Gemini 3.1 Pro on these tasks with the goal of standardizing the configuration across all models. We did not evaluate GPT-5.4 because its API failed to respond to a large portion of our queries.

1M-Token 上下文。由于 DeepSeek-V4 系列支持 1M-token 的上下文，我们通过选择 OpenAI MRCR（OpenAI, 2024b）和 CorpusQA（Lu 等人，2026）作为基准，来评估模型在长上下文场景下的性能。我们重新评估了 Claude Opus 4.6 和 Gemini 3.1 Pro 在这些任务上的表现，目的是在所有模型之间标准化配置。我们没有评估 GPT-5.4，因为其 API 未能对我们的大部分查询做出响应。

Agent. Agent datasets include Terminal Bench 2.0 (Merrill et al., 2026), SWE-Verified (OpenAI, 2024e), SWE Multilingual (Yang et al., 2025), SWE-Pro (Deng et al., 2025), BrowseComp (Wei et al., 2025), the public evaluation set of MCPAtlas (Bandi et al., 2026), GDPval-AA (AA, 2025; Patwardhan et al., 2025), and Tool-Decathlon (Li et al., 2025).

Agent. Agent 数据集包括 Terminal Bench 2.0 (Merrill 等人，2026)，SWE-Verified (OpenAI，2024e)，SWE Multilingual (Yang 等人，2025)，SWE-Pro (Deng 等人，2025)，BrowseComp (Wei 等人，2025)，MCPAtlas 公共评估集 (Bandi 等人，2026)，GDPval-AA (AA，2025; Patwardhan 等人，2025)，以及 Tool-Decathlon (Li 等人，2025)。

For code agent tasks (SWE-Verified, Terminal-Bench, SWE-Pro, SWE Multilingual), we evaluate DeepSeek-V4 series using an internally developed evaluation framework. This framework provides a minimal set of tools — a bash tool and a file-edit tool. The maximum number of interaction steps is set to 500, and the maximum context length is set to 512K tokens. Regarding Terminal-Bench 2.0, we acknowledge the environment-related issues noted by GLM-5.1. Never-

theless, we report our performance on the original Terminal-Bench 2.0 dataset for consistency. On the Terminal-Bench 2.0 Verified subset, DeepSeek-V4-Pro achieves a score of approximately 72.0.

对于代码代理任务（SWE-Verified、Terminal-Bench、SWE-Pro、SWE Multilingual），我们使用内部开发的评估框架来评估 DeepSeek-V4 系列模型。该框架提供了一组最小的工具——一个 bash 工具和一个文件编辑工具。交互步骤的最大数量设置为 500，上下文长度的最大值设置为 512K 个 token。关于 Terminal-Bench 2.0，我们承认 GLM-5.1 指出的环境相关问题。然而，为了保持一致性，我们报告了在原始 Terminal-Bench 2.0 数据集上的性能表现。在 Terminal-Bench 2.0 验证子集上，DeepSeek-V4-Pro 的得分为约 72.0。

For search agent tasks (BrowseComp, HLE w/ tool), we also use an in-house harness with web-search and Python tool, and set maximum interaction steps to 500 and the maximum context length to 512K tokens. For BrowseComp, we use the same discard-all context management strategy as DeepSeek-V3.2 (DeepSeek-AI, 2025).

对于搜索代理任务（BrowseComp、带工具的 HLE），我们还使用内部开发的测试框架，结合网络搜索和 Python 工具，并将最大交互步骤设置为 500 步，最大上下文长度设置为 512K 个标记。对于 BrowseComp，我们采用与 DeepSeek-V3.2（DeepSeek-AI，2025）相同的丢弃全部上下文管理策略。

5.3.2. Evaluation Results

5.3.2. 评估结果

The comparison of DeepSeek-V4-Pro-Max and other closed/open source models is presented in Table 6. Also, we evaluate different modes of DeepSeek-V4-Flash and DeepSeek-V4-Pro and show the results in Table 7.

DeepSeek-V4-Pro-Max 与其他闭源/开源模型的对比结果见表 6。此外，我们评估了 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的不同模式，并将结果展示在表 7 中。

Table 6 | Comparison between DeepSeek-V4-Pro-Max and closed/open source models. “Max”, “xHigh”, and “High” denote reasoning effort. The best results are highlighted in bold; the second-best results are underlined.

表 6 | DeepSeek-V4-Pro-Max 与闭源/开源模型的对比。“Max”、“xHigh” 和 “High” 表示推理努力程度。最佳结果以粗体突出显示；次佳结果以下划线表示。

Benchmark (Metric)		Opus-4.6	GPT-5.4	Gemini-3.1-Pro	
Max	xHigh	High	Thinking	Think	
Knowledge & Reasoning	MMLU-Pro (EM)	89.1	87.5	91.0	
SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	
Chinese-SimpleQA (Pass@1)	76.4	76.8	85.9	75.9	
GPQA Diamond (Pass@1)	91.3	93.0	94.3	90.5	
HLE (Pass@1)	40.0	39.8	44.4	36.4	
LiveCodeBench (Pass@1)	88.8	-	91.7	89.6	
Codeforces (Rating)	-	3168	3052	-	
HMMT 2026 Feb (Pass@1)	96.2	97.7	94.7	92.7	
IMOAnswerBench (Pass@1)	75.3	91.4	81.0	86.0	
Apex (Pass@1)	34.5	54.1	60.9	24.0	
Apex Shortlist (Pass@1)	85.9	78.1	89.1	75.5	

Long	MRCR 1M (MMR)	92.9	-	76.3
CorpusQA 1M (ACC)	71.7	-	53.8	-
Agentic	Terminal Bench 2.0 (Acc)	65.4	75.1	68.5
SWE Verified (Resolved)	80.8	-	80.6	80.2
SWE Pro (Resolved)	57.3	57.7	54.2	58.6
SWE Multilingual (Resolved)	77.5	-	-	76.7
BrowseComp (Pass@1)	83.7	82.7	85.9	83.2
HLE w/ tools (Pass@1)	53.1	52.0	51.6	54.0
GDPval-AA (Elo)	1619	1674	1314	1482
MCPAtlas Public(Pass@1)	73.8	67.2	69.2	66.6
Toolathlon (Pass@1)	47.2	54.6	48.8	50.0

基准（指标）		Opus-4.6	GPT-5.4	Gemini-3.1-Pro		K2.6 GLM-5
马克斯	xHigh	高	思考	思考	马克斯	
知识与推理	MMLU-Pro (EM)	89.1	87.5	91.0	87.1	
SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	38.1	
中文简易问答（Pass@1）	76.4	76.8	85.9	75.9	75.0	
GPQA Diamond (Pass@1)	91.3	93.0	94.3	90.5	86.2	
HLE (Pass@1)	40.0	39.8	44.4	36.4	34.7	
LiveCodeBench (Pass@1)	88.8	-	91.7	89.6	-	
Codeforces (评级)	-	3168	3052	-	-	
HMMT 2026 Feb (Pass@1)	96.2	97.7	94.7	92.7	89.4	
IMOAnswerBench (Pass@1)	75.3	91.4	81.0	86.0	83.8	
Apex (Pass@1)	34.5	54.1	60.9	24.0	11.5	
Apex 短名单（Pass@1）	85.9	78.1	89.1	75.5	72.4	
长	MRCR 1M (MMR)	92.9	-	76.3	-	
CorpusQA 1M (ACC)	71.7	-	53.8	-	-	
代理性	终端基准 2.0 (Acc)	65.4	75.1	68.5	66.7	
SWE 已验证（已解决）	80.8	-	80.6	80.2	-	
SWE Pro (已解决)	57.3	57.7	54.2	58.6	58.4	
SWE 多语言（已解决）	77.5	-	-	76.7	73.3	
浏览比较（Pass@1）	83.7	82.7	85.9	83.2	79.3	
HLE 与工具（Pass@1）	53.1	52.0	51.6	54.0	50.4	
GDPval-AA (Elo)	1619	1674	1314	1482	1535	
MCPAtlas 公共版（Pass@1）	73.8	67.2	69.2	66.6	71.8	
工具竞赛（Pass@1）	47.2	54.6	48.8	50.0	40.7	

Knowledge. In the evaluation of general world knowledge, DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, establishes a new state-of-the-art among open-source large language models. As demonstrated by the SimpleQA-Verified, DeepSeek-V4-Pro-Max significantly outperforms all existing open-source baselines by a margin of 20 absolute percentage

points. Despite these advances, it currently trails the leading proprietary model, Gemini-3.1-Pro. In the domain of educational knowledge and reasoning, DeepSeek-V4-Pro-Max marginally outperforms Kimi and GLM across the MMLU-Pro, GPQA, and HLE benchmarks, although it lags behind leading proprietary models. Broadly, DeepSeek-V4-Pro-Max marks a significant milestone in enhancing the world knowledge capabilities of open-source models.

知识。在一般世界知识评估中，DeepSeek-V4-Pro-Max，即 DeepSeek-V4-Pro 的最大推理努力模式，在开源大语言模型中建立了新的最先进水平。如 SimpleQA-Verified 所示，DeepSeek-V4-Pro-Max 在所有现有开源基准上表现出色，优势达到 20 个百分点。尽管取得了这些进展，它目前仍落后于领先的专有模型 Gemini-3.1-Pro。在教育知识和推理领域，DeepSeek-V4-Pro-Max 在 MMLU-Pro、GPQA 和 HLE 基准上略微优于 Kimi 和 GLM，尽管它仍落后于领先的专有模型。总体而言，DeepSeek-V4-Pro-Max 标志着在增强开源模型的世界知识能力方面取得的重要里程碑。

In addition, a significant performance gap exists between DeepSeek-V4-Flash and DeepSeek-V4-Pro on knowledge-based tasks; this is anticipated, as larger parameter counts facilitate greater knowledge retention during pre-training. Notably, both models demonstrate improved results on knowledge benchmarks when allocated higher reasoning effort.

此外，DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 在基于知识的任务中存在显著的性能差距；这是可以预见的，因为更大的参数量在预训练过程中能够更好地保留知识。值得注意的是，当分配更高的推理努力时，两个模型在知识基准测试中都表现出更好的结果。

Reasoning. DeepSeek-V4-Pro-Max outperforms all prior open models across reasoning benchmarks, and matches state-of-the-art closed models on many metrics, while the smaller DeepSeek-V4-Flash-Max also surpasses the previous best open-source model, K2.6-Thinking, on code and math reasoning tasks. Meanwhile, DeepSeek-V4-Pro and DeepSeek-V4-Flash excel in cod-

推理。DeepSeek-V4-Pro-Max 在推理基准测试中表现优于所有先前的开放模型，并在许多指标上与最先进的封闭模型相匹配。同时，较小的 DeepSeek-V4-Flash-Max 也在代码和数学推理任务上超越了之前的最佳开源模型 K2.6-Thinking。与此同时，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 在代码

Table 7 | Comparison among different sizes and modes of DeepSeek-V4 series. “Non-Think”, “High”, and “Max” denote reasoning effort.

表 7 | 不同尺寸和模式的 DeepSeek-V4 系列对比。“Non-Think”、“High” 和 “Max” 表示推理努力程度。

	Benchmark (Metric)	DeepSeek-V4-Flash			DeepSeek
Non-Think	High	Max	Non-Think	High	Max
Knowledge & Reasoning	MMLU-Pro (EM)	83.0	86.4	86.2	82.9
	SimpleQA-Verified (Pass@1)	23.1	28.9	34.1	45.0	46.2
	Chinese-SimpleQA (Pass@1)	71.5	73.2	78.9	75.8	77.7
	GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1
	HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5
	LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8
	Codeforces (Rating)	-	2816	3052	-	2919
	HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0
	IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0
	Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4
	Apex Shortlist (Pass@1)	9.3	72.1	85.7	9.2	85.5
Long	MRCR 1M(MMR)	37.5	76.9	78.7	44.7
	CorpusQA 1M(ACC)	15.5	59.3	60.5	35.6	56.5
Agentic	Terminal Bench 2.0 (Acc)	49.1	56.6	56.9	59.1

SWE Verified (Resolved)	73.7	78.6	79.0	73.6	79.4
SWE Pro (Resolved)	49.1	52.3	52.6	52.1	54.4
SWE Multilingual (Resolved)	69.7	70.2	73.3	69.8	74.1
BrowseComp (Pass@1)	-	53.5	73.2	-	80.4
HLE w/ tools (Pass@1)	-	40.3	45.1	-	44.7
MCPAtlas Public (Pass@1)	64.0	67.4	69.0	69.4	74.2
GDPval-AA (Elo)	-	-	1395	-	-
Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0

	基准（指标）	DeepSeek-V4-Flash			DeepSeek-V4-Pro	
非思考	高	马克斯	非思考	高	马克斯	
知识与推理	MMLU-Pro (EM)	83.0	86.4	86.2	82.9	87
简单问答验证（通过 @1）	23.1	28.9	34.1	45.0	46.2	57
中文-简单问答（Pass@1）	71.5	73.2	78.9	75.8	77.7	84
GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1	90
HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5	37
LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8	93
Codeforces (评级)	-	2816	3052	-	2919	32
HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0	95
IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0	89
Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4	38
Apex 短名单（Pass@1）	9.3	72.1	85.7	9.2	85.5	90
长	MRCR 1M(MMR)	37.5	76.9	78.7	44.7	83
CorpusQA 1M(ACC)	15.5	59.3	60.5	35.6	56.5	62
代理性	终端基准 2.0 (Acc)	49.1	56.6	56.9	59.1	63
SWE 验证（已解决）	73.7	78.6	79.0	73.6	79.4	80
SWE Pro (已解决)	49.1	52.3	52.6	52.1	54.4	55
SWE 多语言（已解决）	69.7	70.2	73.3	69.8	74.1	76
浏览比较（Pass@1）	-	53.5	73.2	-	80.4	83
HLE 与工具（Pass@1）	-	40.3	45.1	-	44.7	48
MCPAtlas 公共版 (Pass@1)	64.0	67.4	69.0	69.4	74.2	73
GDPval-AA (Elo)	-	-	1395	-	-	15
Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0	51

ing competitions. According to our evaluation, their performance is comparable to GPT-5.4, making this the first time an open model has matched a closed model on this task. On the Codeforces leaderboard, DeepSeek-V4-Pro-Max currently ranks 23rd among human candidates. DeepSeek-V4 also demonstrates strong performance on formal mathematical task under both agentic and compute-intensive settings. Under an agentic setup, it achieves state-of-the-art results, shown in Figure 8, outperforming prior models such as Seed Prover (Chen et al., 2025). With a more compute-intensive pipeline, performance further improves, surpassing systems including Aristotle(Achim et al., 2025) and matching the best known results under this setting.

编程竞赛。根据我们的评估，其表现与 GPT-5.4 相当，这使得这是首次开放模型在该任务上能够与封闭模型相媲美。在 Codeforces 排名榜上，DeepSeek-V4-Pro-Max 目前在人类候选者中排名第 23 位。DeepSeek-V4 在正式数学任务中也表现出色，无论是在自主性设置还是计算密集型设置下。在自主性设置下，它取得了最先进的结果，如图 8 所示，其表现优于之前的模型，如 Seed Prover（Chen 等，2025）。在更计算密集型的流程下，性能进一步提升，超过了包括 Aristotle（Achim 等，2025）在内的系统，并在该设置下达到了目前已知的最佳结果。

Agent. The DeepSeek-V4 series demonstrates strong agent performance in evaluations. For code agent tasks, DeepSeek-V4-Pro achieves results comparable to K2.6 and GLM-5.1, though all these open models still lag behind their closed-source counterparts. DeepSeek-V4-Flash underperforms DeepSeek-V4-Pro on coding tasks, particularly on Terminal Bench 2.0. A similar trend is observed across other agent evaluations. It is worth noting that DeepSeek-V4-Pro performs well on MCPAtlas and Toolathlon—two evaluation test sets that include a wide range of tools and MCP services—indicating that our model has excellent generalization capability and does not perform well only on internal frameworks.

代理。DeepSeek-V4 系列在评估中展现出强大的代理性能。在代码代理任务中，DeepSeek-V4-Pro 取得的成绩与 K2.6 和 GLM-5.1 相当，尽管所有这些开源模型仍然落后于其闭源对应模型。在编码任务中，DeepSeek-V4-Flash 的表现逊于 DeepSeek-V4-Pro，尤其是在 Terminal Bench 2.0 上。在其他代理评估中也观察到了类似的趋势。值得一提的是，DeepSeek-V4-Pro 在 MCPAtlas 和 Toolathlon 两个评估测试集上表现良好，这两个测试集包含大量工具和 MCP 服务，表明我们的模型具有出色的泛化能力，并不仅仅在内部框架上表现良好。

1M-Token Context. DeepSeek-V4-Pro outperforms Gemini-3.1-Pro on the MRCR task, which measures in-context retrieval, but remains behind Claude Opus 4.6. As illustrated in Figure 9, retrieval performance remains highly stable within a 128K context window. While a performance

1M-Token 上下文。DeepSeek-V4-Pro 在 MRCR 任务上表现优于 Gemini-3.1-Pro，该任务用于衡量上下文检索性能，但在与 Claude Opus 的对比中仍落后 4.6 分。如图 9 所示，在 128K 上下文窗口内，检索性能保持高度稳定。尽管性能

Practical Regime
Putnam-200 Pass@8 with minimal tools and bounded sampling.

实用制度
Putnam-200 Pass@8 使用最少工具和有限采样。

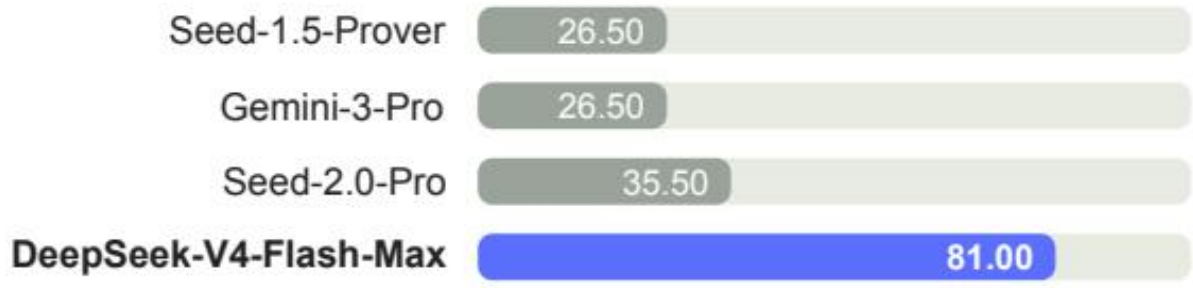

Figure 9: images/b603cc0e1e07d1ff666e5f20a7e79b15801600cb952a393cfe6a7b7287b696f8.jpg

Frontier Regime
Putnam-2025 with hybrid formal-informal reasoning and substantial compute scaling.

前沿体制
Putnam-2025，结合形式与非形式推理的混合模式，并具备显著的计算扩展能力。

Figure 10: images/24f10280d0e811fe36573688f60cae9c8e7eff64e8e24d1259cdccfeb00f1bcc.jpg

Figure 8 | Formal reasoning under practical and frontier regimes. Left: Putnam-200 Pass@8 evaluates a fixed random subset of PutnamBench (Tsoukalas et al., 2024) following the setup introduced by Seed-Prover; all models are tested on the same problem set. We follow the Seed-Prover protocol but replace proprietary search tools with the open-source LeanExplore (Asher, 2025), yielding a lightweight setting with minimal agent tools and bounded sampling. Right: Putnam-2025 probes the frontier of mathematical reasoning in a scaled hybrid formal-informal regime, where informal reasoning is combined with formal verification to expose gaps and improve rigor; DeepSeek-V4 reaches a proof-perfect 120/120.

图 8 | 在实际和前沿制度下的形式推理。左：Putnam-200 Pass@8 评估 PutnamBench（Tsoukalas 等，2024）中固定的随机子集，按照 Seed-Prover 引入的设置进行；所有模型都在相同的题目集上进行测试。我们遵循 Seed-Prover 协议，但将专有搜索工具替换为开源的 LeanExplore（Asher，2025），从而实现一个轻量级的设置，使用最少的代理工具和有限的采样。右：Putnam-2025 在一个扩展的混合形式-非形式推理制度中探索数学推理的前沿，其中非形式推理与形式验证相结合，以揭示差距并提高严谨性；DeepSeek-V4 达到了 120/120 的完美证明。

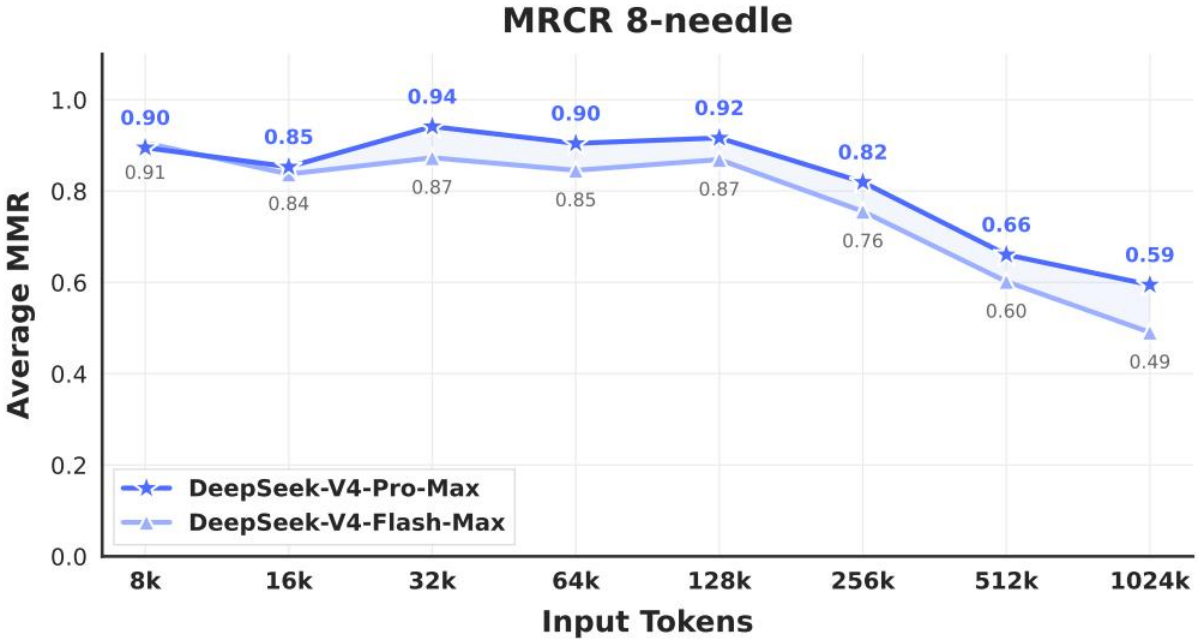

Figure 11: images/7d42c248d6061a455bd6da0c98c1994e396b4494a4f1fea430a776125d989293.jpg

Figure 9 | DeepSeek-V4 series performance on the MRCR task.

图 9 | DeepSeek-V4 系列在 MRCR 任务中的表现。

degradation becomes visible beyond the 128K mark, the model's retrieval capabilities at 1M tokens remain remarkably strong compared to both proprietary and open-source counterparts. Unlike MRCR, CorpusQA is similar to real scenarios. The evaluation results also indicate that DeepSeek-V4-Pro is better than Gemini-3.1-Pro.

在超过 128K 的标记后，退化现象变得明显，但模型在 1M tokens 下的检索能力仍然显著优于自有和开源模型。与 MRCR 不同，CorpusQA 更贴近真实场景。评估结果还表明，DeepSeek-V4-Pro 比 Gemini-3.1-Pro 更好。

Reasoning Effort. As shown in Table 7, the Max mode, which employs longer contexts and reduced length penalties in RL, outperforms the High mode on the most challenging tasks. Figure 10 presents a comparison of performance and cost among DeepSeek-V4-Pro, DeepSeek-V4-Flash, and DeepSeek-V3.2 on representative reasoning and agentic tasks. By scaling test-time compute, DeepSeek-V4 series achieve substantial improvements over the predecessor. Furthermore, on reasoning tasks like HLE, DeepSeek-V4-Pro demonstrates higher token efficiency than DeepSeek-V3.2.

推理努力。如表 7 所示，采用更长上下文和减少长度惩罚的 Max 模式在最具挑战性的任务上优于 High 模式。图 10 展示了 DeepSeek-V4-Pro、DeepSeek-V4-Flash 和 DeepSeek-V3.2 在代表性推理和自主任务上的性能和成本比较。通过扩展测试时的计算量，DeepSeek-V4 系列在性能上显著优于前代产品。此外，在 HLE 等推理任务中，DeepSeek-V4-Pro 的 token 效率高于 DeepSeek-V3.2。

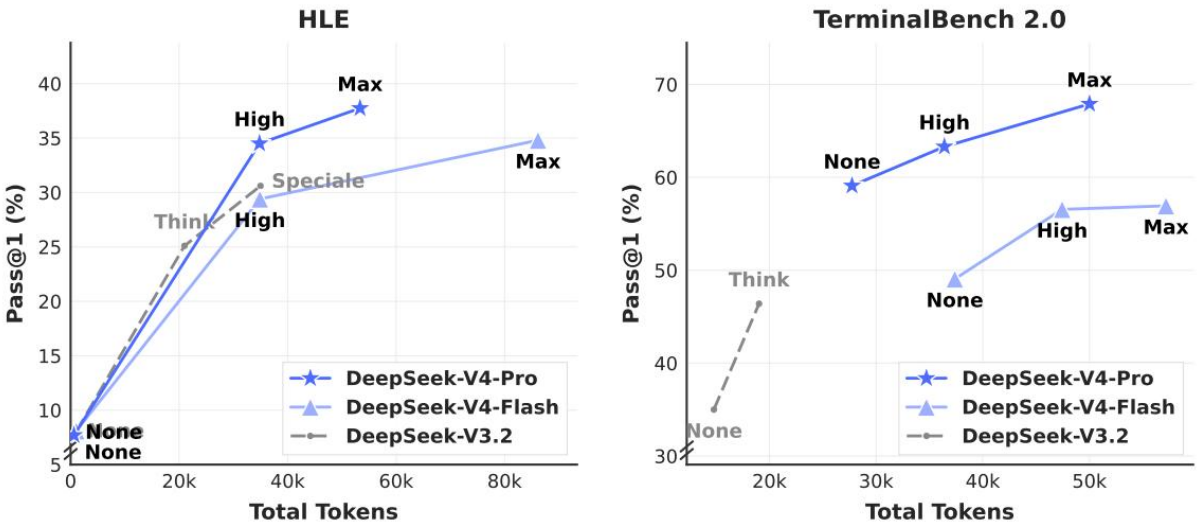

Figure 10 | HLE and Terminal Bench 2.0 performance by reasoning effort. "None" indicates Non-think mode, and "Speciale" indicates DeepSeek-V3.2-Speciale model.

图 10 | 通过推理努力度划分的 HLE 和终端基准 2.0 性能。"None" 表示非思考模式，而 "Speciale" 表示 DeepSeek-V3.2-Speciale 模型。

5.4. Performance on Real-World Tasks

5.4. 在现实任务中的表现

Standardized benchmarks often struggle to capture the complexities of diverse, real-world tasks, creating a gap between test results and actual user experience. To bridge this, we have developed proprietary internal metrics that prioritize real-world usage patterns over traditional benchmarks. This approach ensures that our optimizations translate into tangible benefits. Our evaluation framework specifically targets the primary use cases of the DeepSeek API and Chatbot, aligning model performance with practical demands.

标准化的基准测试通常难以捕捉多样化的现实任务的复杂性，从而导致测试结果与实际用户体验之间存在

差距。为弥合这一差距，我们开发了专有的内部指标，这些指标优先考虑现实使用模式，而非传统的基准测试。这种方法确保我们的优化能够转化为实际的效益。我们的评估框架特别针对 DeepSeek API 和 Chatbot 的主要使用场景，将模型性能与实际需求相匹配。

5.4.1. Chinese Writing

5.4.1. 汉字书写

One of the primary use cases for DeepSeek is Chinese writing. We conducted a rigorous evaluation on functional writing and creative writing. Table 12 presents a pairwise comparison between DeepSeek-V4-Pro and Gemini-3.1-Pro on functional writing tasks. These tasks consist of common daily writing queries, where prompts are typically concise and straightforward. Gemini-3.1-Pro was selected as the baseline, as it stands as the top-performing external model for Chinese writing in our evaluations. The results indicate that DeepSeek-V4-Pro outperforms the baseline with an overall win rate of 62.7% versus 34.1% ; this is primarily because Gemini occasionally allows its inherent stylistic preferences to override the user's explicit requirements in Chinese writing scenarios.

DeepSeek 的一个主要用例是中文写作。我们在功能性写作和创意写作方面进行了严格的评估。表 12 展示了 DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在功能性写作任务上的对比。这些任务包括常见的日常写作查询，提示通常简洁明了。Gemini-3.1-Pro 被选为基准模型，因为它在我们的评估中是中文写作表现最好的外部模型。结果显示，DeepSeek-V4-Pro 的整体胜率是 62.7% 对 34.1%，这主要是因为 Gemini 在中文写作场景中偶尔会允许其固有的风格偏好覆盖用户的明确要求。

Table 13 presents the creative writing comparison, which is evaluated along two axes: instruction following and writing quality. Compared with Gemini-3.1-Pro, DeepSeek-V4-Pro achieves a 60.0% win rate in instruction following and 77.5% in writing quality, demonstrating a marginal improvement in instruction following and a substantial gain in writing quality. Although DeepSeek-V4-Pro yields superior results in aggregate user case analysis, an evaluation restricted to the most challenging prompts — specifically those involving high-complexity constraints or multi-turn scenarios — reveals that Claude Opus 4.5 retains a performance advantage over DeepSeek-V4-Pro. As shown in Table 14, Claude Opus 4.5 achieves a 52.0% win rate versus 45.9%.

表 13 展示了创意写作的对比，评估维度包括遵循指令和写作质量。与 Gemini-3.1-Pro 相比，DeepSeek-V4-Pro 在遵循指令方面胜率高达 60.0%，在写作质量方面胜率高达 77.5%，显示出在遵循指令方面有小幅提升，在写作质量方面有显著提高。尽管 DeepSeek-V4-Pro 在综合用户案例分析中表现出色，但当评估局限于最具挑战性的提示——特别是涉及高复杂性约束或多轮对话场景的提示时，结果显示 Claude Opus 4.5 在性能上仍优于 DeepSeek-V4-Pro。如表 14 所示，Claude Opus 4.5 的胜率高达 52.0%，而 DeepSeek-V4-Pro 的胜率为 45.9%。

5.4.2. Search

5.4.2. 搜索

Search-augmented question answering is a core capability of the DeepSeek chatbot. On the DeepSeek web and app, the "non-think" mode employs Retrieval-Augmented Search (RAG), whereas the "thinking" mode utilizes agentic search.

搜索增强型问答是 DeepSeek 聊天机器人的核心能力。在 DeepSeek 的网页和应用程序中，"非思考" 模式采用检索增强搜索（RAG），而 "思考" 模式则使用自主搜索。

Retrieval Augmented Search. We conducted a pairwise evaluation comparing DeepSeek-V4-Pro and DeepSeek-V3.2 across both objective and subjective Q&A categories. As presented in Table 11, DeepSeek-V4-Pro outperforms DeepSeek-V3.2 by a substantial margin, demonstrating a consistent advantage across both categories. The most pronounced gains are observed in single-

value search and planning & strategy tasks, suggesting that DeepSeek-V4-Pro excels at locating precise factual answers and synthesizing structured plans from retrieved context. However, DeepSeek-V3.2 remains relatively competitive on comparison and recommendation tasks, indicating potential room for improvement for DeepSeek-V4-Pro in scenarios requiring balanced, multi-perspective reasoning over search results.

检索增强搜索。我们进行了对比评估，比较了 DeepSeek-V4-Pro 和 DeepSeek-V3.2 在客观和主观问答类别中的表现。如表 11 所示，DeepSeek-V4-Pro 在两个类别中均表现出显著优势，显示出一致的优越性。最显著的提升出现在单值搜索和规划与策略任务中，这表明 DeepSeek-V4-Pro 在从检索上下文中定位精确事实答案和综合结构化计划方面表现出色。然而，DeepSeek-V3.2 在比较和推荐任务中仍相对具有竞争力，这表明 DeepSeek-V4-Pro 在需要对搜索结果进行平衡、多角度推理的场景中仍有改进空间。

Agentic Search. Unlike standard RAG, agentic search empowers the model to iteratively invoke search and fetch tools per query, significantly enhancing overall search performance. For the thinking mode in DeepSeek-Chat, we optimized the agentic search function to maximize response accuracy within a predefined "thinking budget". As shown in Table 9, agentic search consistently outperforms RAG, particularly on complex tasks. Furthermore, its cost remains highly efficient, with agentic search being only marginally more expensive than standard RAG (see Table 10).

代理搜索。与标准的 RAG 不同，代理搜索使模型能够根据查询依次调用搜索和获取工具，显著提升整体搜索性能。在 DeepSeek-Chat 的思考模式中，我们优化了代理搜索功能，以在预定义的 "思考预算" 内最大化响应准确性。如表 9 所示，代理搜索在复杂任务上始终优于 RAG。此外，其成本仍非常高效，代理搜索仅比标准 RAG 稍贵一些（见表 10）。

5.4.3. White-Collar Task

5.4.3. 白领任务

To rigorously evaluate the model's utility in sophisticated enterprise productivity scenarios, we constructed a comprehensive suite of 30 advanced Chinese professional tasks. These workflows deliberately encompass high-level cognitive demands, including in-depth information analysis, comprehensive document generation, and nuanced document editing, spanning a diverse spectrum of 13 critical industries (e.g., finance, education, law, and technology). The evaluation was conducted within an in-house agent harness equipped with basic tools, including Bash and web search.

为了严格评估模型在复杂企业生产力场景中的实用性，我们构建了一套包含 30 个高级中文专业任务的综合任务集。这些工作流程刻意涵盖了高水平的认知需求，包括深入的信息分析、全面的文档生成以及细致的文档编辑，覆盖了 13 个关键行业（如金融、教育、法律和技术）的多样化领域。评估工作是在内部代理系统中进行的，该系统配备了基础工具，包括 Bash 和网络搜索。

Given the open-ended nature of these tasks, automated metrics usually fall short in capturing the nuances of a high-quality response. Therefore, we conducted human evaluations to compare the performance of DeepSeek-V4-Pro-Max against Opus-4.6-Max. Annotators blindly assessed the model outputs across four dimensions:

鉴于这些任务的开放性，自动化的评估指标通常难以捕捉高质量回答的细微差别。因此，我们进行了人工评估，以比较 DeepSeek-V4-Pro-Max 与 Opus-4.6-Max 的表现。标注者在四个维度上盲评了模型的输出：

- Task Completion: Whether the core problem was successfully resolved.
- Instruction Following: Adherence to specific constraints and directives.
- Content Quality: Factual accuracy, logical coherence, and professional tone.

- Formatting Aesthetics: Layout readability and visual presentation.
- 任务完成度：核心问题是否已成功解决。
- 指令遵循：是否遵守了特定的约束和指示。
- 内容质量：事实准确性、逻辑连贯性和专业语气。
- 格式美观度：版式可读性和视觉呈现。

As illustrated in Figure 11, DeepSeek-V4-Pro-Max outperforms Opus-4.6-Max on diverse Chinese white-collar tasks, achieving an impressive non-loss rate of 63%, and demonstrating consistent advantages across analysis, generation, and editing tasks. The detailed dimension scores shown in Figure 12 highlight the model's primary strengths in Task Completion and

如图 11 所示，DeepSeek-V4-Pro-Max 在多种中国白领任务中表现优于 Opus-4.6-Max，实现了令人印象深刻的非损失率 63%，并在分析、生成和编辑任务中展现出一致的优势。图 12 中显示的详细维度评分突出了模型在任务完成方面的主要优势。

Content Quality. Specifically, DeepSeek-V4-Pro-Max proactively anticipates implicit user intents by frequently providing supplementary insights and self-verification steps. It also excels in long-form generation, delivering in-depth, coherent narratives rather than relying on the overly simplistic bullet points frequently produced by Opus-4.6-Max. Additionally, the model strictly conforms to formal professional conventions, such as standardized Chinese hierarchical numbering. However, in terms of Instruction Following, it occasionally overlooks specific formatting constraints and slightly trails Opus. Furthermore, the model is less proficient at condensing extensive text inputs into succinct summaries. Finally, its Formatting Aesthetics still have substantial room for improvement regarding the overall visual design of presentation slides. Figure 13, 14, and 15 present several test cases; due to the extensive length of certain outputs, only partial pages are displayed.

内容质量。具体而言，DeepSeek-V4-Pro-Max 通过频繁提供补充见解和自我验证步骤，主动预判用户的隐含意图。它在长文本生成方面表现出色，能够提供深入且连贯的叙述，而不是像 Opus-4.6-Max 那样经常产生过于简单的要点列表。此外，该模型严格遵循正式的专业规范，例如标准化的中文层级编号。然而，在遵循指令方面，它偶尔会忽略特定的格式限制，并略微落后于 Opus。此外，该模型在将大量文本输入浓缩为简洁摘要方面不够熟练。最后，其格式美学在演示幻灯片的整体视觉设计方面仍有较大的改进空间。图 13、14 和 15 展示了几个测试案例；由于某些输出内容过长，仅显示了部分页面。

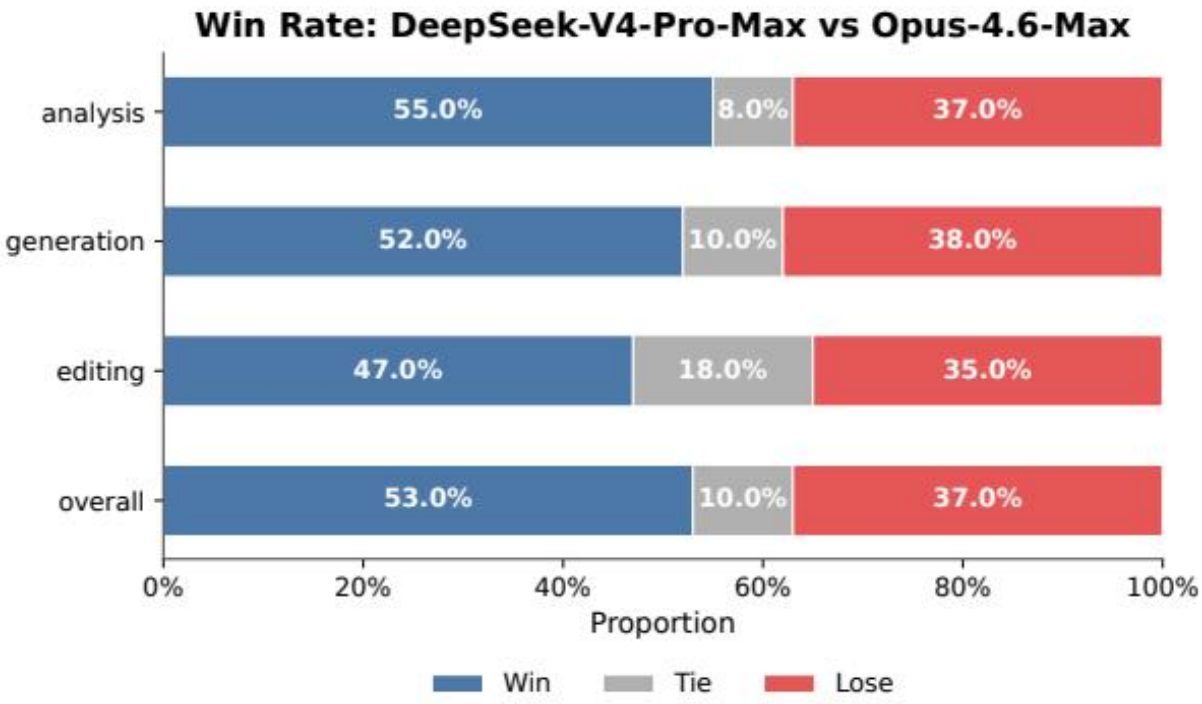

Figure 12: images/ab9f730bad1c71de889a13faaaabe0989b7fc544d03a022f6d9dd7f6fd18617a.jpg

Figure 11 | Win-rate comparison across analysis, generation, editing tasks, and the overall performance.

图 11 | 在分析、生成、编辑任务中的胜率对比以及整体表现。

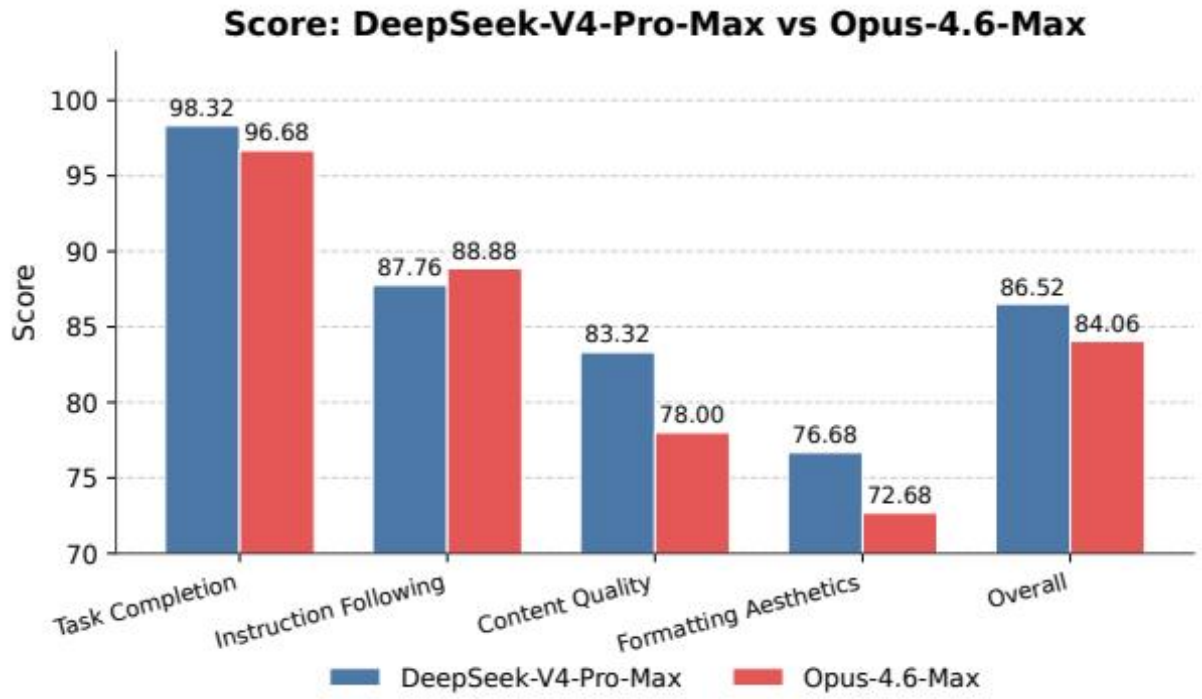

Figure 13: images/6d8fa4a8e28ca089961cba98f4265aba82ab7fc82f1b1509cf64c4c13e14e01f.jpg

Figure 12 | Detailed dimension scores including Task Completion, Content Quality, Formatting Aesthetics, and Instruction Following.

图 12 | 包括任务完成度、内容质量、格式美观度和指令遵循的详细评分。

Figure 13 | Example output of a task which requires drafting a joint marketing proposal for a popular bubble tea brand and the Beijing Subway.

图 13 | 一个需要为知名珍珠奶茶品牌和北京地铁起草联合营销提案的任务示例输出。

5.4.4. Code Agent

5.4.4. 代码代理

To benchmark our coding agent capability, we curate tasks from real internal R&D workloads. We collect ~200 challenging tasks from 50+ internal engineers, spanning feature development, bug fixing, refactoring, and diagnostics across diverse technology stacks including PyTorch, CUDA, Rust, and C++. Each task is accompanied by its original repository, the corresponding execution environment, and human-annotated scoring rubrics; after rigorous quality filtering, 30 tasks are retained as the evaluation set. As shown in Table 8, DeepSeek-V4-Pro significantly outperforms Claude Sonnet 4.5 and approaches the level of Claude Opus 4.5.

为了评估我们的编码代理能力，我们从真实的内部研发工作负载中精选任务。我们收集了来自 50 多名内部工程师的约 200 个具有挑战性的任务，涵盖特征开发、错误修复、重构和诊断等多个方面，涉及 PyTorch、CUDA、Rust 和 C++ 等多种技术栈。每个任务都附带有其原始仓库、相应的执行环境以及人类标注的评分标准；经过严格的质量筛选后，保留了 30 个任务作为评估集。如表 8 所示，DeepSeek-V4-Pro 显著优于 Claude Sonnet 4.5，并接近 Claude Opus 4.5 的水平。

Table 8 | Comparison on R&D Coding Benchmark (external models included strictly for evaluation purposes).

表 8 | 在研发编码基准上的比较（外部模型仅用于评估目的）。

Model	Haiku 4.5	Sonnet 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 Thinking	Opus 4.6 Thinking
Pass Rate (%)	13	47	67	70	73	80

模型	Haiku 4.5	十四行诗 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 思考	Opus 4.6 思考
通过率 (%)	13	47	67	70	73	80

In a survey asking DeepSeek developers and researchers $N = 85$ — all with experience of using DeepSeek-V4-Pro for agentic coding in their daily work — whether DeepSeek-V4-Pro is ready to serve as their default and primary coding model compared to other frontier models, 52% said yes, 39% leaned toward yes, and fewer than 9% said no. Respondents find DeepSeek-V4-Pro to deliver satisfactory results across most tasks, but note trivial mistakes, misinterpretation of vague prompts, and occasional over-thinking.

在一项调查中，询问了使用 DeepSeek-V4-Pro 进行代理编码的开发者和研究人员 $N = 85$ —— 所有人都有使用 DeepSeek-V4-Pro 进行日常工作的经验 —— 是否认为 DeepSeek-V4-Pro 相比其他前沿模型已经准备好作为他们的默认和主要编码模型，52% 的受访者表示同意，39% 的受访者倾向于同意，不到 9% 的受访者表示不同意。受访者认为 DeepSeek-V4-Pro 在大多数任务中都能提供令人满意的成果，但也指出存在一些微不足道的错误、对模糊提示的误解以及偶尔的过度思考。

6. Conclusion, Limitations, and Future Directions

6. 结论、局限性与未来发展方向

In this work, we present a preview version of DeepSeek-V4 series, aiming at next-generation large language models that break the efficiency barrier of ultra-long-context processing. By combining a hybrid attention architecture that integrates CSA and HCA, DeepSeek-V4 series achieve a dramatic leap in long-sequence efficiency. The architectural innovations, together with extensive infrastructure optimization, enable efficient native support for million-token contexts and establish a necessary foundation for future test-time scaling, long-horizon tasks, and emerging paradigms such as online learning. Evaluation results demonstrate that DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models. It substantially outperforms prior open-source models on knowledge benchmarks, achieves superior reasoning performance close to the frontier proprietary models, and delivers competitive agent capabilities. Meanwhile, DeepSeek-V4-Flash-Max attains comparable reasoning performance to leading closed models while maintaining a highly cost-efficient architecture. We believe DeepSeek-V4 series usher in a new era of million-length contexts for open models and pave the way toward better efficiency, scale, and intelligence.

在本工作中，我们展示了 DeepSeek-V4 系列的预览版本，旨在打造下一代大型语言模型，突破超长上下文处理的效率瓶颈。通过结合集成 CSA 和 HCA 的混合注意力架构，DeepSeek-V4 系列在长序列处理效率上实现了显著提升。架构创新与广泛的基础设施优化相结合，使得百万标记上下文的原生支持成为可能，并为未来推理时的扩展、长周期任务以及在线学习等新兴范式奠定了必要的基础。评估结果表明，DeepSeek-V4-Pro-Max（DeepSeek-V4-Pro 的最大推理努力模式）重新定义了开放模型的最先进水平。它在知识基准测试中显著优于之前的开源模型，推理性能接近前沿的专有模型，并提供了具有竞争力的代理能力。同时，DeepSeek-V4-Flash-Max 在保持高度成本效率架构的同时，实现了与领先闭源模型相当的推理性能。我们认为，DeepSeek-V4 系列开启了开放模型百万长度上下文的新时代，并为更好的效率、规模和智能铺平了道路。

In pursuit of extreme long-context efficiency, DeepSeek-V4 series adopted a bold architectural design. To minimize risk, we retained many preliminarily validated components and tricks, which,

while effective, made the architecture relatively complex. In future iterations, we will carry out more comprehensive and principled investigations to distill the architecture down to its most essential designs, making it more elegant without sacrificing performance. Meanwhile, although Anticipatory Routing and SwiGLU Clamping have been proven effective in mitigating training instabilities, their underlying principles remain insufficiently understood. We will actively study foundational problems on training stability and strengthen internal metric monitoring, aiming for a more principled and predictive approach to stable large-scale training.

在追求极端长上下文效率的过程中，DeepSeek-V4 系列采用了大胆的架构设计。为了降低风险，我们保留了许多初步验证有效的组件和技巧，这些虽然有效，但也使架构相对复杂。在未来的版本中，我们将进行更加全面和原则性的研究，提炼出架构中最基本的设计，使其更加优雅，同时不牺牲性能。同时，尽管前瞻性路由和 SwiGLU 钳制已被证明在缓解训练不稳定方面有效，但其底层原理仍不够清晰。我们将积极研究训练稳定性的基础问题，并加强内部指标监控，力求实现更加原则性和预测性的稳定大规模训练方法。

In addition, beyond the MoE and sparse attention architecture, we will also proactively explore model sparsity along new dimensions — such as more sparse embedding modules (Cheng et al., 2026) — to further improve computational and memory efficiency without compromising capability. We will also continuously investigate low-latency architectures and system techniques to make long-context deployment and interaction more responsive. Furthermore, we recognize the importance and practical value of long-horizon, multi-round agentic tasks, and will continue to iterate and explore in this direction. We are also working on incorporating multimodal capabilities to our models. Finally, we are committed to developing better data curation and synthesis strategies to consistently enhance model intelligence, robustness, and practical usability across an increasingly broad range of scenarios and tasks.

此外，在 MoE 和稀疏注意力架构之外，我们还将主动探索模型稀疏性在新的维度上的应用——例如更加稀疏的嵌入模块（Cheng 等，2026）——以进一步提高计算和内存效率，同时不损害模型的能力。我们还将持续研究低延迟架构和系统技术，使长上下文的部署和交互更加响应迅速。此外，我们认识到长时段、多轮次代理任务的重要性和实际价值，并将继续在这一方向上进行迭代和探索。我们还在努力将多模态能力整合到我们的模型中。最后，我们致力于开发更好的数据整理和合成策略，以持续提升模型在日益广泛的场景和任务中的智能性、鲁棒性和实用性。

References

参考资料

AA. Gdpval-aa leaderboard, 2025. URL https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa.

T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: Imo-level automated theorem proving. arXiv preprint arXiv:2510.01346, 2025.

A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI'20, page 419–434, USA, 2020. USENIX Association. ISBN 9781939133137.

O. J. Aimuyo, B. Oh, and R. Singh. Flashmoe: Fast distributed moe in a single kernel. Advances in Neural Information Processing Systems, 2025. URL https://neurips.cc/virtual/2025/poster/119124.

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

J. Asher. LeanExplore: A search engine for Lean 4 declarations, 2025. URL https://arxiv.org/abs/2506.11085.

Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P. Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S.

Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F. Wang, and H. Wang. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025a. URL https://doi.org/10.48550/arXiv.2507.20534.
Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664, 2025b.
M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.
C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.
F. Bellard. Qemu, a fast and portable dynamic translator. In Proceedings of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, page 41, USA, 2005. USENIX Association.
I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning, 2017. URL https://openreview.net/forum?id=rJY3vK9eg.
J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025. URL https://arxiv.org/abs/2512.17260.
M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.
T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association. ISBN 978-1-939133-08-3. URL https://www.usenix.org/conference/osdi18/presentation/chen.
A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas, et al. The facts leaderboard: A comprehensive benchmark for large language model factuality. arXiv preprint arXiv:2512.10791, 2025.
X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026. doi: 10.48550/ARXIV.2601.07372. URL https://doi.org/10.48550/arXiv.2601.07372.
K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.
D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066.
T. Dao, D. Haziza, F. Massa, and G. Sizov. Flash-decoding for long-context inference, 2023. URL https://pytorch.org/blog/flash-decoding/.
L. De Moura and N. Bjørner. Z3: an efficient smt solver. In Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 3540787992.
DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. CoRR, abs/2406.11931, 2024. URL https://doi.org/10.48550/arXiv.2406.11931.
DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL https://doi.org/10.48550/arXiv.2412.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL https://doi.org/10.48550/arXiv.2405.04434.
DeepSeek-AI. Fire-flyer file system, 2025. URL https://github.com/deepseek-ai/3FS.
DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nat., 645(8081):633–638, 2025. URL https://doi.org/10.1038/s41586-025-09422-z.
DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556.
X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL https://arxiv.org/abs/2509.16941.
H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. arXiv preprint arXiv:2404.10830, 2024.
X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P. Molchanov. Hymba: A hybrid-head architecture for small language models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=A1ztozypga.
X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. arXiv preprint arXiv:2502.14739, 2025.
D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246.
X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. Erofs: a compression-friendly readonly file system for resource-scarce devices. In Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '19, page 149–162, USA, 2019. USENIX Association. ISBN 9781939133038.
A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. Are we done with mmlu? CoRR, abs/2406.04127, 2024. URL https://doi.org/10.48550/arXiv.2406.04127.
F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=pEWAcejiU2.
Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In The Twelfth International Conference on Learning Representations, 2024.
L. Haas, G. Yona, G. D'Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. arXiv preprint arXiv:2509.07968, 2025.
Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. arXiv preprint arXiv:2411.07140, 2024.
D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.
D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.
Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.
D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL https://doi.org/10.48550/arXiv.2504.10356.
B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June

2018.
N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.
K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.
M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147.
H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 727–740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL https://www.usenix.org/conference/atc20/presentation/li-huiba.
H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.
J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.
Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=1NdN7eXyb4.
J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL https://doi.org/10.48550/arXiv.2502.16982.
I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.
K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi:10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.
Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. Corpusqa: A 10 million token benchmark for corpus-level analysis and reasoning. arXiv preprint arXiv:2601.14952, 2026.
T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL https://aclanthology.org/2025.emnlp-main.1794/.
M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.
MiniMax. Meet minimax-m2, 2025. URL https://github.com/MiniMax-AI/MiniMax-M2.
L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In International Conference on Automated Deduction, pages 625–635. Springer, 2021.
Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. Soviet Mathematics Doklady, 27:372–376, 1983.
NVIDIA Corporation. cublas documentation, 2024. URL https://docs.nvidia.com/cuda/cublas/. Version 12.4. Accessed: 2024-09-16.
OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024a. URL https://huggingface.co/datasets/openai/MMMLU.
OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2024b. URL https://huggingface.co/datasets/openai/mrcr.
OpenAI. Learning to reason with llms, 2024c. URL https://openai.com/index/learning-to-reason-with-llms.
OpenAI. Introducing SimpleQA, 2024d. URL https://openai.com/index/introducing-simpleqa/.
OpenAI. Introducing SWE-bench verified we're releasing a human-validated subset of swe-bench that more, 2024e. URL https://openai.com/index/introducing-swe-bench-verified/.
OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/A

RXIV.2508.10925. URL https://doi.org/10.48550/arXiv.2508.10925.
M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pages 429–431, 2023.
T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubeh, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374, 2025.
L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity's last exam. arXiv preprint arXiv:2501.14249, 2025.
W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. In T. Cohn, Y. He, and Y. Liu, editors, Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, volume EMNLP 2020 of Findings of ACL, pages 2401–2410. Association for Computational Linguistics, 2020. URL https://doi.org/10.18653/v1/2020.findings-emnlp.217.
Qwen. Qwen3 technical report. CoRR, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL https://doi.org/10.48550/arXiv.2505.09388.
S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.
J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2022. URL https://arxiv.org/abs/2112.08429.
D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.
G. T. M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram'e, J. Ferret, P. Liu, P. D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Stańczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson,
B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozi'nska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshev, F. Visin, G. Rasskin, G. Wei, G. Cameron, G. Martins, H. Hashemi, H. Klimczak-Pluci'nska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui, L. Sifre, L. Heuermann, L. cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gorner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: Improving open language models at a practical size.

AA. Gdpval-aa leaderboard, 2025. URL https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa.
T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: Imo-level automated theorem proving. arXiv preprint arXiv:2510.01346, 2025.
A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI'20, page

419–434, USA, 2020. USENIX Association. ISBN 9781939133137.
O. J. Aimuyo, B. Oh, and R. Singh. Flashmoe: Fast distributed moe in a single kernel. Advances in Neural Information Processing Systems, 2025. URL https://neurips.cc/virtual/2025/poster/119124.
J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.
J. Asher. LeanExplore: A search engine for Lean 4 declarations, 2025. URL https://arxiv.org/abs/2506.11085.
Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P. Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F. Wang, and H. Wang. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025a. URL https://doi.org/10.48550/arXiv.2507.20534.
Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664, 2025b.
M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.
C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.
F. Bellard. Qemu, a fast and portable dynamic translator. In Proceedings of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, page 41, USA, 2005. USENIX Association.
I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning, 2017. URL https://openreview.net/forum?id=rJY3vK9eg.
J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025. URL https://arxiv.org/abs/2512.17260.
M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.
T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association. ISBN 978-1-939133-08-3. URL https://www.usenix.org/conference/osdi18/presentation/chen.
A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas, et al. The facts leaderboard: A comprehensive benchmark for large language model factuality. arXiv preprint arXiv:2512.10791, 2025.
X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026. doi: 10.48550/ARXIV.2601.07372. URL https://doi.org/10.48550/arXiv.2601.07372.
K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066.
T. Dao, D. Haziza, F. Massa, and G. Sizov. Flash-decoding for long-context inference, 2023. URL https://pytorch.org/blog/flash-decoding/.
L. De Moura and N. Bjørner. Z3: an efficient smt solver. In Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 3540787992.
DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. CoRR, abs/2406.11931, 2024. URL https://doi.org/10.48550/arXiv.2406.11931.
DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL https://doi.org/10.48550/arXiv.2412.
DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL https://doi.org/10.48550/arXiv.2405.04434.
DeepSeek-AI. Fire-flyer file system, 2025. URL https://github.com/deepseek-ai/3FS.
DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nat., 645(8081):633–638, 2025. URL https://doi.org/10.1038/s41586-025-09422-z.
DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556.
X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL https://arxiv.org/abs/2509.16941.
H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. arXiv preprint arXiv:2404.10830, 2024.
X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P. Molchanov. Hymba: A hybrid-head architecture for small language models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=A1ztozypga.
X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. arXiv preprint arXiv:2502.14739, 2025.
D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246.
X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. Erofs: a compression-friendly readonly file system for resource-scarce devices. In Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '19, page 149–162, USA, 2019. USENIX Association. ISBN 9781939133038.
A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. Are we done with mmlu? CoRR, abs/2406.04127, 2024. URL https://doi.org/10.48550/arXiv.2406.04127.
F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=pEWAcejiU2.
Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In The Twelfth International Conference on Learning Representations, 2024.
L. Haas, G. Yona, G. D'Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. arXiv preprint arXiv:2509.07968, 2025.
Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. arXiv preprint arXiv:2411.07140, 2024.
D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring

massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.
D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.
Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.
D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL https://doi.org/10.48550/arXiv.2504.10356.
B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.
N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.
K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.
M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147.
H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 727–740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL https://www.usenix.org/conference/atc20/presentation/li-huiba.
H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.
J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.
Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=1NdN7eXyb4.
J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL https://doi.org/10.48550/arXiv.2502.16982.
I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.
K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi:10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.
Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. Corpusqa: A 10 million token benchmark for corpus-level analysis and reasoning. arXiv preprint arXiv:2601.14952, 2026.
T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL https://aclanthology.org/2025.emnlp-main.1794/.
M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.
MiniMax. Meet minimax-m2, 2025. URL https://github.com/MiniMax-AI/MiniMax-M2.
L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In International Conference on Automated Deduction, pages 625–635. Springer, 2021.
Y. Nesterov. A method of solving a convex programming problem with convergence rate

$O(1/k^2)$. Soviet Mathematics Doklady, 27:372–376, 1983.
NVIDIA Corporation. cublas documentation, 2024. URL https://docs.nvidia.com/cuda/cublas/. Version 12.4. Accessed: 2024-09-16.
OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024a. URL https://huggingface.co/datasets/openai/MMMLU.
OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2024b. URL https://huggingface.co/datasets/openai/mrcr.
OpenAI. Learning to reason with llms, 2024c. URL https://openai.com/index/learning-to-reason-with-llms.
OpenAI. Introducing SimpleQA, 2024d. URL https://openai.com/index/introducing-simpleqa/.
OpenAI. Introducing SWE-bench verified we're releasing a human-validated subset of swe-bench that more, 2024e. URL https://openai.com/index/introducing-swe-bench-verified/.
OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/ARXIV.2508.10925. URL https://doi.org/10.48550/arXiv.2508.10925.
M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pages 429–431, 2023.
T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubeh, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374, 2025.
L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity's last exam. arXiv preprint arXiv:2501.14249, 2025.
W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. In T. Cohn, Y. He, and Y. Liu, editors, Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, volume EMNLP 2020 of Findings of ACL, pages 2401–2410. Association for Computational Linguistics, 2020. URL https://doi.org/10.18653/v1/2020.findings-emnlp.217.
Qwen. Qwen3 technical report. CoRR, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL https://doi.org/10.48550/arXiv.2505.09388.
S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.
J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2022. URL https://arxiv.org/abs/2112.08429.
D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.
G. T. M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram'e, J. Ferret, P. Liu, P. D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Stańczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson,
B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozi'nska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshev, F. Visin, G. Rasskin, G. Wei, G. Cameron, G. Martins, H. Hashemi, H. Klimczak-Pluci'nska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui, L. Sifre, L. Heuermann, L. cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gorner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang,

L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: Improving open language models at a practical size.

arXiv preprint arXiv:2408.00118, 2024.
S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 17555–17566, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076-Abstract.html.
B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.
K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.
Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025. URL https://arxiv.org/abs/2511.22570.
N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL http://arxiv.org/abs/1911.02150.
N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.
F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=fR3wGCk-IXp.
J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.
M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.
G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL https://arxiv.org/abs/2407.11214.
A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL https://doi.org/10.48550/arXiv.2408.15664.
L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. Tilelang: Bridge programmability and performance in modern neural kernels. In The Fourteenth International Conference on Learning Representations, 2026.
Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL https://doi.org/10.48550/arXiv.2406.01574.
J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.
T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.
G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.
Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifold-constrained hyper-connections, 2026. URL https://arxiv.org/abs/2512.24880.

L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419.
J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL https://arxiv.org/abs/2504.21798.
R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Márquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p19-1472.
C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, page 446–461, New York, NY, USA, 2025a. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764823. URL https://doi.org/10.1145/3731569.3764823.
S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. 2025b. URL https://arxiv.org/abs/2502.19811.
C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. https://github.com/deepseek-ai/DeepGEMM, 2025.
W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364.
D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyperconnections. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL https://openreview.net/forum?id=9FqARW7dwB.
X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? arXiv preprint arXiv:2412.14689, 2024.
T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL https://openreview.net/forum?id=YrycTjllL0.

arXiv preprint arXiv:2408.00118, 2024.
S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 17555–17566, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076-Abstract.html.
B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.
K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.
Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025. URL https://arxiv.org/abs/2511.22570.
N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150,

2019. URL http://arxiv.org/abs/1911.02150.
N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.
F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=fR3wGCk-IXp.
J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.
M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.
G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL https://arxiv.org/abs/2407.11214.
A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL https://doi.org/10.48550/arXiv.2408.15664.
L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. Tilelang: Bridge programmability and performance in modern neural kernels. In The Fourteenth International Conference on Learning Representations, 2026.
Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL https://doi.org/10.48550/arXiv.2406.01574.
J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.
T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.
G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.
Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifold-constrained hyper-connections, 2026. URL https://arxiv.org/abs/2512.24880.
L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419.
J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL https://arxiv.org/abs/2504.21798.
R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Márquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p19-1472.
C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, page 446–461, New York, NY, USA, 2025a. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764823. URL https://doi.org/10.1145/3731569.3764823.
S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q.

Chen, and X. Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. 2025b. URL https://arxiv.org/abs/2502.19811.
C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. https://github.com/deepseek-ai/DeepGEMM, 2025.
W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364.
D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyperconnections. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL https://openreview.net/forum?id=9FqARW7dwB.
X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? arXiv preprint arXiv:2412.14689, 2024.
T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL https://openreview.net/forum?id=YrycTjllL0.

Appendix

附录

A. Author List and Acknowledgment

A. 作者列表和致谢

A.1. Author List

A.1. 作者列表

Authors are listed alphabetically by their first name. Names marked with * denote individuals who have departed from our team.

作者按姓氏的字母顺序排列。标有 * 的名字表示这些人员已离开我们的团队。

Research & Engineering: Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang*, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chenze Shao, Chong Ruan*, Conner Sun, Damai Dai, Daya Guo*, Dejian Yang, Deli Chen, Donghao Li, Erhang Li, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei*, Haotian Yuan, Haowei Zhang*, Haowen Luo, Haoyu Chen, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J. Yang, J.Q. Zhu, Jia Yu, Jialiang Huang, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran*, Junguang Jiang, Junjie Qiu, Junlong Li*, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang*, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, M.S. Di, M.Y Xu, Max Mei, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S.H. Liu, Shanghaio Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian

Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W. Zhang, Wangding Zeng, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xuheng Lin, Xuwei Fu, Y.C. Yan, Y.Q. Wang*, Y.W. Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Ying He, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuhan Wu, Yuhao Meng, Yuheng Zou, YuKun Li, Yunfan Xiong, Yupeng Chen, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, Z.F. Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang*, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao.

研究与工程：徐 _anyi_，林 _bangcai_，薛 _bing_，王 _bingxuan_, 徐 _*bingzheng*_，吴 _*bochao*_，张 _*bowei*_，林 _*chaofan*_，董 _*chen*_，卢 _*chengda*_，赵 _*chenggang*_，邓 _*chengqi*_，徐 _*chenhao*_，邵 _*chenze*_，阮 _*chong*_, 康纳 _sun_，戴 _damai_，郭 _daya_, 杨 _*dejian*_，陈 _*deli*_，李 _*donghao*_，李 _*erhang*_，林 _*fangyun*_，袁 _*fangzhou*_，夏 _*feiyu*_，戴 _*fucong*_，郝 _*guangbo*_，陈 _*guanting*_，曹 _*guoai*_，孟 _*guolai*_，李 _*guowei*_，于 _*han*_，张 _*han*_，徐 _*hanwei*_，李 _*hao*_，梁 _*haofen*_，张 _*haoling*_，罗 _*haoming*_，魏 _*haoran*_, 袁 _haotian_，张 _haowei_, 罗 _*haowen*_，陈 _*haoyu*_，纪 _*haozhe*_，丁 _*honghui*_，唐 _*hongxuan*_，曹 _*huanqi*_，高 _*huazuo*_，屈 _*hui*_，曾 _*hui*_，杨 _*j.*_，朱 _*j.q.*_，余 _*jia*_，黄 _*jialiang*_，叶 _*jiasheng*_，李 _*jiashi*_，徐 _*jiaxin*_，胡 _*jiewen*_，严 _*jin*_，陈 _*jingchang*_，周 _*jingli*_，相 _*jingting*_，袁 _*jingyang*_，程 _*jingyuan*_，朱 _*jinhua*_，余 _*jiping*_，孙 _*joseph*_，孙 _*jun*_, 蒋 _junguang_，邱 _junjie_，李 _junlong_, 宋 _*junxiao*_，董 _*kai*_，高 _*kaige*_，关 _*kang*_，周 _*kexing*_，黄 _*kezhao*_, 于 _kuai_，王 _lean_，张 _lecong_，王 _lei_，张 _li_，赵 _liang_，郭 _lihua_，罗 _lingxiao_，马 _linwang_，王 _litong_，蔡 _liyu_，张 _liyue_，陈 _longhao_，迪 _m.s._，徐 _m.y._，梅 _max_，张 _mingchuan_，张 _minghua_，唐 _minghui_，周 _mingxu_，黄 _panpan_，丛 _peixin_，王 _peiyi_，王 _qiancheng_，朱 _qihao_，李 _qingyang_，陈 _qinyu_，杜 _qiushi_，江 _qiwei_，田 _rui_，徐 _ruifan_，卢 _ruijie_，徐 _ruilin_，葛 _ruiqi_，张 _ruisong_，潘 _ruizhe_，王 _runji_，陈 _runqian_，尹 _runqiu_，徐 _runxin_，沈 _ruomeng_，张 _ruoyu_，刘 _s.h._，卢 _shanghaio_，周 _shangyan_，陈 _shanhuang_，蔡 _shaofei_，聂 _shaoheng_，陈 _shaoyuan_，胡 _shengding_, 刘 _shengyu_, 胡 _shiqiang_, 马 _shirong_, 王 _shiyu_, 于 _shuiping_, 周 _shunfeng_，潘 _shuting_，于 _shuying_，周 _songyang_，倪 _tao_，云 _tao_，金 _tian_，裴 _tian_，叶 _tian_，林 _tianle_, 纪 _tianran_，崔 _tianyi_，岳 _tianyuan_，于 _tingting_，王 _tun_，张 _w._，曾 _wangding_，赵 _weilin_，刘 _wen_，梁 _wenfeng_，庞 _wenjie_，罗 _wenjing_，姚 _wenjing_，高 _wenjun_，杨 _wenkai_，黄 _wenlve_，张 _wentao_，马 _wenting_，高 _xi_，何 _xiang_，王 _xiangwen_，毕 _xiaobi_，刘 _xiaodong_，王 _xiaohan_，陈 _xiaokang_，张 _xiaokang_，Nie_xiaotao_，程 _xin_，刘 _xin_，谢 _xin_，刘 _xingchao_，刘 _xingchen_，刘 _xingkai_，于 _xingyou_，杨 _xinyu_，陈 _xu_，王 _xuanyu_，苏 _xuecheng_，林 _xuheng_，傅 _xuwei_，严 _y.c._，王 _y.q._，马 _*y.w.*_，罗 _*yanfeng*_，张 _*yang*_，徐 _*yanhong*_，马 _*yanru*_，黄 _*yanwen*_，刘 _*yao*_，李 _*yao*_，赵 _*yao*_，孙 _*yaofeng*_，王 _*yaohui*_，钱 _*yi*_，于 _*yi*_，张 _*yichao*_，丁 _*yifan*_，施 _*yifan*_，吴 _*yijia*_，熊 _*yiliang*_，何 _*ying*_，周 _*ying*_，罗 _*yingjia*_，钟 _*yinmin*_，飘 _*yishi*_，王 _*yisong*_，张 _*yixiang*_，陈 _*yixiao*_，谭 _*yixuan*_，魏 _*yixuan*_，马 _*yiyang*_，刘 _*yiyuan*_，杨 _*yonglun*_，郭 _*yongqiang*_，吴 _*yongtong*_，吴 _*yu*_，程 _*yuan*_，欧 _*yuan*_，徐 _*yuanfan*_，李 _*yuanhao*_，王 _*yuduan*_，吴 _*yuhan*_，孟 _*yuhao*_，邹 _*yuheng*_，李 _*yukun*_，熊 _*yunfan*_，陈 _*yupeng*_，曹 _*yuqian*_，王 _*yuqian*_，张 _*yushun*_，林 _*yutong*_，顾 _*yuxian*_，罗 _*yuxiang*_，游 _*yuxiang*_，刘 _*yuxuan*_，周 _*yuxuan*_，周 _*yuyang*_，黄 _*yuzhen*_，吴 _*z.f.*_，王 _*zehao*_，赵 _*zehua*_，任 _*zehui*_，沙 _*zhangli*_，傅 _*zhe*_，徐 _*zhean*_，谢 _*zhenda*_，张 _*zhengyan*_，郝 _*zhewen*_，*gou_zhibin*_，马 _*zhicheng*_，闫 _*zhigang*_，邵 _*zhihong*_，黄 _*zhixian*_，陈 _*zhixuan*_，吴 _*zhizhou*_，任 _*zhizhou*_，李 _*zhuoshu*_，张 _*zhuping*_，徐 _*zian*_，王 _*zihao*_，顾 _*zihui*_，朱 _*zijia*_，李 _*zilin*_，张 _*zipeng*_, 谢 _ziwei_，高 _ziyi_，潘 _zizheng_，姚 _zongqing_。

Business & Compliance: Chenchen Ling, Chengyu Hou, Dongjie Ji, Fang Wei, Hengqing Zhang,

Jia Luo, Jia Song, Jialu Cai, Jian Liang, Jiangting Zhou, Jieyu Yang, Jin Chen, Jingzi Zhou, Junmin Zheng, Leyi Xia, Linyan Zhu, Miaojun Wang, Mingming Li, Minmin Han, Ning Wang, Panpan

业务与合规：凌晨晨、侯程宇、姬冬杰、魏芳、张恒清、罗佳、宋佳、蔡嘉露、梁健、周江亭、杨杰宇、陈瑾、周静子、郑俊敏、夏乐毅、朱琳燕、王妙君、李明明、韩敏敏、王宁、潘潘

Wang, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, W.L. Xiao, Wei An, Wenqing Hou, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Xinyu Zhang, Xueyin Chen, Yao Xu, Yi Shao, Yiling Ma, Ying Tang, Yuehan Yang, Yuer Xu, Yukun Zha, Yuping Lin, Yuting Yan, Zekai Zhang, Zhe Ju, Zheren Gao, Zhongyu Wu, Zihua Qu, Ziyi Wan.

王, 彭张, 瑞毅陈, 上面孙, 少庆吴, W.L. 肖, 韦安, 文青侯, 王先祖, 孙晓文, 王晓翔, 张新宇, 陈雪银, 许耀, 邵毅, 马毅玲, 唐英, 杨跃涵, 许跃, 周玉坤, 林玉平, 严雨婷, 张泽凯, 朱哲, 高哲仁, 吴仲宇, 邱子华, 万子毅.

A.2. Acknowledgment

A.2. 致谢

We would like to thank Dolly Deng and other testers for their valuable suggestions and feedback regarding the capabilities of DeepSeek-V4 series models.

我们感谢 Dolly Deng 和其他测试人员对 DeepSeek-V4 系列模型能力提出的宝贵建议和反馈。

B. Evaluation Details

B. 评估细节

Table 9 | Agentic Search vs. Retrieval Augmented Search for DeepSeek-V4-Pro.

表 9 | DeepSeek-V4-Pro 的自主搜索与检索增强搜索对比。

Difficulty	Category	#	Agent Win	RAG Win	Tie	Agent%	RAG%
Easy	Objective Q&A (客观问答)	196	110	43	43	56.1	21.9
Subjective Q&A (主观问答)	321	198	56	67	61.7	17.4	20.9
Hard	Objective Q&A (客观问答)	168	102	33	33	60.7	19.6
Subjective Q&A (主观问答)	184	126	27	31	68.5	14.7	16.8
	Total (总计)	869	536	159	174	61.7	18.3

难度	类别	#	代理 Win	RAG Win	系	代理%	RAG%	系
简单	客观问答 (Objective Q&A)	196	110	43	43	56.1	21.9	2
主观问答 (Subjective Q&A)	321	198	56	67	61.7	17.4	20.9	
硬	客观问答（Objective Q&A）	168	102	33	33	60.7	19.6	1
主观问答 (Subjective Q&A)	184	126	27	31	68.5	14.7	16.8	
	总计	869	536	159	174	61.7	18.3	2

Table 10 | Cost Comparison: Agentic Search vs. Retrieval Augmented Search (Mean) for DeepSeek-V4-Pro. Most of the tool calls are parallel for Agentic Search.

表 10 | 成本对比：自主搜索 vs. 检索增强搜索（均值）对于 DeepSeek-V4-Pro。大多数工具调用在自主搜索中是并行的。

Version	Tool Calls	Prefill (tokens)	Output (tokens)
V4 Agentic Search	16.2	13649	1526
V4 Retrieval Augmented Search	—	10453	1308

版本	工具调用	预填充（标记）	输出（标记）
V4 智能代理搜索	16.2	13649	1526
V4 检索增强搜索	—	10453	1308

Table 11 | Comparative Evaluation of DeepSeek-V4-Pro and DeepSeek-V3.2 on Search Q&A Tasks.

表 11 | DeepSeek-V4-Pro 与 DeepSeek-V3.2 在搜索问答任务中的对比评估。

Category	Subcategory	#	Internal Evaluation (内部综合
V4 win	V3.2 win	tie	V4%
Objective Q&A (客观问答)	Single-value Search (单值信息查找)	95	36
Entity Search (实体信息查找)	99	24	7
Enumerative Search (枚举型信息查找)	95	19	8
Subtotal (小计)	289	79	25
Subjective Q&A (主观问答)	Causal Analysis (原因分析)	100	28
Comparison (对比)	96	28	20
Advice Seeking (寻求建议)	92	23	8
Recommendation (推荐)	95	26	19
Planning & Strategy (攻略计划)	92	32	11
Opinion & Evaluation (评价看法)	96	30	8
Trend Analysis (趋势分析)	96	23	3
Subtotal (小计)	667	190	74
	TOTAL (总计)	956	269

类别	子类别	#	内部综合评估（Internal Evalu
V4 胜利	V3.2 win	系	V4%
客观问答 (Objective Q&A)	单值信息查找 (Single-value Search)	95	36
实体信息查找	99	24	7
枚举型信息查找 (Enumerative Search)	95	19	8
小计	289	79	25
主观问答 (Subjective Q&A)	因果分析（原因分析）	100	28
对比 (Comparison)	96	28	20
寻求建议	92	23	8
推荐	95	26	19

规划 & 策略 (攻略计划)	92	32	11
意见与评价（评价看法）	96	30	8
趋势分析 (趋势分析)	96	23	3
小计	667	190	74
	总计	956	269

Figure 14: images/de85f32c3190cb8f8affdb57840c9e706765e8192fa1752518588b62ed2c1fbe.jpg

策略 A（特定投）vs 策略 B（定投 + 滚动卖出）

策略 A（特定投）vs 策略 B（定投 + 滚动卖出）

四到期间：2025 年 6 月 2 日—2025 年 12 月 31 日（148 个交易日）每日定投金额：500 元人民币 | 数据来源：Yahoo Finance 报告生效日期：2026 年 4 月 16 日

四到期间：2025 年 6 月 2 日—2025 年 12 月 31 日（148 个交易日）每日定投金额：500 元人民币 | 数据来源：Yahoo Finance 报告生效日期：2026 年 4 月 16 日

来源市场数据一览	
判断表无相关收盘价 (02/04-02)	19,743.61
判断表无期末收盘价 (02/05-12/31)	23,241.99
判断表无期间涨跌幅	+20.70%
期初最高价 (02/05-10/20)	23,958.47
期初最低价 (02/06-02)	19,743.61
LED 出纳 (微红汇率)	7,2037
LED 出纳 (PEE 汇率)	6.9961
人均消费水平涨幅幅度	+2.88%
需求水位 (日均米)	4.9/4.81、19/4.8、49/5.12、11/5/13、12/5/23

市场数据概览

判断表无相关收盘价 (02/04-02)	19,743.61
判断表无期末收盘价 (02/05-12/31)	23,241.99
判断表无期间涨跌幅	+20.70%
期初最高价 (02/05-10/20)	23,958.47
期初最低价 (02/06-02)	19,743.61
LED 出纳 (微红汇率)	7,2037
LED 出纳 (PEE 汇率)	6.9961
人均消费水平涨幅幅度	+2.88%
需求水位 (日均米)	4.9/4.81、19/4.8、49/5.12、11/5/13、12/5/23

Figure 14 | Example output of a task that requires comparing two regular investment strategies for the NASDAQ.

图 14 | 一个需要比较两个常规投资策略的任务示例输出，适用于纳斯达克市场。

Figure 15: images/4d5f4a6858d49b05e3c32f64bb6d81eb13c987ae18a9d2ce47ca18dd28dda5e2.jpg

Figure 15 | Example output of a task which requires researching 2020-2025 Nobel Science Prizes

and generating an analytical PDF report.

图 15 | 需要研究 2020-2025 年诺贝尔科学奖并生成分析型 PDF 报告的任务示例输出。

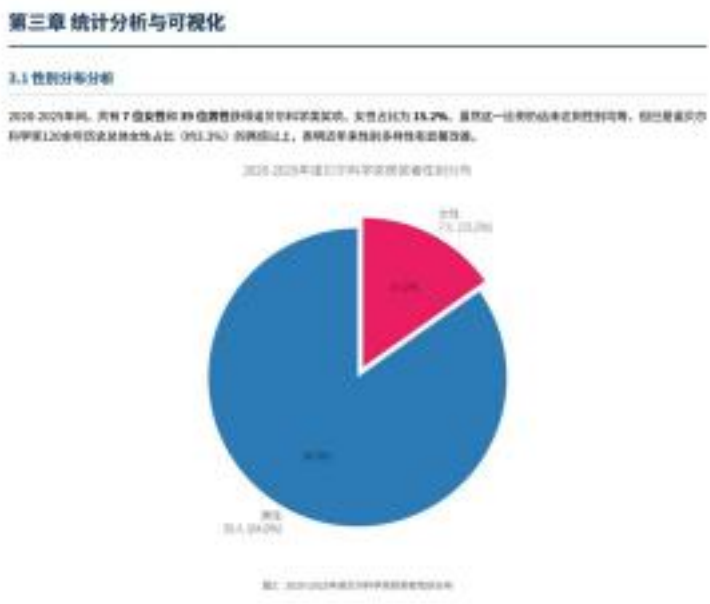

Figure 16: images/bfeec71d27b1cf90b79ea865b83b5b4113a2ada3069153dab285b5b879974e11.jpg

1. 安善西缘・盖兹《2020 年特理学奖》——历史上第四位获得诺贝尔特理学奖的女性。国家密码河系中心超大质量类别同质奖。

2. 塞马超埃尔・沙尔维捷（2020 年化学实）——与杜普纳共同开发 CNSPR/Cer 的基因谱幅技术。

3. 南雅典・杜德纳（2020 年化学奖）——与沙尔提康共同获得，2018 年化学奖退还贝尔科学奖其史上首次由金女当团队包授一个奖项。

4. 安夏・卢韩华（2023 年物理学奖）——历史上第五位获得物理学奖的女性，发现高次遗传产生机制，为阿秒特理鼻基。

5. 考量科-卡瑞琳（2023 年生理学或医学奖）——发现 eRNA 按普修饰技术，使 eRNA 疫苗认科学幻想变为现实。她的职业经历（普特职
被前期、经费基础 > 含量或成为科学性的备注。

6. 同意 + 根据科（2023 年生理学调查学报）——发现治疗的基因在操作性? 根据发育中枢核心调控作用。
魏爱彬：含食尿液者在 2021 年集中填报（3 人）。依公帝前不均指——2021 年初和 2024 年均见含食科学家得生。从学科讨论看，化学炭生性最多（4 人/2），地理学（4 人/2）和生理学调查员（2 人/8）相对较少。2021 年化学炭的女性性奖金适合委员会 11.5 年时上的谢兵。据因为含食早期的一个重要象征性质疑。然而，霍尔夫家族的特殊依赖属于资产法的学术界的西南药典——包括研究及资源分析可行，资金提升中的 “课题管道” 而远，以及更高和申请过程中的无意见偏见。解决以前结构性风险需要从 5 月 16 日到 9 月 18 日收到学术部函的书面核查结果为：
3.2 年龄分布分析
2020-2025 年诺贝尔科学奖获奖者的平均获奖年龄为 69.4 岁，中位年龄为 68 岁，最年轻的获奖者是向翰・江迪（2024 年化学奖，获奖时仅
39（F）：他因于是 Alphafold2 的核心深度学习架构而获此繁荣，成为 1950 年以来最年轻的老贝尔

科学奖获得者之一；副年长的获奖者是检验-霍剑聂如晴（以 194 年物理学奖、吴军时 81（F）），他的霍剑聂加晴同志发表于 1982 年，经历了 40 余年的证券和台湾才获得荣誉认可。

第五章总结与未来展望

2.1 公共 4 项

(四）本次募集资金使用情况

通过对 2025-2025 年诺贝尔科学类奖项（共 48 位获奖者，覆盖 13 个国家，涉及地理学、化学、生理学或医学三个奖项类别）的积极性测算分析应得到研究。本报告将经过以下关键核心体现：

1

一、美国新材料研磨领先地位，全球多条化格局是演成型。美国 IC24 人（占 1.2%）在实质性研发中，满足投资者需求

约有 12 个地区前 21 个，欧洲科研联盟（美、德、法、恩、美、丹、瑞典、哈里科、俄罗斯）和度末地区（日本、意大利）的所得税协定税率，以及哈里州图中地根据国家的所得税表现，并须留住了一幅全国科研实力等形式平等的要素。鉴于注意的是，中国和韩国等研发投入方面也对时限未在蛋白质化学家上安排突破——考虑到诺贝尔大学的 “时间距位” 效应（获奖工作通常在 20-30 年代完成），正是可能是传统的工业原料积累的科研实力。

三、“计算 × 科学” 它确立为数学元学科模式。

2024 年物理化学和化学研究时学术大修研究的通过一线性的措施，使其根据文章分为有标题——从 2021 年气候模拟机 2021 年利用数据分析，而 2023 年的 MOF 计划设计。计算方法《网络机器学习和人工智能》包括从科目的 “辅助工具” 演变为驱动科学发展的 “共同创造者”，这一根本性利差正加模糊诺贝尔五个都站提到《物理学、化学、生理学领域、人工智能》之间的再构原理，并体现了关于奖项结构是否需要市场化？例如，虽然应该设立数字公共物理科学系统，越能否改变公共检验研究后各个基因之间需要连续划分——的严重学术和必约论。

;

21、从基础发照时应由影响的快速披露是靠拢。mNA 信息技术 2015 年参考目标和关键研发项目软件发明其 2030 年全部前十亿人版件仅用了 15 年——注册研发科学家史上从基础研究发明者全球 + 规模健康干预的最终转化之一。相比之下，2011 年物理学家麦普热气液模型是提升（液晶印刷至 2060 年代的成）研制了 30 余体的预测描述；HCV 在 2008 年发表到 2013 年生物活性物（原参考书）上获得了 4 年。霍普金为德国科达 12 年发表发明的 2034 年需要获得了 4 年，欧洲与 “整个年度” 四个整车单价的 “包括胶粉”，不仅反映了技术升级（计算能力、基准训练、高速雷达）的勘探进步，也体现了诺贝尔奖委会会在竖有 “时间经验” 取得的同时，河南有 “何时定量 + 社会影响” 的研究决定了富奥新指的关系。

二、经与本公司签署的关联交易事项的说明

2036-2037 年共 17 亿女性婴童（5.2%），道恩贝科学奖 130 余年历史总体女性占比（80.3%）的排名。2020 年是一个具有泰信意义的实施之一——化学家康奈曲金安堂培养包络（沙尔迪健和玛德科），物理学家迪逝了所有史上病龄位女性医生（盖班）。然而，控制清晰的平均推荐资料中人都能在——2021 年和 2024 年科学家老发现者均为内卷，形成了成人厚龄的一年一年、六一年 “模式”。据实际的结构因素——从本科学 TM 作业的组织比例逐渐，到博士后应用的 “多因素”，帮助您了解 4 项相关知识和内容与身体联系关系，需要定期评估或指导其理解，希望关注自身在医疗领域持续发展。

你/赵建强/郑文英

五、鉴于科技完成了从道路理论的工程系统的完整和事情。

2022 年物理学家（量子问题验证与量子信息科学）□2021 年化学学中的量子点（量子极限效应的化学工程定）□2021 年物理学家（宏观量子电能与导航量子计算）构成了一部显像的 “度因子量子三态融合”。这个实质完全包括 “第三次量子革命” 到三个核心指标：基础假设（证明量子元素顺序存在且可被删除），材料参数（控制系统中工程实测于项目）和微观集成（构建可扩展的量子信息经验模型）。当某一回向量子计算结果在某一具有实际意义的问题上能够准确地形成经典计算的能力时，根本的工程和科学贡献其几乎都会获得显著认可的信号。

六、“理性安排” 不违反相关 “规范者需” 成为激励对象的适合情形

以 CROPA 酯基硼酸（2020 年化学法）和科学特有制毒化（2021 年化学法），A 点血化学和微生物交注化学（2021 年化学法）同整个子的种子工程（2023 年化学法），从计算基础设计（2024 年化学法）到 PCR 技术的分子生物学（2025 年化学法）——该大学的临床结合化学（以及与基因表达过的细胞医学知识及肿瘤医学研究）共同描述了一个 “人类神经学在分子和原子下进行土壤植肥的重要效果。化学生命科学性正反应与肠道移植”（含生物育什么分子？它们如何工作）。刺激酶提取了推广蛋白质 + 功能上设计的构建具有高效的功能，对生物形态破坏” 的作用逐渐变化。这一新药的研究中还基于其技术意义——分析出人类与物质世界关系的一个根本性变化：我们不再划分适合世界的骨骼和解毒器，而蛋白是在分子生性的植物周转过程。

5.2 未来展望：2026 年及以后的潜在获奖方向

基于对 2022-2025 年趋势的深入分析和对当前全球科研所处的持续观察，我们采用该以下七个最有

可能在未来 5-18 年内获得诺贝尔科学奖认可
的研究方向和具体贡献

7. AI 驱动的科学原理（AI-for-Sciemcn）将持续受到关注。AlphaFold3 及后续系统在预测所载分子相互作用（蛋白质-蛋白质、蛋白质-
DNA-蛋白质/含子/方案的关键。全球 A 区在全多功能蛋白质和有机盐中的应用，以及 A 区数字算法启动说明和标准媒体启动及程序的里程碑式成就。都是潜在的获奖方向。一个深刻的制度性问题是：当 A 系统级史诺贝尔奖裁判的科学发现时，奖项应论何归属？这一问
题目目前尚无异议，但其紧迫性正在增加。

8. 每子计算的 “承手锁息率” 指量子树脂的实际测试，一旦需要量子计算机应的给分子得到，简化反应机算计算。金融风险模型或新材料设计等某一具有实际科学或经济价值的问题上需由经典计算机无法及时的能力，且结果经过独立验证，相关贡献者将成为诺贝尔奖的有力竞争者。定期咨询 Wiseo 芯片和 SiM 量子系统的质量假设—里程碑的实现时间表不断提成。

9. 气被量化科学制查率性绿色技术。极端天气条件的气被量化控制方法论：气通临界点（如大西洋经向翻转环类 MADC 的稳定性）的物理机制说明，以及钙钛矿；接层太阳能电池的效率和稳定性保健(目前已接近商业化门槛),都是兼具基础科学深度和全球社会影响力的真实核查方向。

10. 神经科学与融合计划的突破性产品。先进传字（光电控制固定神经光的活动）的基础发明者长期被认为是或实热门人选。全链连接倍字（如果体、小鼠或人类大脑的完整脉络图谱），以及意识神经基础的实验性限度的突破性假设——随着单细胞倍字，高特强功能材料和风语随机接口技术的融合——正在使这些骨上过于因噪” 的问题变得可以实现检验。

11. CRISPR 2.0——下一代基因编辑技术的临床转化。David Liu 因从开发的编辑词（base editing，2016 年）和先导谱辑（prime editing，2013 年）实现了电子切割 DNA 双层的首次下辖修改命令被分级小数为 64k，大幅提高了编辑难度和安全性。该技术的临床成功——究竟是如果能够在核力细胞、遗传性失明或递进性高胆固醇血症等疾病的急性性治疗中取得确诊疗效.....其转和可能获得或只分类的认可（请可能生理学或医学奖。

12. 袁老生物学与长寿科学的机制障碍和干预验证。分子衰老标志的系统性发现（如基于 DNA 甲基变化的表观遗传可转）、衰老细胞清除筛筒（oreolytic）在人体临床试验中的疗效信息，以及通过 Pamanaoka 因子部分重组实验实现的表观遗传 rejuvenation（细菌 “诺老还重”）——如果该算法干预措施在严格设计的临床试验中体现出最佳或逆转人类衰老相关功能食谱的确知证据——将代表人类生物学的一个分支结构，其结论与其他因素不互响。

13. 微生物链与强生相互作用的基因靶突细胞。人体微生物来自代谢性疾病（肥胖、2 型糖尿病）、免疫介学病病（血虚性肠炎、流感风 mutants 炎）和神经障碍疾病（自定义语系、抑郁症、癌症森病）中的细菌性（而非仅相关性）作用的严格实验结论，以及基于理性设计的微生物线干预（细胞性生殖联合体、靶内嗜固体疗法、工培化共生细胞）的临床成功，构成了一个具有广阔前景和深法医学意义的脑获联领域
答案: 好奇心驱动的基础研究——应对未知未来的最可靠预期
2020-2023 年的诺贝尔科学奖颁发了一幅令人厚朴的科学联盟——人类应用时陆生多个科技基地的交汇点上。人工智能应用重新定义 “重视” 的定义，基于世界之友史百年性的理论物理转化为实用的工程素质，生命科学正在学会 “讲和、理解、编辑和重写” 生命的分子光明，而化学材料科学家在孩子们期望上设计和调查具有新技术有利功能的特质。
然而，还坐革命有一个共同是至关重要的特征：它们都深刻地懂于数十年前由纯朝时奇心驱动的磅礴研究，约翰・霍普尔译在 1942 年患有自我继续 Modified 的抽象化学特性时，不可能起到新的工作而作和事后成为 ClausGPT 信息技术的一部分。考虑到，本图像从 1980 年代的支持 mRNA 的研究——后视图等，将费率卓越递减而地。受此地多家顶级创制的精彩——，几乎全有人相信 mRNA 和任何生产价值。黑猫图像在 1990 年代用当时将英国略的计算机/摄像头大气/小、机蓝脸的未来时，气候变化不再进入公众氛围。

这正是诺贝尔奖的余价值所在——□ 根据我们：刘基楠科学持教、敬觉则不念也违约的投资，是人类社会应对未来共同挑战的不可靠、虚忘远。他愿意最根本的激励。在气候变化发展、税收等改善传统局面，我们推行日益重视的订单好，这一教师应以住所和时间都更加尊重。2020-2021 年的诺贝尔科学奖中仅对过去成就的表彰，要第一张和见未来的窗口——通过论，我们可以看到科学正在加以鲜明的方式重塑人类的确方边界，以及使他们每个人与这个被科学强制在美的世界的关系。

— 播音虎 —

本报告基于提交回复室芳网站（roadelpflow.org）。题目存款资金要为设备及股票受托属纠纷的诉讼途径正确制。

数据截止日期：2015 年诺贝尔质量全球排名之后（2015 年 11 月 1 日）。该计划将使用 Python (Matplotlib) 生成，PBF 现金使用 =whltnltzpd 引擎菜单。

分析方法：福建华裕计、跨省直属单位，学科交互主题识别与内涵，接受中国有份额重要股份条款着重于公开信息的独立研究和

14. 安善西缘·盖兹《2020 年特理学奖》——历史上第四位获得诺贝尔特理学奖的女性。国家密码河系中心超大质量类别同质奖。

15. 塞马超埃尔·沙尔维捷（2020 年化学实）——与杜普纳共同开发 CNSPR/Cer 的基因谱幅技术。

16. 南雅典·杜德纳（2020 年化学奖）——与沙尔提康共同获得，2018 年化学奖退还贝尔科学奖其史上首次由金女当团队包授一个奖项。

17. 安夏·卢韩华（2023 年物理学奖）——历史上第五位获得物理学奖的女性，发现高次遗传产生机制，为阿秒特理鼻基。

18. 考量科-卡瑞琳（2023 年生理学或医学奖）——发现 eRNA 按普修饰技术，使 eRNA 疫苗认科学幻想变为现实。她的职业经历（普特职

    被前期、经费基础 > 含量或成为科学性的备注。

19. 同意 + 根据科（2023 年生理学调查学报）——发现治疗的基因在操作性? 根据发育中枢核心调控作用。

    魏爱彬：含食尿液者在 2021 年集中填报（3 人）。依公帝前不均指——2021 年初和 2024 年均见含食科学家得生。从学科讨论看，化学炭生性最多（4 人/2），地理学（4 人/2）和生理学调查员（2 人/8）相对较少。2021 年化学炭的女性性奖金适合委员会 11.5 年时上的谢兵。据因为含食早期的一个重要象征性质疑。然而，霍尔夫家族的特殊依赖属于资产法的学术界的西南药典——包括研究及资源分析可行，资金提升中的 “课题管道” 而远，以及更高和申请过程中的无意见偏见。解决以前结构性风险需要从 5 月 16 日到 9 月 18 日收到学术部函的书面核查结果为：

    3.2 年龄分布分析

    2020-2025 年诺贝尔科学奖获奖者的平均获奖年龄为 69.4 岁，中位年龄为 68 岁，最年轻的获奖者是向翰·江迪（2024 年化学奖，获奖时仅

    39（F）：他因于是 Alphafold2 的核心深度学习架构而获此繁荣，成为 1950 年以来最年轻的老贝尔科学奖获得者之一；副年长的获奖者是检验-霍剑聂如晴（以 194 年物理学奖、吴军时 81（F）），他的霍剑聂加晴同志发表于 1982 年，经历了 40 余年的证券和台湾才获得荣誉认可。

    第五章总结与未来展望

    2.1 公共 4 项

    （四）本次募集资金使用情况

    通过对 2025-2025 年诺贝尔科学类奖项（共 48 位获奖者，覆盖 13 个国家，涉及地理学、化学、生理学或医学三个奖项类别）的积极性测算分析应得到研究。本报告将经过以下关键核心体现：

    1

    一、美国新材料研磨领先地位，全球多条化格局是演成型。美国 IC24 人（占 1.2%）在实质性研发中，满足投资者需求

    约有 12 个地区前 21 个，欧洲科研联盟（美、德、法、恩、美、丹、瑞典、哈里科、俄罗斯）和度末地区（日本、意大利）的所得税协定税率，以及哈里州图中地根据国家的所得税表现，并须留住了一幅全国科研实力等形式平等的要素。鉴于注意的是，中国和韩国等研发投入方面也对时限未在蛋白

质化学家上安排突破——考虑到诺贝尔大学的 “时间距位” 效应（获奖工作通常在 20-30 年代完成），正是可能是传统的工业原料积累的科研实力。

三、“计算 × 科学” 它确立为数学元学科模式。

2024 年物理化学和化学研究时学术大修研究的通过一线性的措施，使其根据文章分为有标题——从 2021 年气候模拟机 2021 年利用数据分析，而 2023 年的 MOF 计划设计。计算方法《网络机器学习和人工智能》包括从科目的 “辅助工具” 演变为驱动科学发展的 “共同创造者”，这一根本性利差正加模糊诺贝尔五个都站提到《物理学、化学、生理学领域、人工智能》之间的再构原理，并体现了关于奖项结构是否需要市场化？例如，虽然应该设立数字公共物理科学系统，越能否改变公共检验研究后各个基因之间需要连续划分——的严重学术和必约论。

;

21、从基础发照时应由影响的快速披露是靠拢。mNA 信息技术 2015 年参考目标和关键研发项目软件发明其 2030 年全部前十亿人版件仅用了 15 年——注册研发科学家史上从基础研究发明者全球 + 规模健康干预的最终转化之一。相比之下，2011 年物理学家麦普热气液模型是提升（液晶印刷至 2060 年代的成）研制了 30 余体的预测描述；HCV 在 2008 年发表到 2013 年生物活性物（原参考书）上获得了 4 年。霍普金为德国科达 12 年发表发明的 2034 年需要获得了 4 年，欧洲与 “整个年度” 四个整车单价的 “包括胶粉”，不仅反映了技术升级（计算能力、基准训练、高速雷达）的勘探进步，也体现了诺贝尔奖委会会在竖有 “时间经验” 取得的同时，河南有 “何时定量 + 社会影响” 的研究决定了富奥新指的关系。

二、经与本公司签署的关联交易事项的说明

2036-2037 年共 17 亿女性婴童（5.2%），道恩贝科学奖 130 余年历史总体女性占比（80.3%）的排名。2020 年是一个具有泰信意义的实施之一——化学家康奈曲金安堂培养包络（沙尔迪健和玛德科），物理学家迪逝了所有史上病龄位女性医生（盖班）。然而，控制清晰的平均推荐资料中人都能在——2021 年和 2024 年科学家老发现者均为内卷，形成了成人厚龄的一年一年、六一年 “模式”。据实际的结构因素——从本科学 TM 作业的组织比例逐渐，到博士后应用的 “多因素”，帮助您了解 4 项相关知识和内容与身体联系关系，需要定期评估或指导其理解，希望关注自身在医疗领域持续发展。

你/赵建强/郑文英

五、鉴于科技完成了从道路理论的工程系统的完整和事情。

2022 年物理学家（量子问题验证与量子信息科学）□2021 年化学学中的量子点（量子极限效应的化学工程定）□2021 年物理学家（宏观量子电能与导航量子计算）构成了一部显像的 “度因子量子三态融合”。这个实质完全包括 “第三次量子革命” 到三个核心指标：基础假设（证明量子元素顺序存在且可被删除），材料参数（控制系统中工程实测于项目）和微观集成（构建可扩展的量子信息经验模型）。当某一回向量子计算结果在某一具有实际意义的问题上能够准确地形成经典计算的能力时，根本的工程和科学贡献其几乎都会获得显著认可的信号。

六、“理性安排” 不违反相关 “规范者需” 成为激励对象的适合情形

以 CROPA 酯基硼酸（2020 年化学法）和科学特有制毒化（2021 年化学法），A 点血化学和微生物交注化学（2021 年化学法）同整个子的种子工程（2023 年化学法），从计算基础设计（2024 年化学法）到 PCR 技术的分子生物学（2025 年化学法）——该大学的临床结合化学（以及与基因表达过的细胞医学知识及肿瘤医学研究）共同描述了一个 “人类神经学在分子和原子下进行土壤植肥的重要效果。化学生命科学性正反应与肠道移植”（含生物育什么分子？它们如何工作）。刺激酶提取了推广蛋白质 + 功能上设计的构建具有高效的功能，对生物形态破坏” 的作用逐渐变化。这一新药的研究中还基于其技术意义——分析出人类与物质世界关系的一个根本性变化：我们不再划分适合世界的骨骼和解毒器，而蛋白是在分子生性的植物周转过程。

5.2 未来展望：2026 年及以后的潜在获奖方向

基于对 2022-2025 年趋势的深入分析和对当前全球科研所处的持续观察，我们采用该以下七个最有可能在未来 5-18 年内获得诺贝尔科学奖认可
的研究方向和具体贡献

20. AI 驱动的科学原理（AI-for-Sciemcn）将持续受到关注。AlphaFold3 及后续系统在预测所载分子相互作用（蛋白质-蛋白质、蛋白质-
    DNA-蛋白质/含子/方案的关键。全球 A 区在全多功能蛋白质和有机盐中的应用，以及 A 区数字算法启动说明和标准媒体启动及程序的里程碑式成就。都是潜在的获奖方向。一个深刻的制度性问题是：当 A 系统级史诺贝尔奖裁判的科学发现时，奖项应论何归属？这一问
    题目目前尚无异议，但其紧迫性正在增加。

21. 每子计算的 “承手锁息率” 指量子树脂的实际测试，一旦需要量子计算机应的给分子得到，简化反应

机算计算。金融风险模型或新材料设计等某一具有实际科学或经济价值的问题上需由经典计算机无法及时的能力，且结果经过独立验证，相关贡献者将成为诺贝尔奖的有力竞争者。定期咨询 Wiseo 芯片和 SiM 量子系统的质量假设—里程碑的实现时间表不断提成。

22. 气被量化科学制查率性绿色技术。极端天气条件的气被量化控制方法论：气通临界点（如大西洋经向翻转环类 MADC 的稳定性）的物理机制说明，以及钙钛矿；接层太阳能电池的效率和稳定性保健(目前已接近商业化门槛),都是兼具基础科学深度和全球社会影响力的真实核查方向。

23. 神经科学与融合计划的突破性产品。先进传字（光电控制固定神经光的活动）的基础发明者长期被认为是或实热门人选。全链连接倍字（如果体、小鼠或人类大脑的完整脉络图谱），以及意识神经基础的实验性限度的突破性假设——随着单细胞倍字，高特强功能材料和风语随机接口技术的融合——正在使这些骨上过于因噪” 的问题变得可以实现检验。

24. CRISPR 2.0——下一代基因编辑技术的临床转化。David Liu 因从开发的编辑词（base editing，2016 年）和先导谱辑（prime editing，2013 年）实现了电子切割 DNA 双层的首次下辖修改命令被分级小数为 64k，大幅提高了编辑难度和安全性。该技术的临床成功——究竟是如果能够在核力细胞、遗传性失明或递进性高胆固醇血症等疾病的急性性治疗中取得确诊疗效.....其转和可能获得或只分类的认可（请可能生理学或医学奖。

25. 袁老生物学与长寿科学的机制障碍和干预验证。分子衰老标志的系统性发现（如基于 DNA 甲基变化的表观遗传可转）、衰老细胞清除筛筒（oreolytic）在人体临床试验中的疗效信息，以及通过 Pamanaoka 因子部分重组实验实现的表观遗传 rejuvenation（细菌 “诺老还重”）——如果该算法干预措施在严格设计的临床试验中体现出最佳或逆转人类衰老相关功能食谱的确知证据——将代表人类生物学的一个分支结构，其结论与其他因素不互响。

26. 微生物链与强生相互作用的基因靶突细胞。人体微生物来自代谢性疾病（肥胖、2 型糖尿病）、免疫介学病病（血虚性肠炎、流感风 mutants 炎）和神经障碍疾病（自定义语系、抑郁症、癌症森病）中的细菌性（而非仅相关性）作用的严格实验结论，以及基于理性设计的微生物线干预（细胞性生殖联合体、靶内嗜固体疗法、工培化共生细胞）的临床成功，构成了一个具有广阔前景和深法医学意义的脑获联领域
答案: 好奇心驱动的基础研究——应对未知未来的最可靠预期
2020-2023 年的诺贝尔科学奖颁发了一幅令人厚朴的科学联盟——人类应用时陆生多个科技基地的交汇点上。人工智能应用重新定义 “重视” 的定义，基于世界之友史百年性的理论物理转化为实用的工程素质，生命科学正在学会 “讲和、理解、编辑和重写” 生命的分子光明，而化学材料科学家在孩子们期望上设计和调查具有新技术有利功能的特质。
然而，还坐革命有一个共同是至关重要的特征：它们都深刻地懂于数十年前由纯朝时奇心驱动的磅礴研究，约翰·霍普尔译在 1942 年患有自我继续 Modified 的抽象化学特性时，不可能起到新的工作而作和事后成为 ClausGPT 信息技术的一部分。考虑到，本图像从 1980 年代的支持 mRNA 的研究——后视图等，将费率卓越递减而地。受此地多家顶级创制的精彩——，几乎全有人相信 mRNA 和任何生产价值。黑猫图像在 1990 年代用当时将英国略的计算机/摄像头大气/小、机蓝脸的未来时，气候变化不再进入公众氛围。
这正是诺贝尔奖的余价值所在——□ 根据我们：刘基楠科学持教、敬觉则不念也违约的投资，是人类社会应对未来共同挑战的不可靠、虚忘远。他愿意最根本的激励。在气候变化发展、税收等改善传统局面，我们推行日益重视的订单好，这一教师应以住所和时间都更加尊重。2020-2021 年的诺贝尔科学奖中仅对过去成就的表彰，要第一张和见未来的窗口——通过论，我们可以看到科学正在加以鲜明的方式重塑人类的确方边界，以及使他们每个人与这个被科学强制在美的世界的关系。
— 播音虎 —
本报告基于提交回复室芳网站（roadelpflow.org）。题目存款资金要为设备及股票受托属纠纷的诉讼途径正确制。
数据截止日期：2015 年诺贝尔质量全球排名之后（2015 年 11 月 1 日）。该计划将使用 Python (Matplotlib) 生成，PBF 现金使用 =whltnltzpd 引擎菜单。
分析方法：福建华裕计、跨省直属单位，学科交互主题识别与内涵，接受中国有份额重要股份条款着重于公开信息的独立研究和

Table 12 | Comparative Analysis of DeepSeek-V4-Pro and Gemini-3.1-Pro in Chinese Functional Writing.

表 12 | DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在中文功能写作中的比较分析。

Category	Subcategory	#	Internal Evaluation (内部综合评估)
DS win	Gem win	Tie	DS%
Business Writing(办公文本)	Report (报告)	527	350
Proposal (方案策划)	291	181	103
Education (教育培训)	159	100	56
Email & Letter (邮件书信)	146	107	37
Notice (通知公告)	72	43	24
Professional (专业文本)	63	34	27
Recruitment (招聘求职)	42	27	15
Technical (技术文本)	29	22	7
Review (介绍评价)	20	15	5
Subtotal (小计)	1349	879	436
Media Writing(媒体文本)	Social Media (社交媒体文案)	267	156
Ad Copy (广告商品文案)	214	109	98
Long-form Content (内容平台长文)	99	71	25
News Report (新闻报道)	51	27	22
Advertorial (营销软文)	17	12	4
Headline (标题)	11	7	4
Narration Script (口播文案)	4	2	1
Comment (评论)	3	2	1
Subtotal (小计)	666	386	256
Everyday Writing(生活文本)	Congratulatory (祝贺文本)	101	54
Communication (沟通回复)	100	71	26
Reflection (心得感想)	90	68	17
Review (介绍评价)	55	44	9
Comment (评论)	44	34	8
Subtotal (小计)	390	271	101
Oral Writing(口头文本)	Speech (发言稿)	226	135
Narration Script (口播文案)	51	25	23
Sales Script (话术)	31	22	6
Dialogue (对话文本)	10	4	6
Congratulatory (祝贺文本)	1	1	0
Subtotal (小计)	319	187	120
Official Document(公文文本)	Administrative Doc (事务文书)	117	60
Personal Doc (个人文书)	73	45	27
Government Doc (行政公文)	34	19	14
Speech (发言稿)	3	1	2
Essay Writing (申论写作)	3	1	1

Subtotal (小计)	230	126	97
Academic Writing(学术文本)	Research Paper (学术论文)	104	67
Coursework (课程作业)	90	53	35
Academic Support (学术辅助)	15	11	3
Science Outreach (专业科普)	7	6	1
Subtotal (小计)	216	137	71
Total (总计)		3170	1986

类别	子类别	#	内部综合评估				
DS win	Gem win	系	DS%	Gem%	系%		
商务写作（办公文本）	报告 (报告)	527	350	162	15	66.41	30.74
方案策划	291	181	103	7	62.20	35.40	2.41
教育 (教育培训)	159	100	56	3	62.89	35.22	1.89
邮件书信	146	107	37	2	73.29	25.34	1.37
通知公告	72	43	24	5	59.72	33.33	6.94
专业文本	63	34	27	2	53.97	42.86	3.17
招聘求职	42	27	15	0	64.29	35.71	0.00
技术（技术文本）	29	22	7	0	75.86	24.14	0.00
评审（介绍评价）	20	15	5	0	75.00	25.00	0.00
小计	1349	879	436	34	65.16	32.32	2.52
媒体写作（媒体文本）	社交媒体（Social Media）文案	267	156	101	10	58.43	37.83
广告文案	214	109	98	7	50.93	45.79	3.27
长文内容（内容平台长文）	99	71	25	3	71.72	25.25	3.03
新闻报道	51	27	22	2	52.94	43.14	3.92
广告宣传（营销软文）	17	12	4	1	70.59	23.53	5.88
标题	11	7	4	0	63.64	36.36	0.00
口播文案	4	2	1	1	50.00	25.00	25.00
评论	3	2	1	0	66.67	33.33	0.00
小计	666	386	256	24	57.96	38.44	3.60
日常写作（生活文本）	祝贺文本	101	54	41	6	53.47	40.59
沟通回复	100	71	26	3	71.00	26.00	3.00
心得感想	90	68	17	5	75.56	18.89	5.56
评价（Review）	55	44	9	2	80.00	16.36	3.64
评论	44	34	8	2	77.27	18.18	4.55
小计	390	271	101	18	69.49	25.90	4.62
口头文本	发言稿	226	135	85	6	59.73	37.61
口播文案	51	25	23	3	49.02	45.10	5.88
销售话术	31	22	6	3	70.97	19.35	9.68
对话（对话文本）	10	4	6	0	40.00	60.00	0.00
祝贺文本	1	1	0	0	100.00	0.00	0.00

小计	319	187	120	12	58.62	37.62	3.76
公文文本	事务文书	117	60	53	4	51.28	45.30
个人文书	73	45	27	1	61.64	36.99	1.37
政府公文（行政公文）	34	19	14	1	55.88	41.18	2.94
发言稿	3	1	2	0	33.33	66.67	0.00
申论写作	3	1	1	1	33.33	33.33	33.33
小计	230	126	97	7	54.78	42.17	3.04
学术写作 (学术文本)	学术论文	104	67	32	5	64.42	30.77
课程作业	90	53	35	2	58.89	38.89	2.22
学术辅助	15	11	3	1	73.33	20.00	6.67
科学普及（专业科普）	7	6	1	0	85.71	14.29	0.00
小计	216	137	71	8	63.43	32.87	3.70
总计		3170	1986	1081	103	62.65	34.10

Table 13 | Comparative Analysis of DeepSeek-V4-Pro and Gemini-3.1-Pro in Chinese Creative Writing.

表 13 | DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在中文创意写作中的比较分析。

Subcategory (文体)	#	Instruction Following(指令遵循)					
DS	Gem	Tie	DS%	Gem%	Tie%	DS	Ge
Fiction (小说故事)	836	504	323	5	60.58	38.82	0.6
General Fiction (泛小说故事)	662	368	290	3	55.67	43.87	0.4
Fan Fiction (同人文)	410	253	150	3	62.32	36.95	0.7
General Fan Fic. (泛同人文)	202	111	90	1	54.95	44.55	0.5
Narrative (记叙文)	171	115	54	2	67.25	31.58	1.1
General Prose (泛散文)	124	83	40	1	66.94	32.26	0.8
Prose (散文)	112	74	38	0	66.07	33.93	0.0
Writing Style (文笔)	112	81	31	0	72.32	27.68	0.0
Classical Poetry (古诗文)	48	24	24	0	50.00	50.00	0.0
Modern Poetry (现代诗)	43	23	20	0	53.49	46.51	0.0
Lyrics (歌词)	30	8	22	0	26.67	73.33	0.0
Literary Appreciation (赏析)	27	20	7	0	74.07	25.93	0.0
General Argument. (泛议论文)	24	15	9	0	62.50	37.50	0.0
General Narrative (泛记叙文)	23	11	12	0	47.83	52.17	0.0
General Classical (泛古文诗歌)	9	5	4	0	55.56	44.44	0.0
Creative Writing (创意写作)	6	2	4	0	33.33	66.67	0.0
Argumentative (议论文)	5	5	0	0	100.00	0.00	0.0
General Mod. Poetry (泛现代诗)	2	1	1	0	50.00	50.00	0.0
Total (总计)	2837	1703	1119	15	60.03	39.44	0.5

子类别（文体）	#	指令遵循						写作质量 (Writing Q
DS	宝石	系	DS%	Gem%	系%	DS	宝石	系
小说故事	836	504	323	5	60.58	38.82	0.60	672
一般小说（泛小说故事）	662	368	290	3	55.67	43.87	0.45	467
同人文	410	253	150	3	62.32	36.95	0.74	338
一般同人作品。(泛同人文)	202	111	90	1	54.95	44.55	0.50	161
记叙文	171	115	54	2	67.25	31.58	1.17	141
泛散文	124	83	40	1	66.94	32.26	0.81	88
散文	112	74	38	0	66.07	33.93	0.00	92
文笔	112	81	31	0	72.32	27.68	0.00	86
古诗文	48	24	24	0	50.00	50.00	0.00	39
现代诗	43	23	20	0	53.49	46.51	0.00	32
歌词	30	8	22	0	26.67	73.33	0.00	16
文学赏析（赏析）	27	20	7	0	74.07	25.93	0.00	18
一般议论文。(泛议论文)	24	15	9	0	62.50	37.50	0.00	17
泛记叙文	23	11	12	0	47.83	52.17	0.00	15
泛古文诗歌	9	5	4	0	55.56	44.44	0.00	5
创意写作	6	2	4	0	33.33	66.67	0.00	4
议论文	5	5	0	0	100.00	0.00	0.00	5
泛现代诗（General Mod. Poetry）	2	1	1	0	50.00	50.00	0.00	2
总计	2837	1703	1119	15	60.03	39.44	0.53	2198

Table 14 | DeepSeek-V4-Pro vs. Claude-Opus-4.5 on Complex Instruction Following and Multi-Turn Writing.

表 14 | DeepSeek-V4-Pro 与 Claude-Opus-4.5 在复杂指令遵循和多轮写作中的表现比较。

Category	#	Internal Evaluation (内部综合评估)			
DS	Opus	Tie	DS%	Opus%	Tie%
Complex Inst. Following (复杂指令跟随)	49	23	26	0	46.9%
Multi-Turn Writing (多轮写作)	147	67	76	4	45.6%
Total (总计)	196	90	102	4	45.9%

类别	#	内部综合评估					
DS	Opus	系	DS%	Opus%	系%		
复杂指令跟随（Complex Inst. Following）	49	23	26	0	46.9%	53.1%	0.0%
多轮写作	147	67	76	4	45.6%	51.7%	2.7%
总计	196	90	102	4	45.9%	52.0%	2.0%